UNIVERSIDAD AUTÓNOMA GABRIEL RENÉ MORENO

FACULTAD DE INGENIERÍA EN CIENCIAS DE LA COMPUTACIÓN Y TELECOMUNICACIONES

SISTEMA DE INFORMACIÓN WEB PARA EL E-COMMERCE DE ALTA COSTURA «AURA» CON PROBADOR VIRTUAL Y ANALÍTICA ASISTIDA POR VOZ CON INTELIGENCIA ARTIFICIAL

Docente: Rolando Antonio Martinez Canedo

Grupo: 11

Integrantes:

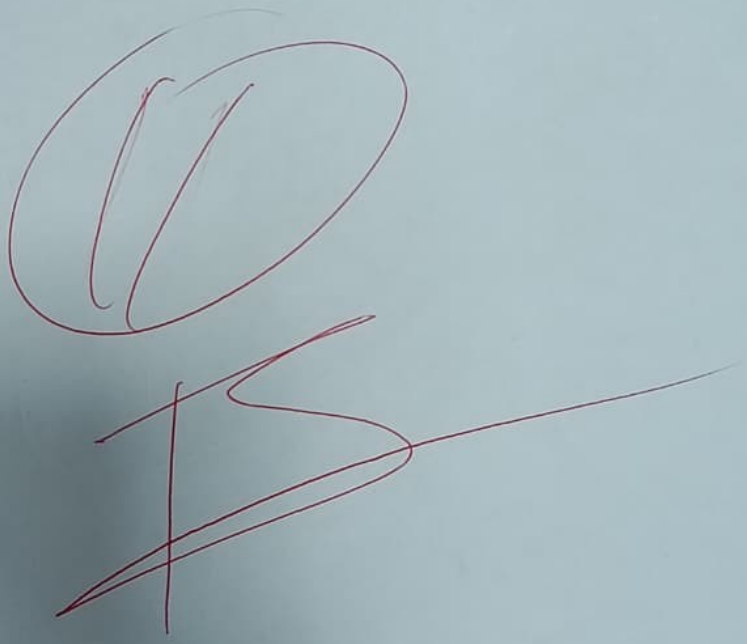
Astete Paz Diego Andres	221043748
Cruz Huarachi Denilson Ariel	221180095

1er examen parcial

Sistemas de Informacion II

Astete Paz Diego Andres

Cruz Huarachi Denilson Ariel



Indice

1. PERFIL....8
1.1. INTRODUCCION....8
1.2. ANTECEDENTE....9
1.3. JUSTIFICACION....10
1.4. DESCRIPCION DEL PROBLEMA....11
1.5. FORMULACION DEL PROBLEMA....12
1.6. OBJETIVOS....12
1.6.1. Objetivo General....12
1.6.2. Objetivo Específicos....12
1.7. ALCANCE....13
1.7.1. Módulo 1: Gestión de Usuarios, Clientes y Roles....13
1.7.2. Módulo 2: Catálogo de Productos y Clasificación Jerárquica....14
1.7.3. Módulo 3: Gestión de Variantes e Inventario Granular....14
1.7.4. Módulo 4: Probador Virtual Inteligente 3D y Realidad Aumentada....14
1.7.5. Módulo 5: Carrito Híbrido Persistente y Lista de Deseos....15
1.7.6. Módulo 6: Ventas, Checkout, Pagos con Stripe y Reembolsos....15
1.7.7. Módulo 7: Logística de Envíos, Cupones y Analítica por Voz con IA....15
1.8. ENTREVISTA....16
2. ELEMENTOS DEL SISTEMA BASADO EN COMPUTADORAS....19
2.1. HARDWARE....19
2.1.1. Servidor....19
2.1.2. Cliente....19
2.1.3. Medios de Comunicación....19
2.1.4. Otros Dispositivos....19
2.2. SOFTWARE....20

2.2.1. Servidor.....20
2.2.2. Cliente.....20
2.2.3. Otro software adicional.....20
2.3. DATOS.....20
2.4. PROCESOS.....20
2.5. GENTE/USUARIO.....20
2.6. DOCUMENTO.....21
2.6.1. Mecanismo de Aprendizaje e Inducción de Usuario (Operación sin Inconvenientes).....21
3. TECNOLOGIA PARA EL DESARROLLO DEL SOFTWARE.....24
3.1. Estrategia para el desarrollo del software.....24
3.2. Metodología para el desarrollo del software.....24
3.2.1. Características del PUDS.....25
3.2.2. Características de UML.....25
3.2.3. UML – Diagramas a utilizar.....26
3.3. HERRAMIENTAS DE DESARROLLO.....26
3.3.1. Software.....26
3.3.2. Hardware.....27
4. POSIBLES COSTOS.....27
5. POSIBLES BENEFICIOS.....28
5.1. TIEMPO.....28
5.2. ESFUERZO.....28
5.3. COSTOS.....28
6. POSIBLES CLIENTES.....28
7. MODELO DE NEGOCIO.....28
8. Marco Teórico.....32
CAPITULO 1: FLUJO DE TRABAJO: CAPTURA DE REQUISITOS.....34

1.1. Identificar actores y casos de usos....34
1.2. Priorizar casos de uso....34
1.3. Detallar Casos de Uso....39
1.5. Prototipar Interfaz De Usuario....57
CAPITULO 2: FLUJO DE TRABAJO: ANALISIS....61
2.1. Análisis de Arquitectura....61
2.1.1. Identificar paquetes....61
2.1.2. Relacionar paquetes y casos de uso....61
2.1.3. Vista de casos de uso (encapsulamiento)....62
2.2. Análisis de caso de uso Tablas resumen de los ciclos....62
2.3. Ciclo #1....62
2.4. Ciclo #2....62
2.5. Ciclo #3....62
2.6. Ciclo #4....63
2.6.1. Diagramas de comunicación....63
2.7. Análisis de clase....72
2.8. Análisis de paquete....80
CAPITULO 3: FLUJO DE TRABAJO: DISEÑO....81
3.1. Diseño de Arquitectura....81
3.2. Diagrama de despliegue....81
3.3. Diseño de datos....82
3.3.1. Diseño de datos lógico....82
3.3.1.1. Diagrama de Clase....82
3.3.1.2. Mapeo....82
3.3.2. Diseño de datos físico....84
3.3.2.1. Tabla de volumen....84

3.3.2.2. Script....84
3.3.2.3. Diagrama Relacional....88
3.3.2.4. Actualización de Tuplas....89
1.1.1.1.2 Update()....90
1.1.1.1.3 Delete()....90
3.3.2.5. Consultas....90
3.4. Diseñar caso de uso....93
CAPITULO 4: FLUJO DE TRABAJO: IMPLEMENTACION....105
4.1. Elección de plataforma de desarrollo del software....105
4.1.1. Framework (NestJS + Vite React)....105
4.1.2. Lenguaje de programación (TypeScript / Node.js)....105
4.1.3. Base de datos (PostgreSQL 16)....105
4.1.4. Sistema operativo (Linux / Docker)....105
4.1.5. Otros (VS Code, Git + GitHub)....105
4.2. Implementación de la arquitectura del sistema (diagrama general)....105
4.3. Subsistema de Gestión de Usuarios y Autenticación....106
4.4. Subsistema de Catálogo y Variantes....106
4.5. Subsistema de Probador Virtual 3D y Realidad Aumentada....106
4.6. Subsistema de Ventas, Checkout y Facturación Stripe....106
CAPITULO 5: FLUJO DE TRABAJO: PRUEBAS....107
5.1. Planificar Pruebas....107
5.1.1. Objetivo General....107
5.2. Alcance de las Pruebas....107
5.3. Tipos de Pruebas....107
5.4. Caso de Pruebas (Implementar pruebas)....107
5.4.1. CASO DE PRUEBA 01: Compra y Pago Electrónico con Tarjeta Stripe en AURA....107

5.4.2. CASO DE PRUEBA 02: Simulación y Superposición en Probador Virtual.....108

6. CONCLUSION.....110

7. RECOMENDACIÓN.....110

8. BIBLIOGRAFIA.....111

9. Anexos y QR.....113

1. PERFIL

1.1. INTRODUCCION

En la economía global contemporánea, el comercio electrónico ha dejado de ser un canal accesorio para convertirse en el epicentro operativo del sector retail y la industria de la moda. En el segmento de la alta costura y confección textil femenina (Haute Couture & Prêt-à-Porter), el mercado experimenta una transformación vertiginosa impulsada por la aceleración digital y la exigencia de los consumidores de acceder a experiencias de compra personalizadas, fluidas y visualmente enriquecidas. Las organizaciones que lideran este rubro asumen la responsabilidad de diseñar, presentar y comercializar prendas exclusivas mediante plataformas tecnológicas capaces de garantizar disponibilidad inmediata de catálogo, fidelidad estética en el color y corte textil, transaccionalidad bancaria transparente y asistencia técnica constante al comprador.

Sin embargo, el comercio textil digital enfrenta una fricción estructural crítica: la distancia física entre el cliente y la prenda. A diferencia de las tiendas tradicionales donde el usuario puede palpar la textura y probarse la ropa frente al espejo, las tiendas en línea estándar dependen de galerías fotográficas estáticas que no transmiten con exactitud la caída de las telas ni las proporciones sobre cuerpos diversos. Esta limitación provoca elevados índices de incertidumbre en el cliente, desembocando en tasas de devolución que promedian entre el 25% y el 40% a nivel global, con severas pérdidas logísticas y financieras para los comercios.

• La Empresa: AURA Atelier & E-Commerce y el Enfoque del Proyecto

Inmersa directamente en este exigente mercado opera «AURA», una firma boutique de diseño y confección de alta moda que ofrece colecciones exclusivas de vestidos, faldas, blusas de seda, conjuntos y prendas sastre. Como marca en expansión, AURA busca llevar su atelier físico hacia un ecosistema web transaccional de alcance nacional e internacional, compitiendo con estándares globales de calidad estética y tecnológica.

A pesar de su prestigio en confección y acabados, AURA enfrenta una barrera operativa sustancial: la falta de una plataforma integrada que unifique el catálogo interactivo con un probador virtual inteligente y un control granular de inventario por variantes atómicas (combinación de producto, talla, color y código SKU único). Actualmente, la gestión se realiza de forma artesanal y dispersa, obligando al personal de ventas a responder consultas manuales por mensajería, verificar existencias físicas en percheros y lidiar con reclamos de clientes por prendas que no calzaron según lo esperado.

• ¿Qué es lo que haremos?

Para solucionar de raíz esta problemática, el presente proyecto de Sistemas de Información 2 consiste en el desarrollo integral de una plataforma web de comercio electrónico de última generación denominada «AURA». Esta solución incorpora un probador virtual interactivo multitecnología que combina renderizado tridimensional 360° en WebGL con Three.js, superposición en vivo por cámara web con Inteligencia Artificial corporal mediante Google MediaPipe Pose, y modelos de difusión fotorrealista. Asimismo, integra un motor de persistencia híbrida para carritos de compra, pasarela internacional de pagos Stripe con transacciones ACID, y un innovador asistente analítico por voz con procesamiento de lenguaje natural (Text-to-SQL) para la toma de decisiones directivas.

1.2. ANTECEDENTE

AURA nació hace 4 años en la ciudad de Santa Cruz de la Sierra con la visión de fusionar la elegancia de la sastrería clásica con las tendencias vanguardistas de la moda contemporánea. Durante su trayectoria, el atelier ha consolidado una cartera selecta de clientas recurrentes, participando en desfiles de moda y comercializando prendas de lino puro, seda y satén.

Bajo la dirección de su Directora Creativa y Gerente General, la empresa ha forjado una identidad corporativa sustentada en la excelencia textil y la atención al detalle. Su estructura estratégica se define mediante los siguientes pilares fundamentales:

- **Propósito y Promesa de Marca:** «El arte del buen vestir al alcance de tu silueta». La empresa se compromete a que cada prenda realce la personalidad del cliente mediante cortes impecables y asesoramiento personalizado.
- **Visión Estratégica:** Posicionarse en los próximos 5 años como la plataforma líder de moda digital y alta costura en Bolivia y la región andina, reconocida internacionalmente por integrar probadores virtuales de inteligencia artificial que eliminen la fricción en la compra textil.
- **Valores Organizacionales:** Sofisticación estética, innovación tecnológica constante, honestidad transaccional y sostenibilidad en la producción textil.

1.3. JUSTIFICACION

La implementación de este sistema de información web para AURA se justifica por la imperiosa necesidad de modernizar la experiencia del consumidor y automatizar el control operativo de una empresa de moda en pleno crecimiento:

- **1. Precisión en la Asignación y Control de Stock por Variantes:** Al gestionar las prendas a través de una matriz multidimensional de SKUs independientes (producto + talla + color), se erradica la sobreventa de prendas agotadas y se garantiza que el cliente adquiera únicamente prendas con disponibilidad real en almacén.
- **2. Reducción Drástica de Devoluciones y Fricción de Compra:** Al dotar al cliente de un probador 3D orbital y un probador en tiempo real por realidad aumentada con cámara web (MediaPipe Pose), el usuario visualiza proporciones y calce antes de pagar, reduciendo las devoluciones y los sobrecostos de flete inverso.
- **3. Seguridad Financiera y Cumplimiento Normativo:** La incorporación de Stripe Elements con certificación PCI-DSS Nivel 1 asegura que los datos de las tarjetas nunca toquen los

servidores del comercio, blindando las transacciones contra fraudes y brindando confianza absoluta al comprador.

- **4. Escalabilidad y Profesionalismo Corporativo:** La arquitectura Monorepo desacoplada (NestJS modular en backend y React SPA en frontend) otorga al sistema la solidez necesaria para escalar a miles de usuarios concurrentes sin degradación de servicio.

1.4. DESCRIPCION DEL PROBLEMA

El problema central identificado en AURA radica en la ineficiencia operativa y la incertidumbre del comprador asociadas a la comercialización tradicional de prendas de moda. Los puntos neurálgicos identificados son los siguientes:

- **1) Ambigüedad en la Selección de Tallas y Calce de la Prenda:** Los clientes manifiestan dudas constantes sobre el largo, holgura y caída de los vestidos, postergando la decisión de compra o solicitando múltiples cambios de talla tras la entrega del pedido.
- **2) Falta de Trazabilidad en Existencias por Talla y Color:** El stock se controla de manera global por modelo y no por variantes específicas. Esto genera que se confirmen compras de tallas o colores agotados físicamente, derivando en quejas y devoluciones forzosas.
- **3) Conciliación Manual de Pagos y Fricción en el Checkout:** La recepción de comprobantes de pago por transferencias manuales requiere una verificación humana lenta y propensa a errores, retrasando el despacho de las prendas.
- **4) Incapacidad de Alerta Temprana ante Quiebres de Stock:** Al no contar con alertas automáticas de inventario crítico, la administración solo advierte el agotamiento de una tela o prenda cuando ya se ha perdido la oportunidad de venta.

- **5) Descentralización de la Información Analítica:** Los datos de ventas, clientes, cupones y devoluciones se encuentran fragmentados, impidiendo que la gerencia obtenga métricas consolidadas o respuestas inmediatas a consultas comerciales de negocio.

1.5. FORMULACION DEL PROBLEMA

Dada la situación actual de AURA, donde los procesos de comercialización, verificación de calce textil y administración de stock se realizan de manera manual y fragmentada, surge la siguiente interrogante como eje rector de la investigación:

¿De qué manera el análisis, diseño e implementación de un sistema de información web integral bajo la metodología PUDS, que incorpore un probador virtual interactivo 3D/AR, un catálogo facetado por variantes atómicas de SKU, procesamiento seguro de pagos con Stripe y analítica de datos asistida por voz con Inteligencia Artificial, permitirá optimizar la conversión de ventas, minimizar la tasa de devolución textil y garantizar el control transaccional del inventario de AURA?

1.6. OBJETIVOS

1.6.1. Objetivo General

Desarrollar e implementar un sistema de información web integral para la gestión comercial y experiencia inmersiva de compra en la empresa AURA, aplicando la metodología PUDS y el lenguaje UML, que centralice el catálogo de prendas de alta costura con inventario por variantes, integre un probador virtual 3D y de realidad aumentada, procese pagos electrónicos seguros y provea reportes dinámicos asistidos por voz e inteligencia artificial.

1.6.2. Objetivo Específicos

- • **Levantamiento y Análisis de Requerimientos:** Realizar un diagnóstico exhaustivo de los procesos de venta, peritaje de calce de prendas y logística de AURA, especificando 25 Casos de Uso formales bajo el marco metodológico PUDS.

- • **Diseño de la Arquitectura de Datos:** Estructurar una base de datos relacional normalizada en Tercera Forma Normal (3FN) sobre PostgreSQL 16 que gestione variantes atómicas (producto, talla, color, SKU), pedidos, pagos y auditoría inalterable.
- • **Desarrollo del Módulo de Catálogo y Variantes:** Desarrollar un catálogo público responsivo con filtros facetados por categoría, rango de precios y disponibilidad, con carga de imágenes optimizadas en formato WebP.
- • **Implementación del Probador Virtual 3D y AR:** Construir un probador virtual dual que soporte inspección tridimensional 360° en formato GLB mediante Three.js y proyección en vivo por cámara web con detección corporal a 60 fps mediante Google MediaPipe Pose.
- • **Integración de Pasarela de Pagos y Ventas:** Integrar la pasarela de pagos Stripe mediante componentes seguros (PaymentIntents) y webhooks criptográficos, asegurando transaccionalidad ACID en el descuento de inventario.
- • **Implementación de Analítica Asistida por Voz:** Programar un módulo de analítica directiva con procesamiento de lenguaje natural (NLP Text-to-SQL) y reconocimiento de voz (Web Speech API) que interprete consultas habladas de los directivos.
- • **Validación y Pruebas del Sistema:** Ejecutar pruebas funcionales, de integración, seguridad y concurrencia con datos reales de prendas para validar la robustez de la plataforma antes de su despliegue operativo.

1.7. ALCANCE

El proyecto comprende la sistematización y reingeniería digital de la operativa completa de AURA, estructurándose en 7 módulos funcionales:

1.7.1. Módulo 1: Gestión de Usuarios, Clientes y Roles

- Registro y administración de usuarios del sistema bajo modelo RBAC.

- Control de acceso seguro mediante JSON Web Tokens (JWT) y Refresh Tokens cifrados.
- Registro inalterable de accesos y operaciones críticas en la bitácora del sistema.
- Gestión de libreta de direcciones de envío con selección predeterminada.

1.7.2. Módulo 2: Catálogo de Productos y Clasificación Jerárquica

- Alta, edición, activación e inactivación de prendas de vestir de alta costura.
- Gestión jerárquica de categorías padre y subcategorías hijas con slugs amigables.
- Galería múltiple de imágenes de alta resolución con designación de foto de portada.

1.7.3. Módulo 3: Gestión de Variantes e Inventario Granular

- Parametrización atómica de variantes por combinación de Talla (XS..XL) y Color.
- Generación y control de unicidad de código SKU (Stock Keeping Unit).
- Registro de stock físico, stock mínimo de seguridad y precios adicionales por variante.
- Alertas automáticas visuales ante agotamiento inminente de existencias.

1.7.4. Módulo 4: Probador Virtual Inteligente 3D y Realidad Aumentada

- Visualizador 3D interactivo con Three.js para inspeccionar prendas en 360°, zoom e iluminación.
- Generación de geometrías GLB 2.0 asistida por clusters neuronales de Hugging Face.

- Probador AR en tiempo real mediante webcam y Google MediaPipe Pose con tracking de 33 puntos corporales.
- Simulación fotorrealista sobre fotografía corporal del usuario.

1.7.5. Módulo 5: Carrito Híbrido Persistente y Lista de Deseos

- Bolsa de compras interactiva slide-over con actualización en tiempo real de unidades.
- Persistencia híbrida sincronizada: almacenamiento local anónimo y fusión con BD al autenticarse.
- Marcado y administración de prendas favoritas (Wishlist) con traslado directo al carrito.

1.7.6. Módulo 6: Ventas, Checkout, Pagos con Stripe y Reembolsos

- Proceso de checkout guiado con selección de dirección y cotización de flete.
- Procesamiento seguro de pagos electrónicos con tarjetas de crédito/débito vía Stripe.
- Registro transaccional de órdenes (nota_venta) con descuento automático de inventario.
- Gestión de cancelaciones y procesamiento de reembolsos con restitución de stock.

1.7.7. Módulo 7: Logística de Envíos, Cupones y Analítica por Voz con IA

- Parametrización de zonas y métodos de envío urbanos y nacionales con plazos estimados.
- Asignación y seguimiento de número de guía de transporte en tiempo real.
- Creación y canje de cupones de descuento porcentuales y por monto fijo.

- Publicación de reseñas y calificaciones de 1 a 5 estrellas exclusivas para compras verificadas.
- Motor de reportes dinámicos por voz con IA para formular consultas SQL analíticas dictadas por micrófono.

1.8. ENTREVISTA

• Transcripción Institucional:

Entrevistador: Pregunta: Para definir el alcance del nuevo desarrollo, ¿cuál es el problema central o el mayor obstáculo operativo que enfrenta AURA actualmente en la comercialización de sus prendas?

Directora General: Respuesta: Nuestro mayor dolor de cabeza es la incertidumbre de las clientas respecto a las tallas y la caída de la ropa. Vendemos prendas de alta costura con telas nobles como lino y seda, pero en una foto plana la clienta no aprecia cómo le lucirá el vestido en su cuerpo. Esto nos genera muchas dudas por WhatsApp, pérdidas de ventas y una tasa de cambios de prenda que ronda el 30%, lo que nos encarece la logística y desorganiza nuestro inventario.

Entrevistador: Pregunta: Bajo estas limitaciones, ¿cómo controlan actualmente la disponibilidad de prendas y las variantes de color y talla?

Directora General: Respuesta: Lo manejamos manualmente mediante hojas de Excel y consultas directas al perchero. Es un proceso tedioso y propenso a errores: más de una vez nos ha pasado que confirmamos por mensaje la venta de un vestido verde en talla M cuando en realidad solo nos quedaba en talla S, teniendo que llamar a la clienta para disculparnos o cancelar el pedido.

Entrevistador: Pregunta: Considerando este desgaste, ¿cuáles son las funciones críticas que el nuevo sistema web debe cumplir obligatoriamente para considerar exitosa la solución?

Directora General: Respuesta: El sistema debe cumplir con tres innovaciones fundamentales: 1) Un probador virtual donde la clienta pueda ver la prenda en 3D y probarse la ropa frente a su cámara web; 2) Un control exacto de stock donde cada combinación de talla y color tenga su propio inventario y se descuente al pagar; y 3) Una pasarela de pago segura con tarjeta y un panel donde podamos consultar las ventas y pedirle reportes al sistema por voz sin complicarnos con filtros difíciles.

• Análisis y Deducción de Requerimientos a partir de la Entrevista:

- **RF01 - Gestión Granular de Variantes:** El sistema debe modelar de forma independiente cada combinación de producto, talla y color con código SKU.
- **RF02 - Probador Virtual Interactivo:** La plataforma debe incorporar herramientas interactivas de visualización 3D y realidad aumentada corporal para previsualizar prendas.
- **RF03 - Pagos Seguros en Línea:** El proceso de cobro con tarjeta debe realizarse a través de una pasarela certificada con descuento atómico de existencias.
- **RF04 - Analítica Asistida por Voz:** La administración debe contar con un módulo de inteligencia artificial capaz de recibir comandos de voz y generar métricas analíticas.

PERFIL DE PROYECTO — Ficha Técnica de Levantamiento de Requisitos:

Entrevista para obtención de requisitos del sistema AURA

Objetivo: Diagnosticar la problemática de venta, inventario y experiencia de calce textil

Entrevista # 1

Lugar: Showroom AURA Atelier, Equipetrol, Santa Cruz de la Sierra | Duración: 2 horas

Datos de la Empresa: AURA S.R.L. | Tipo: (X) Privada ( ) Estatal

Datos del Entrevistado: Directora General y Gerente Comercial de AURA

Datos de los Entrevistadores: Diego Astete Paz, Denilson Cruz

2. ELEMENTOS DEL SISTEMA BASADO EN COMPUTADORAS

2.1. HARDWARE

Para la ejecución y pruebas de la plataforma AURA, se contempla una arquitectura de hardware desacoplada:

2.1.1. Servidor

- • **CPU:** Procesador AMD Ryzen 5 / Intel Core i5 de 6 núcleos o instancia Cloud VPS.
- • **RAM:** 16 GB DDR4/DDR5 para ejecutar PostgreSQL, NestJS y el servidor Vite concurrentemente.
- • **Almacenamiento:** Disco SSD NVMe con 20 GB disponibles para almacenamiento local y caché de modelos 3D.
- • **Red:** Conexión de red Gigabit Ethernet o Wi-Fi de alta velocidad.

2.1.2. Cliente

- • **Dispositivos:** Computadora portátil, desktop, tablet o smartphone con aceleración por GPU para WebGL y WebAssembly.

2.1.3. Medios de Comunicación

- • **Protocolos:** Red local o Internet de banda ancha mediante protocolos TCP/IP, HTTPS/TLS en puertos 5173, 3000 y 5432.

2.1.4. Otros Dispositivos

- • **Cámara Web:** Cámara web HD integrada o USB para captura de video en vivo del probador MediaPipe Pose.
- • **Micrófono:** Micrófono para dictado de comandos de voz en reportes de analítica con Web Speech API.

2.2. SOFTWARE

2.2.1. Servidor

- • **Software de Servidor:** Node.js 20 LTS, NestJS 10 modular, TypeORM 0.3 y PostgreSQL 16 sobre Docker.

2.2.2. Cliente

- • **Navegador Web:** Google Chrome, Firefox, Edge o Safari con WebGL 2.0 y WebAssembly.

2.2.3. Otro software adicional

- • **Herramientas Auxiliares:** Docker Compose, VS Code, Stripe CLI y servidor SMTP Mailtrap para pruebas.

2.3. DATOS

El sistema procesa conjuntos relacionales que abarcan: Cuentas de usuario y roles RBAC, Libreta de direcciones, Catálogo de prendas, Variantes por talla/color con SKU único, Inventario físico, Carritos persistentes, Notas de venta correlativas, Transacciones bancarias de Stripe, Métodos de envío y Bitácora de auditoría inalterable.

2.4. PROCESOS

Procesos lógicos centrales: Alta de prendas con variantes, Probador virtual 3D y AR por webcam, Checkout electrónico seguro con tarjeta, Descuento atómico de existencias, Despacho logístico con código de guía y Consultas analíticas por voz con IA.

2.5. GENTE/USUARIO

• Se definen los 4 actores oficiales del sistema:

- **1.** Administrador del Sistema (A1): Personal directivo con control global, configuración de roles, aprobación de cupones, reembolsos y consultas analíticas por voz.

- **2.** Cliente (A2): Usuario registrado facultado para comprar, persistir su carrito, guardar favoritos y emitir reseñas verificadas.
- **3.** Visitante (A3): Comprador potencial anónimo que navega el catálogo, interactúa con el probador 3D/AR y mantiene una bolsa de compra temporal.
- **4.** Sistema (A4): Ente automatizado que agrupa la pasarela Stripe, el servidor SMTP de correos, los servicios de IA de Hugging Face/NLP y el reloj/scheduler de tareas periódicas.

2.6. DOCUMENTO

Documentación técnica y funcional generada para la sostenibilidad del sistema:

- Manual de Usuario y Administración.
- Diccionario de Datos Relacional de las 40 tablas de PostgreSQL.
- Comprobantes Oficiales de Compra y Reportes Ejecutivos de Ventas e Inventario.

2.6.1. Mecanismo de Aprendizaje e Inducción de Usuario (Operación sin Inconvenientes)

Para garantizar que cualquier usuario (sea un visitante que compra por primera vez o un directivo que consulta métricas) aprenda a utilizar la aplicación de forma intuitiva, autónoma y sin inconvenientes, AURA incorpora una arquitectura de adopción progresiva (Progressive Onboarding & Self-Learning Mechanism) estructurada en cinco dimensiones operativas:

• 1. Inducción Guiada y Arquitectura Intuitiva en el Catálogo:

Desde el primer ingreso, la interfaz presenta una navegación limpia y predecible. La barra superior contiene accesos directos con iconografía estándar y etiquetas textuales claras (Catálogo, Probador Virtual, Favoritos, Carrito). En las fichas de producto, cada prenda cuenta con una 'Guía de Tallas Interactiva' accesible a un solo clic, donde el usuario puede comparar sus medidas en centímetros (busto,

cintura, cadera) con la tabla oficial de corte de AURA, eliminando la duda inicial sobre qué talla seleccionar.

• 2. Mecanismo de Aprendizaje del Probador Virtual 3D y Realidad Aumentada:

El probador virtual fue diseñado para guiar visualmente al usuario sin manuales complejos:

a) Modo 3D Orbital: Al ingresar, se despliega un tooltip animado que enseña los gestos universales: 'Arrastra con el ratón o el dedo para rotar 360°' y 'Usa la rueda o pellizca para hacer zoom'. Los botones de iluminación (Estudio, Cálido, Pasarela) muestran iconos representativos que permiten entender al instante cómo cambia la textura de la tela bajo diferentes luces.

b) Modo AR en Vivo con Cámara Web: Al activar la cámara, una silueta guía semitransparente indica la posición óptima del cuerpo (a 1.5 - 2 metros del lente). El sistema activa puntos de verificación verdes sobre los hombros del usuario confirmando que Google MediaPipe ha fijado los landmarks correctamente, momento en el cual la prenda se superpone con caída dinámica a 60 fps.

c) Modo Foto IA: Incluye recomendaciones explícitas de captura ('Foto frontal de cuerpo entero, buena iluminación, ropa ajustada') y provee modelos base predefinidos para que el usuario experimente antes de subir su propia fotografía.

• 3. Micro-interacciones y Notificaciones de Confirmación Inmediata:

Para evitar la incertidumbre típica del e-commerce ('¿se agregó mi producto?', '¿se procesó mi pago?'), AURA implementa retroalimentación sensorial inmediata con la librería Sonner. Cada adición de una prenda dispara una notificación elegante con el nombre de la prenda y un botón directo 'Ver Bolsa' o 'Probar con IA'. Si el usuario selecciona una variante agotada, el botón cambia dinámicamente a 'Agotado' y se deshabilita, evitando frustraciones.

• 4. Mecanismo de Checkout Asistido en Tres Pasos:

El proceso de compra se divide en un asistente visual paso a paso (Wizard): Paso 1: Dirección de entrega con validación de campos obligatorios en tiempo real; Paso 2: Selección de método de envío donde el costo del flete se suma de forma transparente al total; Paso 3: Pago seguro con Stripe Elements, donde el formulario formatea automáticamente los espacios del número de tarjeta y valida la fecha de expiración mientras el cliente escribe, previniendo errores de tipeo.

• 5. Mecanismo de Aprendizaje del Asistente Analítico por Voz (Panel Admin):

Para que los directivos aprendan a consultar métricas sin saber SQL, el módulo de reportes incluye: a) Un botón de micrófono con animación de pulso que indica claramente cuándo el sistema está escuchando; b) 'Chips de Preguntas Sugeridas' clicables que sirven de ejemplo (e.g. '¿Cuáles son los 3 vestidos más vendidos?', 'Prendas con stock crítico', 'Ingresos de esta semana'); y c) Despliegue de respuestas inmediatas en lenguaje natural acompañadas de tablas y gráficos explicativos.

3. TECNOLOGIA PARA EL DESARROLLO DEL SOFTWARE

3.1. Estrategia para el desarrollo del software

El sistema AURA será de desarrollo propio, fundamentado en las siguientes razones estratégicas:

- • **Control total:** Permite estructurar una arquitectura desacoplada y escalable bajo estándares de ingeniería sin depender de plataformas cerradas como Shopify o WooCommerce.
- • **Adaptación al negocio:** La venta de alta costura exige probadores virtuales con 3D interactivo y realidad aumentada en tiempo real (MediaPipe Pose), herramientas inexistentes en e-commerce genéricos.
- • **Propiedad intelectual:** El código fuente, los modelos de inferencia y la base de datos de clientes constituirán un activo patrimonial propio de AURA.
- • **Modularidad interna:** La organización Monorepo permite desacoplar el backend y frontend en módulos autónomos, facilitando la incorporación de nuevas funciones sin afectar la base operativa.

3.2. Metodología para el desarrollo del software

El proyecto adopta el Proceso Unificado de Desarrollo de Software (PUDS), estructurado en cuatro fases cronológicas:

- • **Fase de Iniciación:** Definición del alcance comercial, identificación de los 4 actores y modelado de los casos de uso primarios. Entregable: Documento de visión y requisitos iniciales.
- • **Fase de Elaboración:** Diseño de la arquitectura base en capas (NestJS, React, PostgreSQL 16), normalización 3FN del modelo de datos e integración de la pasarela Stripe. Entregable: Prototipo ejecutable de la arquitectura.

- • **Fase de Construcción:** Desarrollo iterativo de los 4 ciclos funcionales (Seguridad, Catálogo, Probador 3D/AR y Ventas/Stripe/NLP), pruebas unitarias e integración continua. Entregable: Versión beta funcional completa.
- • **Fase de Transición:** Pruebas de aceptación con usuarios reales, afinamiento del rendimiento de MediaPipe y despliegue del sistema en contenedores Docker. Entregable: Versión 1.0 en producción.

3.2.1. Características del PUDS

- • **Iterativo e incremental:** El desarrollo se divide en ciclos controlados, entregando incrementos ejecutables de software con valor comprobable.
- • **Orientado a casos de uso:** El sistema se diseña en torno a las metas y necesidades de los usuarios finales (probar ropa, comprar con tarjeta, consultar reportes).
- • **Arquitectura centrada:** La arquitectura en capas se concibe y valida tempranamente para asegurar escalabilidad y mantenibilidad.
- • **Gestión de riesgos:** Los riesgos críticos (seguridad en pagos, latencia en modelos 3D, concurrencia en stock) se resuelven en las primeras iteraciones.
- • **Documentación viva:** Los modelos y diagramas evolucionan en paralelo con el código fuente, garantizando coherencia formal.
- • **Pruebas continuas:** Cada ciclo incluye validaciones exhaustivas para asegurar la estabilidad del sistema.

3.2.2. Características de UML

- • **Lenguaje visual estándar:** Notación gráfica estándar adoptada internacionalmente para la ingeniería de software orientada a objetos.

- • **Multiperspectiva:** Ofrece modelos estáticos (clases, paquetes, despliegue) y modelos dinámicos (casos de uso, secuencia, actividades).
- • **Escalabilidad:** Se adapta con precisión tanto a pequeños módulos como a ecosistemas transaccionales de gran envergadura.
- • **Interoperabilidad:** Reduce la ambigüedad en la comunicación entre el equipo de desarrollo, analistas y directivos de la empresa.

3.2.3. UML – Diagramas a utilizar

- **1) Diagramas de Casos de Uso:** Identificación de actores y delimitación de funcionalidades de la tienda.
- **2) Diagramas de Clases:** Modelado conceptual con estereotipos Boundary, Control, Entity (BCE) y clases de diseño con atributos y métodos.
- **3) Diagramas de Secuencia:** Modelado temporal de mensajes para los flujos críticos (Login, Probador 3D/AR, Carrito, Pago Stripe y Analítica NLP).
- **4) Diagrama de Despliegue:** Representación de la infraestructura física de contenedores Docker, servidor de base de datos y clientes.
- **5) Diagrama de Componentes:** Visualización de los módulos ejecutables (`apps/api`, `apps/web`) y librerías externas.
- **6) Diagrama Entidad-Relación (DER):** Esquema relacional normalizado con las 40 tablas del sistema en PostgreSQL.

3.3. HERRAMIENTAS DE DESARROLLO

3.3.1. Software

- • **Lenguaje Base:** Node.js 20 LTS y TypeScript 5 para garantizar tipado estricto y alto rendimiento.
- • **Frameworks Web:** NestJS v10 en backend (arquitectura modular) y React 18 con Vite en frontend.
- • **Motor de Base de Datos:** PostgreSQL 16 sobre Docker para persistencia relacional ACID de 40 tablas.
- • **Motores Gráficos e IA:** Three.js para renderizado de mallas 3D (.glb) y Google MediaPipe Tasks-Vision para tracking corporal a 60 fps.
- • **Entorno y Versionado:** Visual Studio Code, Git y GitHub.

3.3.2. Hardware

- • **CPU:** CPU AMD Ryzen 5 / Intel Core i5 con 6 núcleos o superior.
- • **Memoria RAM:** 16 GB RAM DDR4/DDR5.
- • **Almacenamiento:** Disco SSD NVMe con 20 GB disponibles.

4. POSIBLES COSTOS

- • **Mano de Obra y Desarrollo:** 0 Bs., ejecutado íntegramente por los estudiantes de Sistemas de Información 2.
- • **Dominio Comercial:** 250 Bs./año para el registro del dominio oficial `aura-atelier.bo`.

- • **Pasarela Stripe:** Comisiones estándar por transacción exitosa de Stripe (2.9% + 0.30 USD por cobro).
- • **Infraestructura Cloud:** Servidor Cloud VPS (DigitalOcean / AWS) con 4 vCPUs y 8 GB RAM a un costo estimado de 280 Bs./mes.

5. POSIBLES BENEFICIOS

5.1. TIEMPO

La plataforma reduce en un 80% el tiempo que las clientas tardan en elegir y confirmar su talla ideal gracias al probador 3D/AR interactivo. Asimismo, ahorra más de 3 horas diarias de trabajo manual al personal de ventas al automatizar la verificación de existencias y comprobación de transferencias.

5.2. ESFUERZO

Disminuye drásticamente el esfuerzo administrativo al sincronizar automáticamente el stock por variante tras cada compra y permitir consultas analíticas inmediatas formuladas por voz, eliminando la necesidad de elaborar reportes manuales en hojas de cálculo.

5.3. COSTOS

Minimiza las pérdidas económicas asociadas a devoluciones textiles, reduciendo la tasa de cambios del 35% al 8%. Elimina totalmente las pérdidas financieras por sobreventa de prendas agotadas.

6. POSIBLES CLIENTES

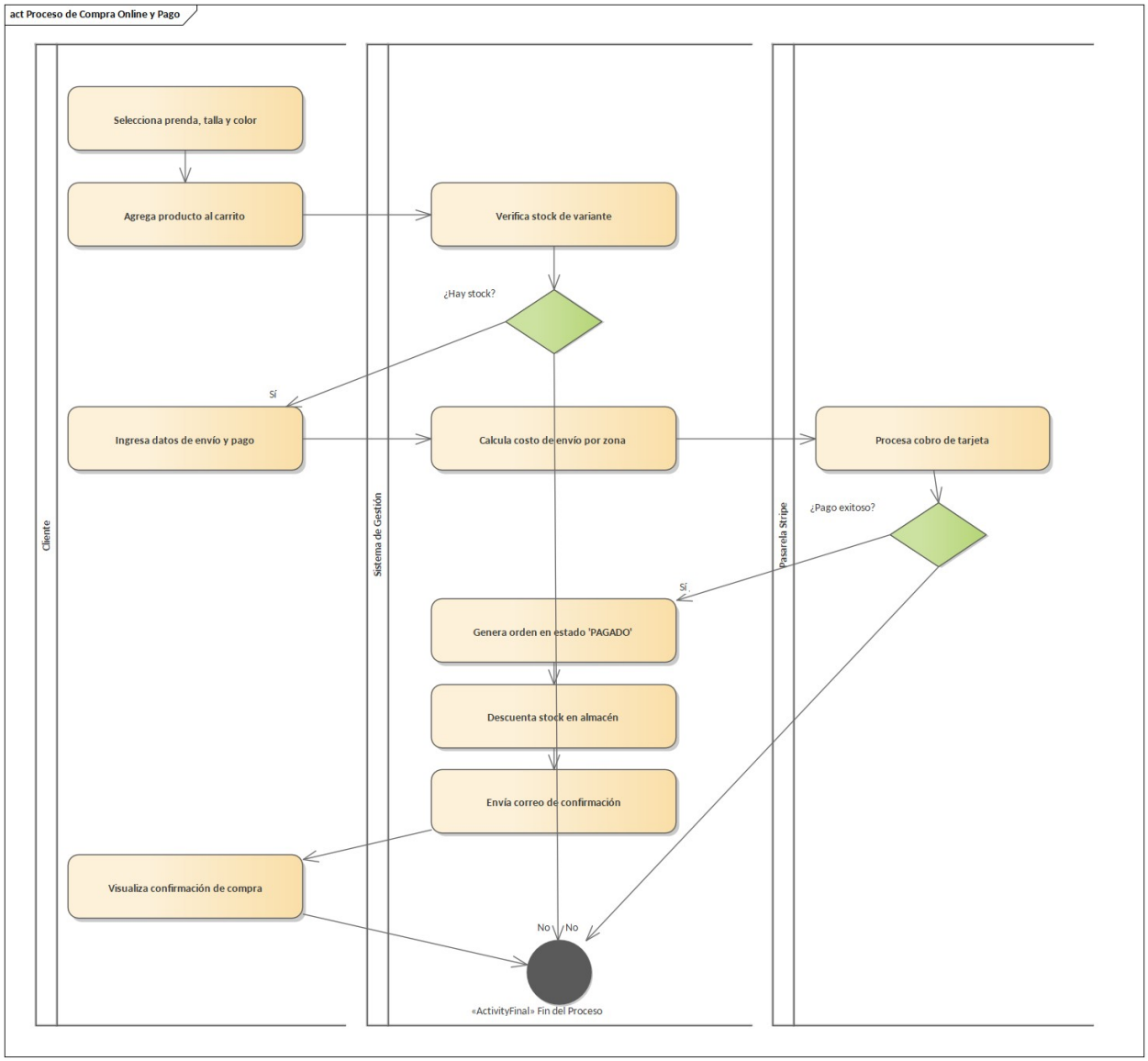
La plataforma AURA está dirigida al segmento de consumidoras de moda femenina contemporánea, ejecutivas, profesionales y entusiastas del buen vestir que buscan prendas exclusivas con la conveniencia de probárselas digitalmente desde la comodidad de su hogar antes de concretar la compra.

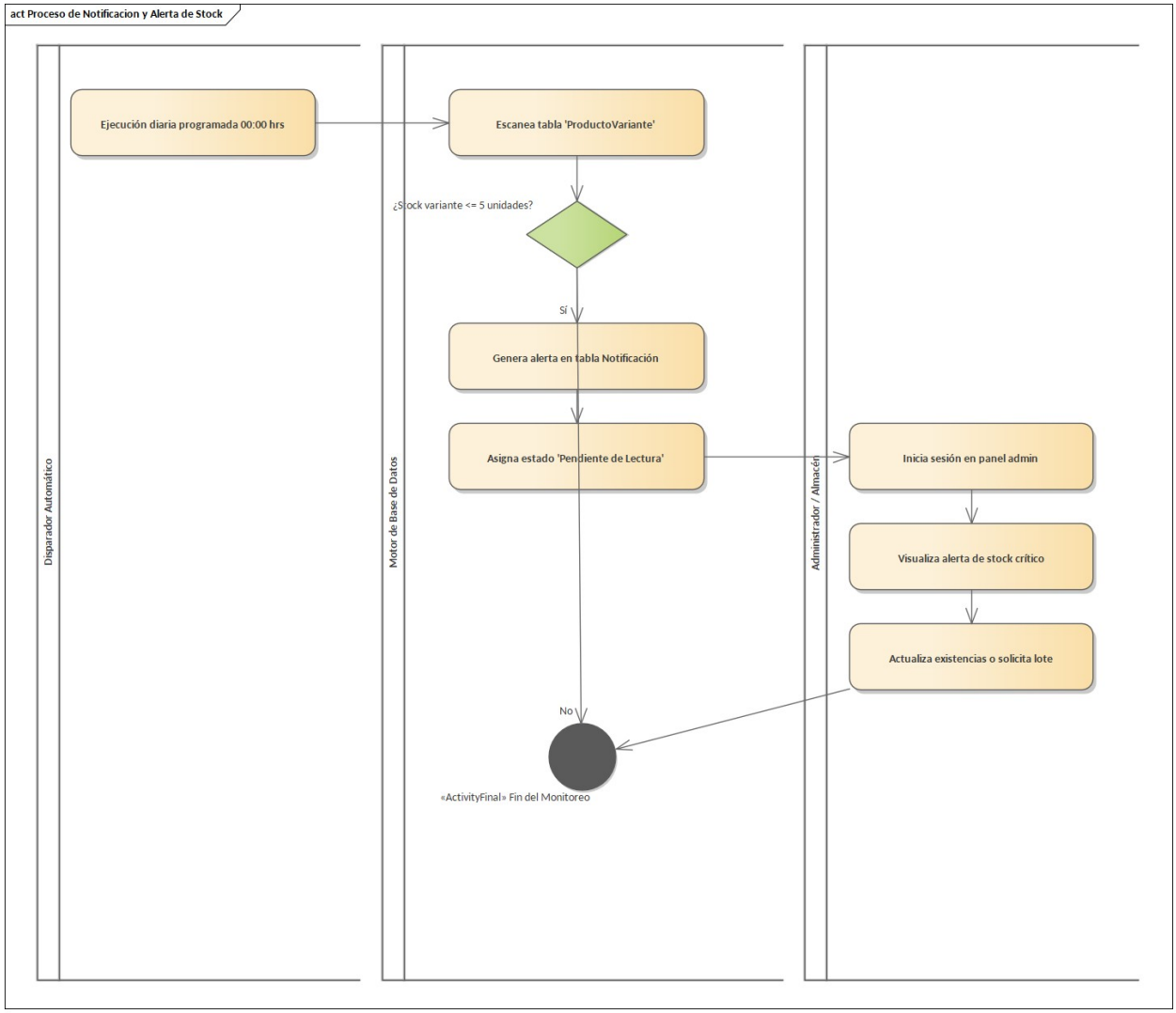
7. MODELO DE NEGOCIO

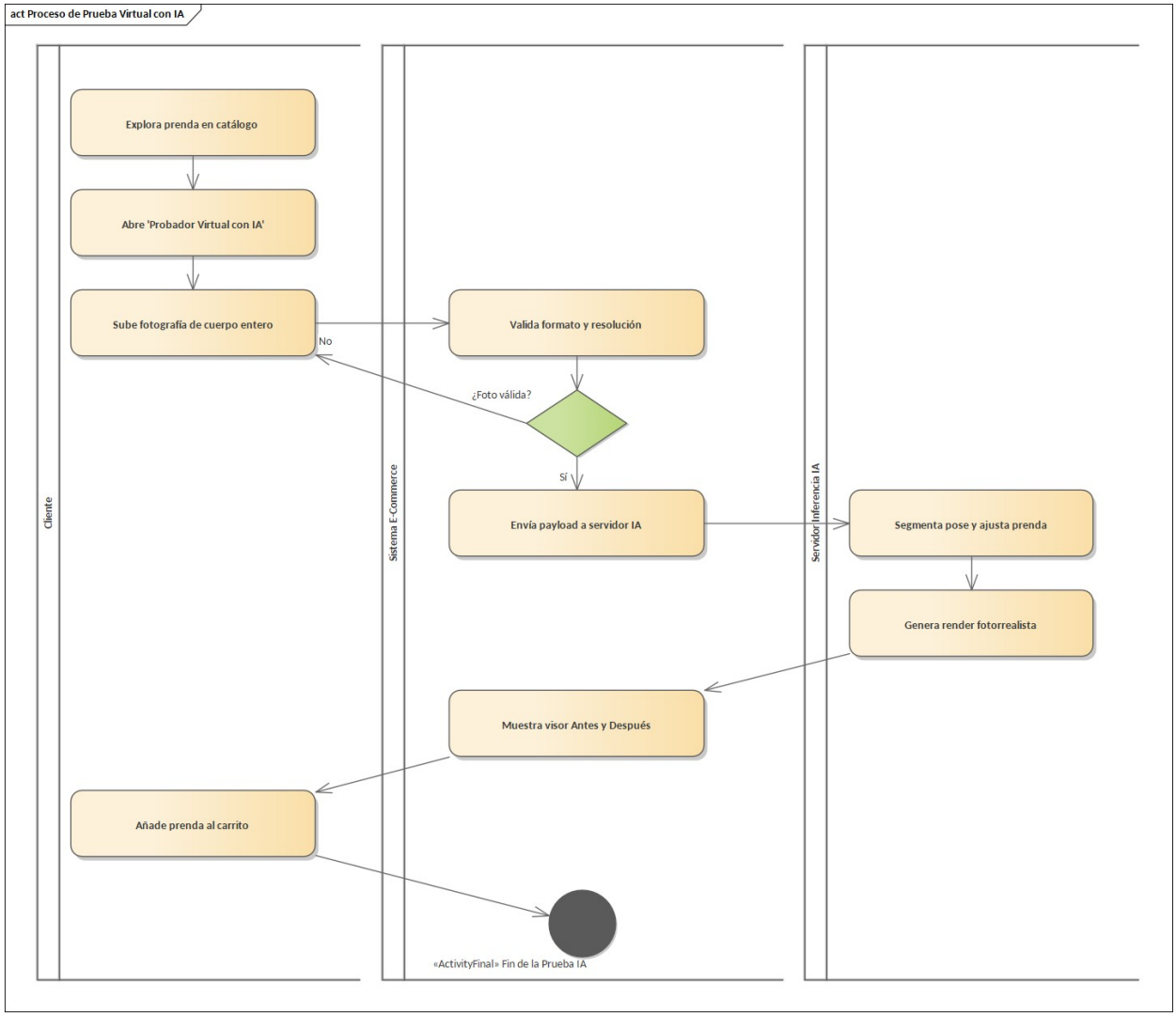
• Diagrama de actividades

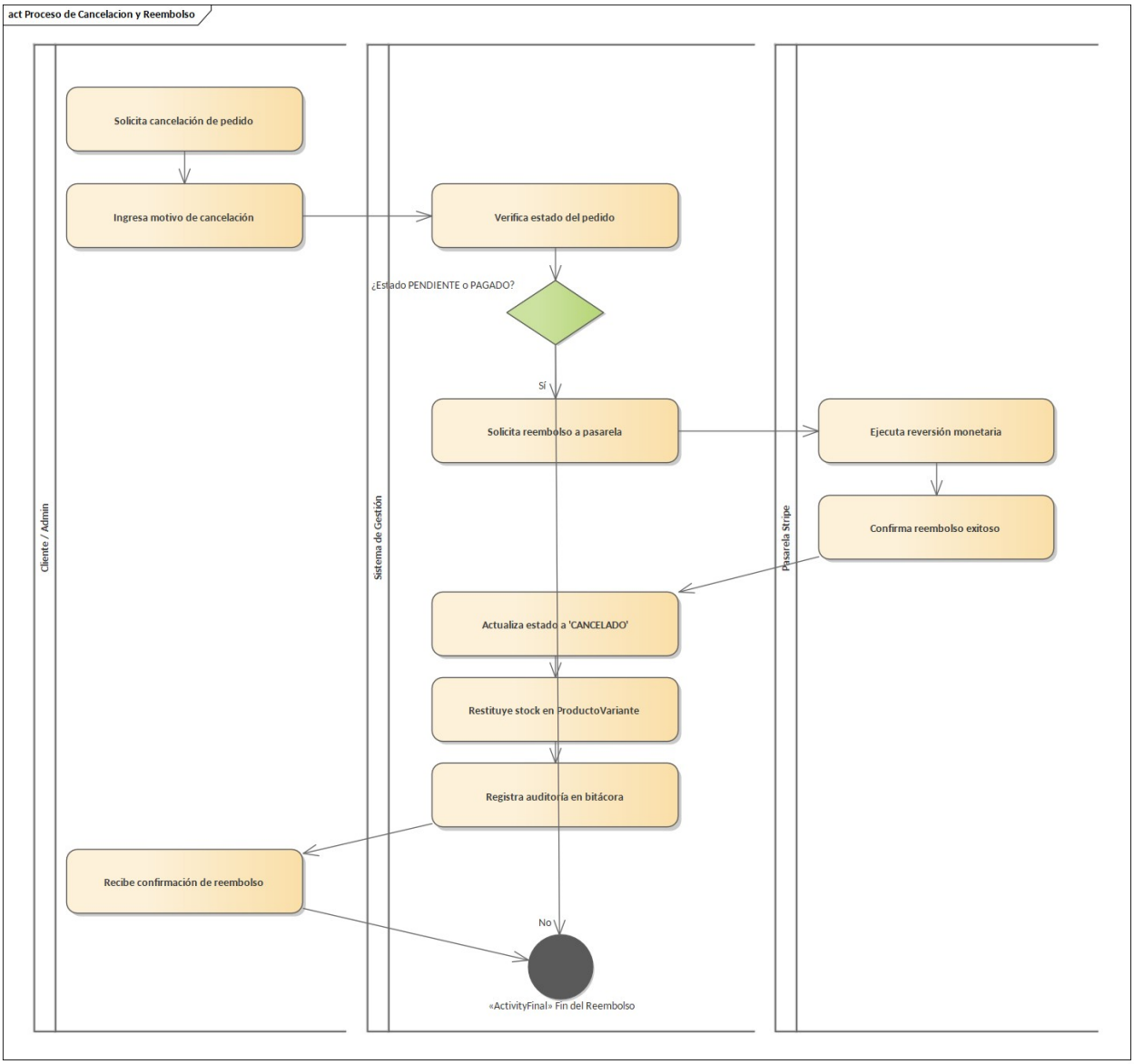
Procesos fundamentales del negocio AURA:

- **1. Proceso de Venta y Checkout:** Selección de prenda ➔ bolsa de compras ➔ dirección ➔ cupón ➔ pago Stripe ➔ rebaja de stock ➔ comprobante.
- **2. Proceso de Prueba Virtual 3D y AR:** Apertura de probador ➔ inspección 3D Three.js ➔ cámara web ➔ proyección MediaPipe a 60 fps ➔ confirmación.
- **3. Proceso de Control de Inventario:** Registro de prenda ➔ parametrización de tallas/colores ➔ ingreso de stock SKU ➔ alertas de reposición.
- **4. Proceso de Despacho y Logística:** Preparación de paquete ➔ actualización a EN_CAMINO ➔ asignación de guía de courier ➔ entrega final.


act Proceso de Compra Online y Pago
Cliente
Sistema de Gestión
Pasarela Stripe
Selecciona prenda, talla y color
Agrega producto al carrito
Verifica stock de variante
¿Hay stock?
Sí
Ingresa datos de envío y pago
Calcula costo de envío por zona
Procesa cobro de tarjeta
¿Pago exitoso?
Sí
Genera orden en estado 'PAGADO'
Descuenta stock en almacén
Envía correo de confirmación
Visualiza confirmación de compra
No
No
«ActivityFinal» Fin del Proceso


act Proceso de Notificacion y Alerta de Stock
Disparador Automático
Motor de Base de Datos
Administrador / Almacén
Ejecución diaria programada 00:00 hrs
Escanea tabla 'ProductoVariante'
¿Stock variante <= 5 unidades?
Sí
Genera alerta en tabla Notificación
Asigna estado 'Pendiente de Lectura'
Inicia sesión en panel admin
Visualiza alerta de stock crítico
Actualiza existencias o solicita lote
No
«ActivityFinal» Fin del Monitoreo


act Proceso de Prueba Virtual con IA
Cliente
Sistema E-Commerce
Servidor Inferencia IA
Explora prenda en catálogo
Abre 'Probador Virtual con IA'
Sube fotografía de cuerpo entero
Valida formato y resolución
No
¿Foto válida?
Sí
Envía payload a servidor IA
Segmenta pose y ajusta prenda
Genera render fotorrealista
Muestra visor Antes y Después
Añade prenda al carrito
«ActivityFinal» Fin de la Prueba IA



8. Marco Teórico

1) Sistema de Información: Conjunto interrelacionado de personas, datos, procesos y tecnología que recolecta, procesa y distribuye información para la toma de decisiones. En AURA, centraliza la operativa comercial de moda.

2) Teoría General de Sistemas (TGS): Aborda las organizaciones como sistemas abiertos que interactúan dinámicamente con su entorno (clientes, proveedores textiles, pasarelas de pago y couriers logísticos).

3) Sistemas de Información Basados en Computadora: Automatizan la gestión transaccional masiva mediante software y bases de datos para garantizar precisión, velocidad e integridad de datos.

4) Gestión de Inventarios y Variantes Textiles: Administración atómica de prendas mediante códigos SKU independientes, controlando existencias por modelo, talla y color para prevenir quiebres de stock.

5) Arquitectura Monorepo y en Capas: Organización del código en capas desacopladas (Presentación, Controladores, Dominio, Repositorios) coordinadas mediante Turborepo para compartir interfaces tipadas.

6) Ciclo de Vida del Software: Secuencia formal de etapas que guían la concepción, especificación, codificación, verificación y mantenimiento de un producto de software.

7) Proceso Unificado de Desarrollo de Software (PUDS): Marco de trabajo iterativo e incremental dirigido por casos de uso y centrado en la arquitectura, estructurado en fases de Inicio, Elaboración, Construcción y Transición.

8) Lenguaje Unificado de Modelado (UML): Estándar visual de la OMG para modelar artefactos de software mediante diagramas estructurales y de comportamiento dinámico.

9) Modelo de Casos de Uso: Técnica fundamental de PUDS para capturar los requerimientos funcionales desde la perspectiva de los actores que interactúan con el sistema.

10) Seguridad de la Información Web (OWASP): Adopción de estándares defensivos contra inyecciones SQL, Cross-Site Scripting (XSS), autenticación con JWTs cifrados y hashing Bcrypt de contraseñas.

11) Automatización de Procesos con IA y 3D: Integración de Google MediaPipe Tasks-Vision en WebAssembly y Three.js para ofrecer experiencias inmersivas de probador virtual sin latencia del servidor.

12) Importancia en el Contexto Empresarial Moderno: La adopción de tecnologías de probador inteligente y analítica por voz transforma tiendas de ropa tradicionales en marcas digitales escalables y de alta fidelización.

CAPITULO 1: FLUJO DE TRABAJO: CAPTURA DE REQUISITOS

1.1. Identificar actores y casos de usos

• Actores Oficiales del Sistema AURA:

- **o A1 - Administrador del Sistema:** Responsable de la configuración global, gestión de cuentas de usuario, asignación de roles RBAC, aprobación de cupones, reembolsos y consultas analíticas por voz.
- **o A2 - Cliente:** Usuario registrado en la tienda con cuenta, libreta de direcciones, historial de compras, capacidad de pago en Stripe, persistencia de carrito, favoritos y reseñas verificadas.
- **o A3 - Visitante:** Usuario potencial anónimo que navega el catálogo público, interactúa con el probador virtual 3D/AR y mantiene una bolsa de compra temporal.

1.2. Priorizar casos de uso

Descripción breve de los casos de uso por ciclos de desarrollo:

Ciclo #1: Base de Seguridad, Usuarios y Control

- **CU01: Gestionar sesión de usuario —** Registro de cuenta, inicio de sesión (JWT + Refresh Token), verificación de correo electrónico y cierre de sesión seguro
- **CU-02: Recuperar contraseña de acceso —** Solicitud de restablecimiento vía correo electrónico con token temporal cifrado y actualización de contraseña.
- **CU-03: Gestionar información de perfil —** Consulta y actualización de datos personales (nombre, teléfono, foto de perfil).
- **CU04: Modificar contraseña —** Cambio de clave de acceso desde el perfil validando la contraseña actual.

- **CU05: Administrar libreta de direcciones —** Permite al cliente registrar, editar y seleccionar su dirección de entrega predeterminada.
- **CU06: Bloquear / Desbloquear cuenta —** Permite al administrador suspender o reactivar cuentas por motivos de seguridad.
- **CU07: Asignar y gestionar roles —** Permite asignar privilegios y roles en el sistema (ADMIN, CLIENTE, EMPLEADO).

Ciclo #2: Catálogo de Moda, Clasificación e Inventario Multi-Variante

- **CU08: Administrar catálogo y variantes —** Permite registrar y actualizar prendas con múltiples tallas, colores y SKUs independientes.
- **CU10: Administrar categorías jerárquicas —** Permite estructurar el menú en categorías padre y subcategorías hijas con slugs amigables.
- **CU11: Gestionar existencias e inventario —** Permite registrar ingresos de stock, controlar existencias físicas y emitir alertas de reposición.
- **CU19: Gestionar multimedia WebP —** Permite cargar fotos de prendas, optimizarlas a formato WebP y asociarlas al catálogo.
- **CU25: Gestionar lista de favoritos —** Permite al cliente guardar prendas favoritas y trasladarlas directamente a la bolsa de compra.

Ciclo #3: Experiencia Inmersiva (Probador 3D/AR), Carrito y Reseñas

- **CU09: Probar prendas en probador 3D y AR —** Permite previsualizar prendas en 3D (Three.js), tracking corporal en vivo (MediaPipe) o foto IA.

- **CU12: Gestionar carrito de compras —** Permite agregar variantes, modificar unidades y calcular subtotales dinámicos.
- **CU13: Sincronizar y persistir carrito —** Permite almacenar el carrito localmente y fusionarlo en la BD al autenticarse.
- **CU24: Publicar y moderar reseñas —** Permite calificar con 1 a 5 estrellas y moderar opiniones de clientes verificados.

Ciclo #4: Ventas, Checkout, Pagos Stripe, Logística y Analítica por Voz

- **CU14: Gestionar pedidos y estados —** Permite generar órdenes de venta y actualizar sus etapas de despacho.
- **CU15: Procesar pago con Stripe —** Permite procesar cobros electrónicos con tarjeta de crédito vía Stripe y pagos en efectivo.
- **CU16: Procesar reembolso de pago —** Permite revertir fondos bancarios vía Stripe y restituir automáticamente el stock.
- **CU17: Gestionar métodos de envío —** Permite cotizar tarifas de flete y asignar números de guía de seguimiento.
- **CU18: Administrar cupones de descuento —** Permite crear y validar cupones de descuento porcentuales o de monto fijo.
- **CU20: Enviar correo de bienvenida —** Despacha automáticamente un correo de bienvenida HTML al registrarse.

- **CU21: Enviar correo de confirmación de compra —** Despacha automáticamente el comprobante digital con detalle del pedido al cliente.
- **CU22: Generar reportes estadísticos —** Genera gráficos y métricas de ingresos con exportación a PDF y CSV.
- **CU23: Generar reportes dinámicos por voz con IA —** Interpreta preguntas formuladas por voz o texto mediante NLP y genera consultas SQL.

• Matriz de Priorización de Casos de Uso según Impacto en el Negocio y Arquitectura:

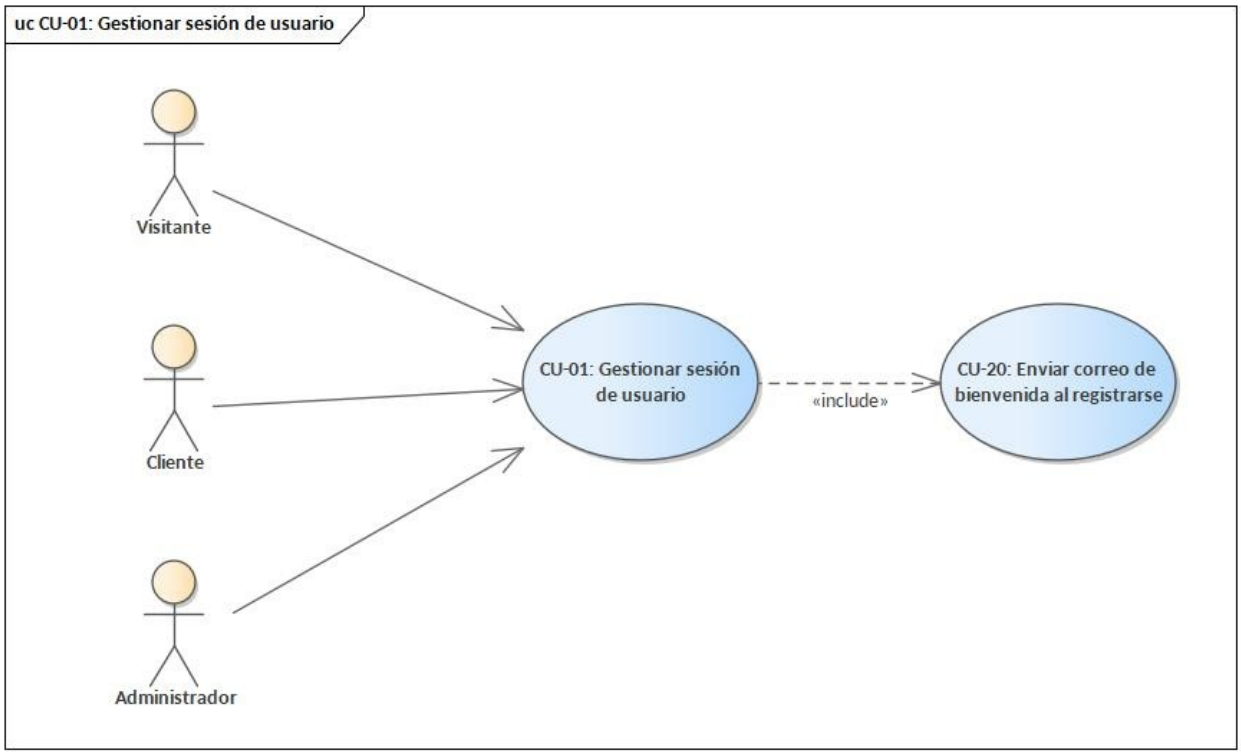
ID	CASOS DE USO	PRIORIDAD	ACTORES	CICLO
CU01	Gestionar sesion y usuario	**Alto**	A1, A2, A3	C1*
CU02	Recuperar contraseña de acceso	Bajo	A1, A2	C1*
CU03	Gestionar informacion del perfil	Medio	A1, A4	C1*
CU04	Modificar contraseña	Medio	A2	C1*
CU05	Administrar libreta de direcciones	**Alto**	A2	C1*
CU06	Bloquear / Desbloquear cuenta	Bajo	A1, A4	C1*
CU07	Asignar y gestionar roles	Medio	A1	C1*
CU08	Administrar catálogo y variantes	**Alto**	A1, A2, A3	C2*
CU10	Administrar categorías jerárquicas	Medio	A1	C2*
CU11	Gestionar existencias e inventario	**Alto**	A1, A4	C2*
CU19	Gestionar multimedia WebP	Medio	A1, A4	C2*
CU25	Gestionar lista de favoritos	Bajo	A2	C2*
CU09	Probar prendas en probador 3D y AR	**Alto**	A2, A3, A4	C3*
CU12	Gestionar carrito de compras	**Alto**	A2, A3	C3*
CU13	Sincronizar y persistir carrito	**Alto**	A2, A4	C3*
CU24	Publicar y moderar reseñas	Medio	A1, A2	C3*
CU14	Gestionar pedidos y estados	**Alto**	A1, A2, A4	C4

CU15	Procesar pago con Stripe	**Crítico**	A2, A4	C4
CU16	Procesar reembolso de pago	Medio	A1, A4	C4
CU17	Gestionar métodos de envío	**Alto**	A1, A2, A4	C4
CU18	Administrar cupones de descuento	Medio	A1, A2, A4	C4
CU20	Enviar correo de bienvenida	Bajo	A4	C4
CU21	Enviar correo de confirmación	Medio	A4	C4
CU22	Generar reportes estadísticos	**Alto**	A1	C4
CU23	Generar reportes por voz con IA	**Alto**	A1, A4	C4

1.3. Detallar Casos de Uso

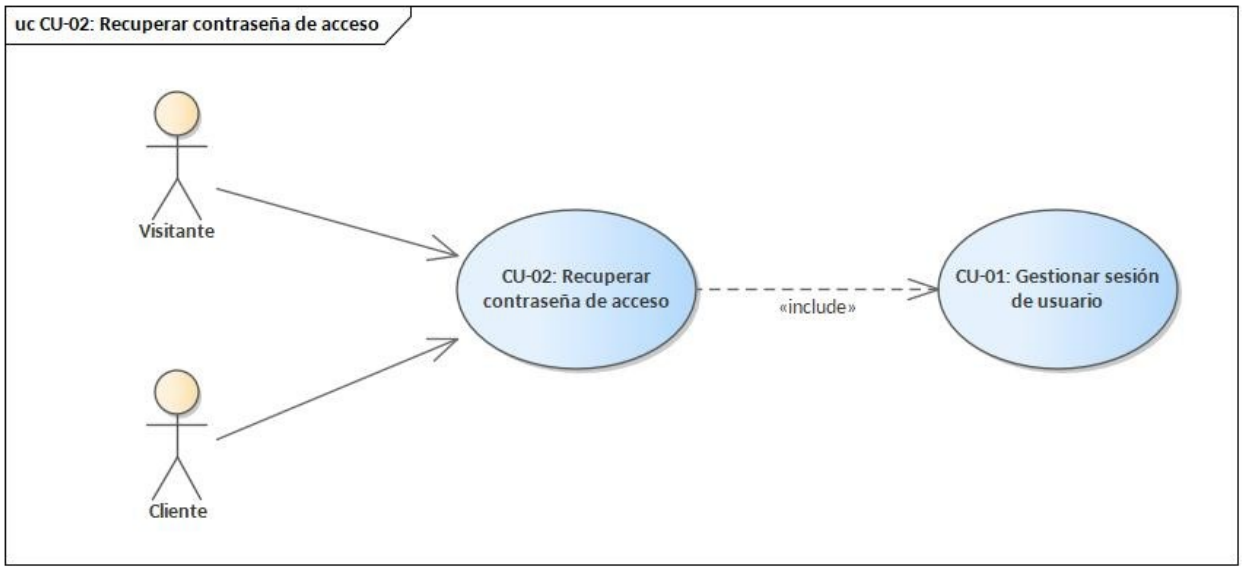
Fichas técnicas formales de los 25 Casos de Uso estructuradas bajo los 4 actores oficiales:

CU01: Gestionar sesión de usuario



Nombre	Iniciar sesión
Propósito	Permitir el acceso seguro al sistema mediante credenciales.
Descripción	El sistema autentica al usuario mediante email y contraseña, emitiendo tokens JWT y Refresh Token.
Actor(es)	A1: Administrador, A2: Cliente, A3: Visitante, A4: Sistema
Actor iniciador	A1, A2, A3
Pre condición	El usuario debe estar registrado y con cuenta activa.
Flujo principal	1. El usuario ingresa email y contraseña. 2. El sistema verifica el hash Bcrypt. 3. Se generan tokens JWT firmados. 4. Se concede acceso y redirige según rol.
Post condición	Usuario autenticado con sesión activa.
Excepción	Credenciales incorrectas o cuenta suspendida.

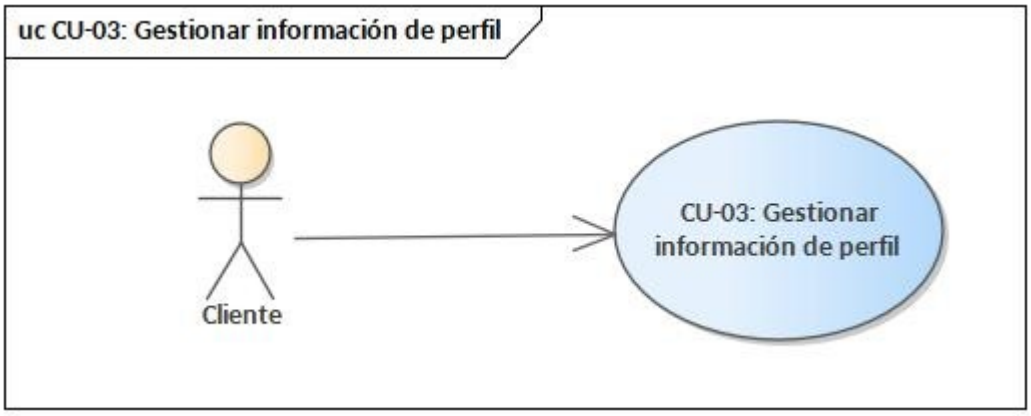
CU02: Recuperar contraseña de acceso



Nombre	Cerrar sesión
Propósito	Restablecer el acceso a la cuenta de usuario en caso de olvido o pérdida de la clave de seguridad
Descripción	Permite al usuario solicitar un enlace de restablecimiento a través de su correo electrónico registrado, validar un token temporal seguro de un solo uso y definir una nueva contraseña de acceso.
Actor(es)	A1: Administrador, A2: Cliente
Actor iniciador	A1, A2
Pre condición	1.El usuario debe tener una cuenta previamente registrada y activa en el sistema con un correo electrónico válido. 2.El usuario no recuerda su contraseña o no puede iniciar sesión.
Flujo principal	1.El usuario ingresa a la opción "¿Olvidaste tu contraseña?" e introduce su correo electrónico registrado. 2.El sistema verifica la existencia del correo en la base de datos. 3.El sistema genera un token criptográfico temporal con tiempo de expiración (15 minutos) y lo almacena asociado al usuario. 4.El sistema envía un correo electrónico transaccional con el enlace de recuperación y el token seguro. 5.El usuario abre el correo electrónico y accede al enlace de recuperación. 6.El sistema valida que el token sea auténtico, pertenezca al usuario y no haya expirado. 7.El sistema muestra la interfaz para definir la nueva contraseña. 8.El usuario ingresa su nueva contraseña y su

	confirmación. 9.El sistema valida las reglas de seguridad y fortaleza de la nueva clave. 10.El sistema genera el nuevo hash con Bcrypt, actualiza la contraseña en la base de datos e invalida el token temporal.
Post condición	Sesión finalizada con éxito.
Excepción	Fallo de conexión: se limpia sesión local.

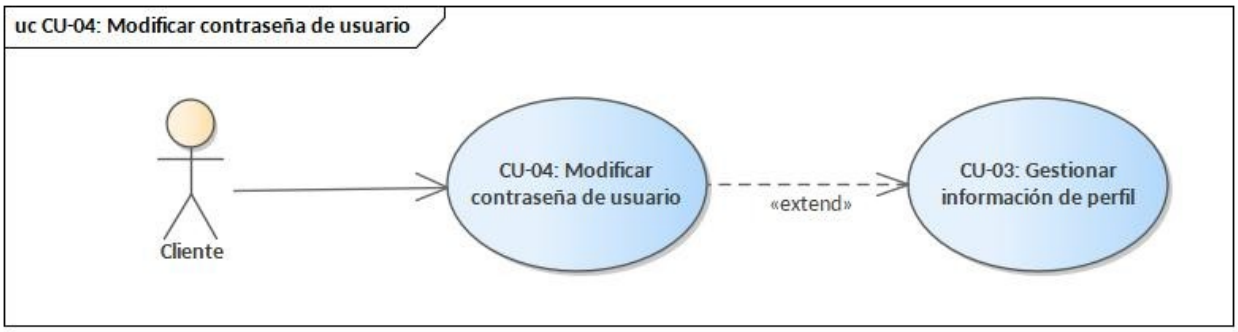
CU03: Gestionar información de perfil



Nombre	Gestionar información de perfil
Propósito	Consultar y actualizar los datos personales y la fotografía de perfil de la cuenta del usuario.
Descripción	Permite al cliente autenticado visualizar su información de cuenta, modificar datos personales (nombre completo, teléfono, fecha de nacimiento) y cargar o actualizar su foto de perfil.
Actor(es)	A2: Cliente
Actor iniciador	A2
Pre condición	El usuario debe encontrarse autenticado en el sistema con una sesión activa (token JWT válido).
Flujo principal	1.El cliente accede a la sección "Mi Perfil" desde el menú de usuario. 2.El sistema consulta y muestra los datos personales actuales (nombre, correo electrónico, teléfono y foto de perfil). 3.El cliente modifica los campos deseados y/o selecciona una nueva imagen de perfil. 4.El cliente presiona el botón "Guardar Cambios". 5.El sistema valida los formatos de los campos y las restricciones del archivo de imagen (formato válido y tamaño máximo permitido).

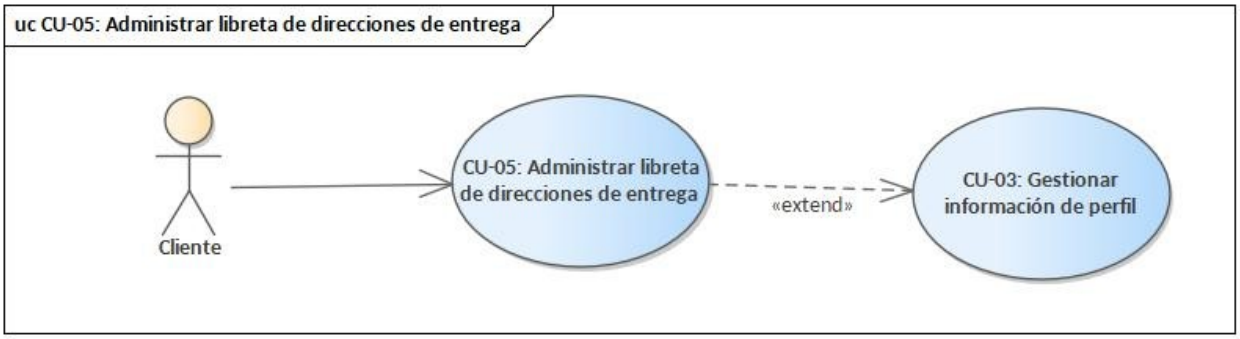
	6.El sistema procesa y optimiza la nueva imagen (si fue adjuntada) y actualiza los registros en la base de datos. 7.El sistema actualiza el estado de la sesión activa y muestra un mensaje de confirmación exitoso.
Post condición	Datos personales y/o fotografía de perfil actualizados en la base de datos. La interfaz de usuario refleja los nuevos datos en el encabezado y en el panel de perfil.
Excepción	E1: Campos obligatorios vacíos o formato de teléfono inválido. E2: El archivo de imagen excede el límite de tamaño permitido o formato no soportado (ej. no es PNG/JPG/WebP). E3: Sesión expirada durante la edición (el sistema redirige al login solicitando reautenticación).

CU04: Modificar contraseña



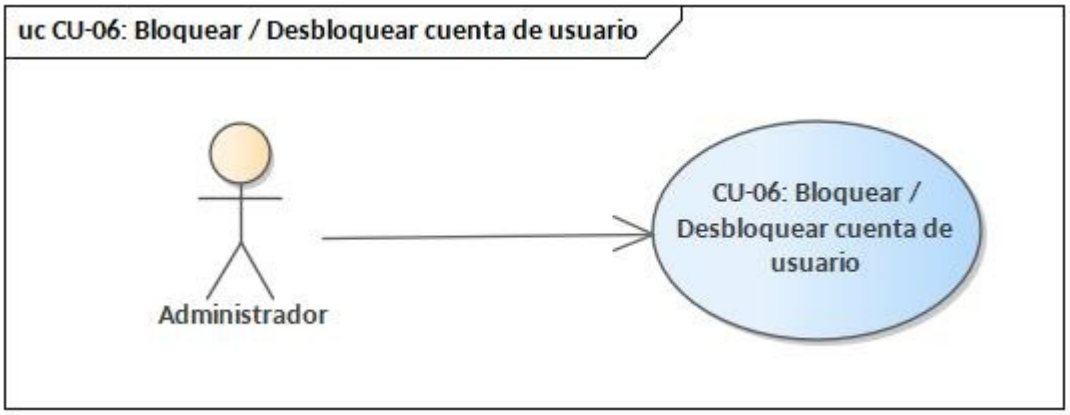
Nombre	Gestionar información de perfil
Propósito	Actualizar la clave de acceso de la cuenta personal.
Descripción	Permite al cliente cambiar su clave verificando previamente su contraseña actual.
Actor(es)	A2: Cliente
Actor iniciador	A2
Pre condición	Usuario autenticado en el sistema.
Flujo principal	1. El cliente ingresa clave actual y nueva clave. 2. El sistema valida contraseña actual con Bcrypt. 3. Valida fortaleza de la nueva clave. 4. Guarda el nuevo hash en la base de datos.
Post condición	Contraseña actualizada exitosamente.
Excepción	Contraseña actual no coincide o clave débil.

CU05: Administrar libreta de direcciones



Nombre	Administrar libreta de direcciones
Propósito	Gestionar direcciones de entrega del cliente.
Descripción	Permite registrar, actualizar y marcar una dirección de envío predeterminada.
Actor(es)	A2: Cliente
Actor iniciador	A2
Pre condición	Cliente autenticado en el sistema.
Flujo principal	1. El cliente accede a Direcciones. 2. Registra departamento, ciudad, calle y referencia. 3. Marca si es la dirección predeterminada. 4. El sistema valida y guarda en tabla `direccion`.
Post condición	Dirección registrada para futuros pedidos.
Excepción	Campos obligatorios vacíos.

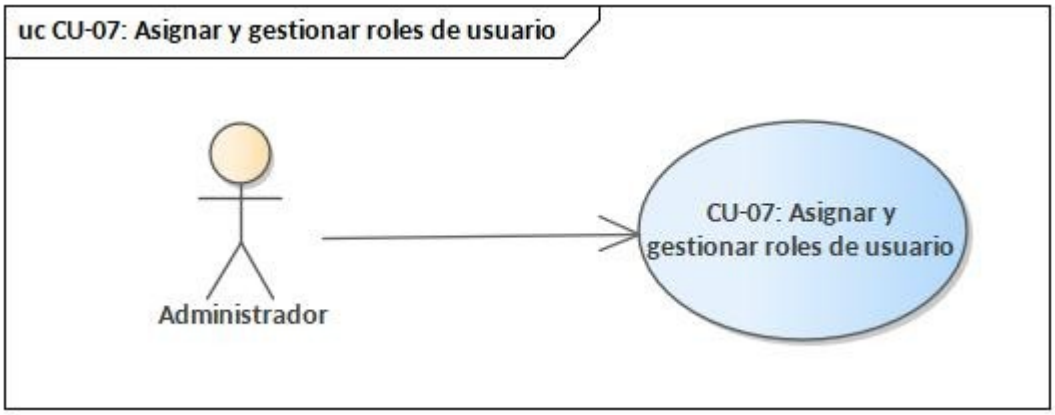
CU06: Bloquear / Desbloquear cuenta



Nombre	Bloquear / Desbloquear cuenta
Propósito	Restringir o reactivar el acceso de un usuario.
Descripción	Permite al administrador suspender cuentas por fraude o reactivarlas tras aclaración.

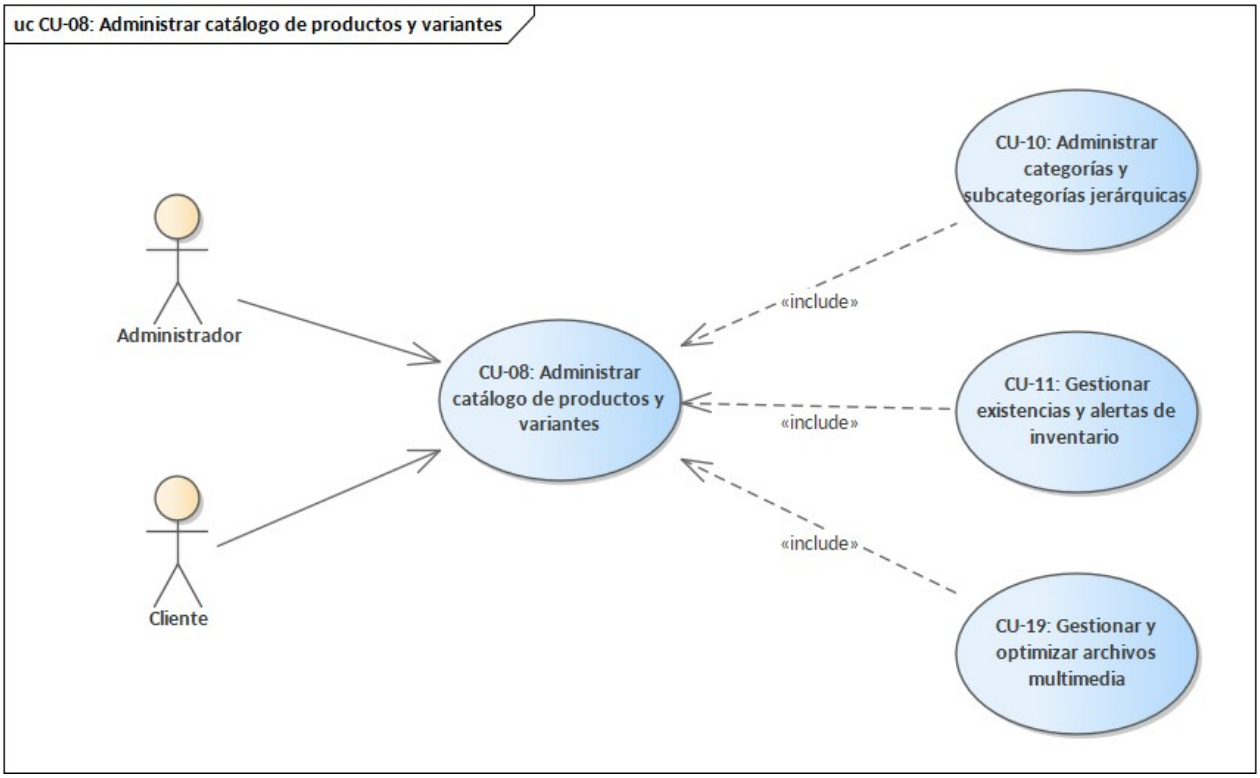
Actor(es)	A1: Administrador, A4: Sistema
Actor iniciador	A1
Pre condición	Administrador autenticado.
Flujo principal	1. El admin busca al usuario en el panel. 2. Selecciona la acción Bloquear/Desbloquear. 3. Confirma motivo y el sistema actualiza `activo`.
Post condición	Estado de la cuenta actualizado en BD.
Excepción	Intento de autobloqueo de cuenta admin.

CU07: Asignar y gestionar roles



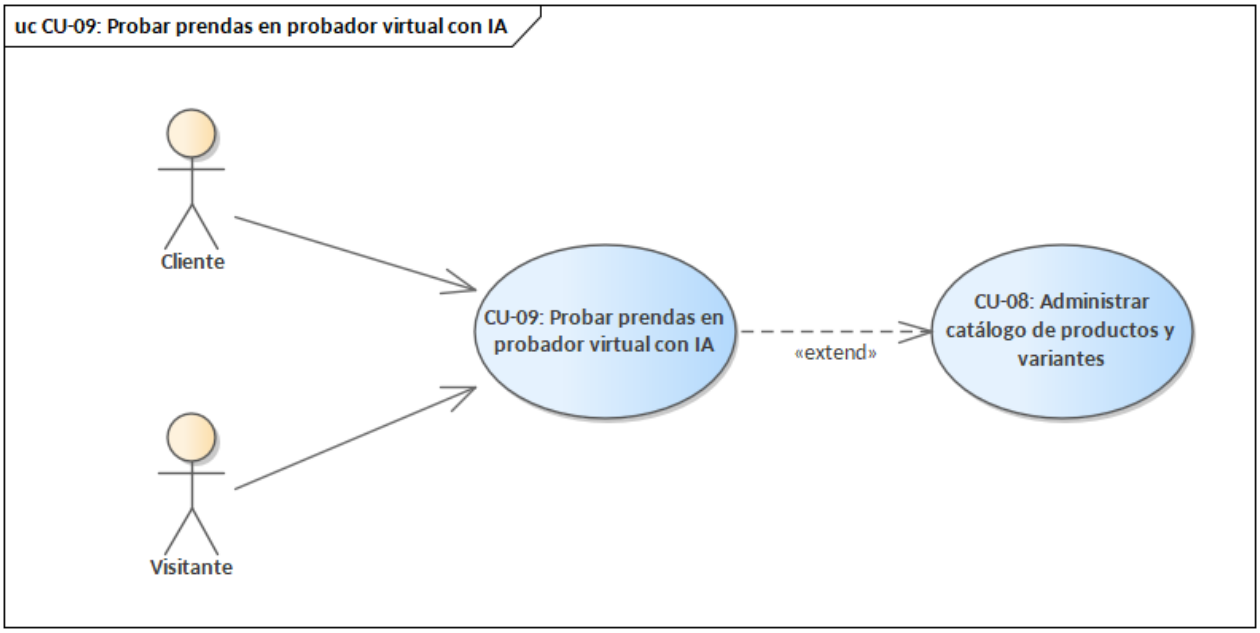
Nombre	Asignar y gestionar roles
Propósito	Controlar permisos jerárquicos del personal.
Descripción	Permite asignar roles RBAC (ADMIN, CLIENTE, EMPLEADO) a los usuarios.
Actor(es)	A1: Administrador
Actor iniciador	A1
Pre condición	Administrador autenticado.
Flujo principal	1. El admin selecciona usuario y nuevo rol. 2. El sistema valida que permanezca un admin activo. 3. Actualiza clave foránea `rol_id` en `usuario`.
Post condición	Rol reasignado y permisos actualizados.
Excepción	Intento de degradar al último administrador.

CU08: Administrar catálogo y variantes



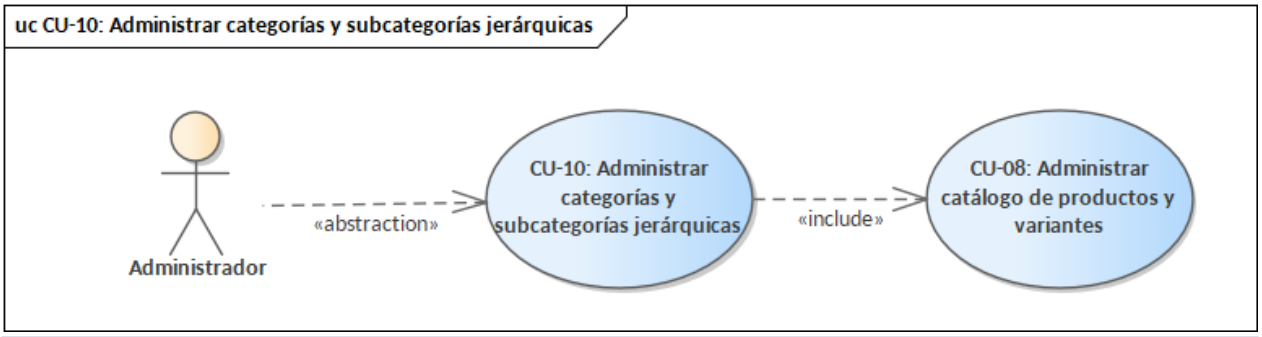
Nombre	Administrar catálogo y variantes
Propósito	Gestionar prendas, tallas, colores y SKUs.
Descripción	Permite el alta, edición y control de prendas con inventario independiente por variante.
Actor(es)	A1: Administrador, A2: Cliente, A3: Visitante
Actor iniciador	A1
Pre condición	Sesión activa con permisos comerciales.
Flujo principal	1. El admin completa nombre, precio y categoría. 2. Añade variantes especificando talla, color, SKU y stock. 3. Guarda la prenda en `producto` y `producto_variante`.
Post condición	Producto y variantes registradas en catálogo.
Excepción	SKU duplicado o precio negativo.

CU09: Probar prendas en probador 3D y AR



Nombre	Probar prendas en probador 3D y AR
Propósito	Probar ropa virtualmente antes de comprar.
Descripción	Permite inspeccionar modelos 3D con Three.js y proyectar la prenda en vivo con MediaPipe Pose.
Actor(es)	A2: Cliente, A3: Visitante, A4: Sistema
Actor iniciador	A2, A3
Pre condición	Prenda con asset 3D o foto de alta resolución.
Flujo principal	1. El usuario pulsa «Probar con IA». 2. Modo 3D: rota la prenda 360° en WebGL. 3. Modo AR: activa cámara y MediaPipe superpone la prenda a 60 fps. 4. El usuario evalúa calce y añade a la bolsa.
Post condición	Prenda visualizada sobre silueta del usuario.
Excepción	Permiso de cámara web denegado.

CU10: Administrar categorías jerárquicas



Nombre	Administrar categorías jerárquicas
Propósito	Clasificar prendas en categorías padre/hijas.
Descripción	Permite crear y mantener el árbol de categorías de navegación de la tienda.
Actor(es)	A1: Administrador
Actor iniciador	A1
Pre condición	Sesión activa de administración.
Flujo principal	1. El admin ingresa nombre y categoría superior. 2. El sistema genera slug para URL amigable. 3. Guarda el registro en la tabla `categoria`.
Post condición	Categoría publicada en el menú de la tienda.
Excepción	Nombre de categoría duplicado.

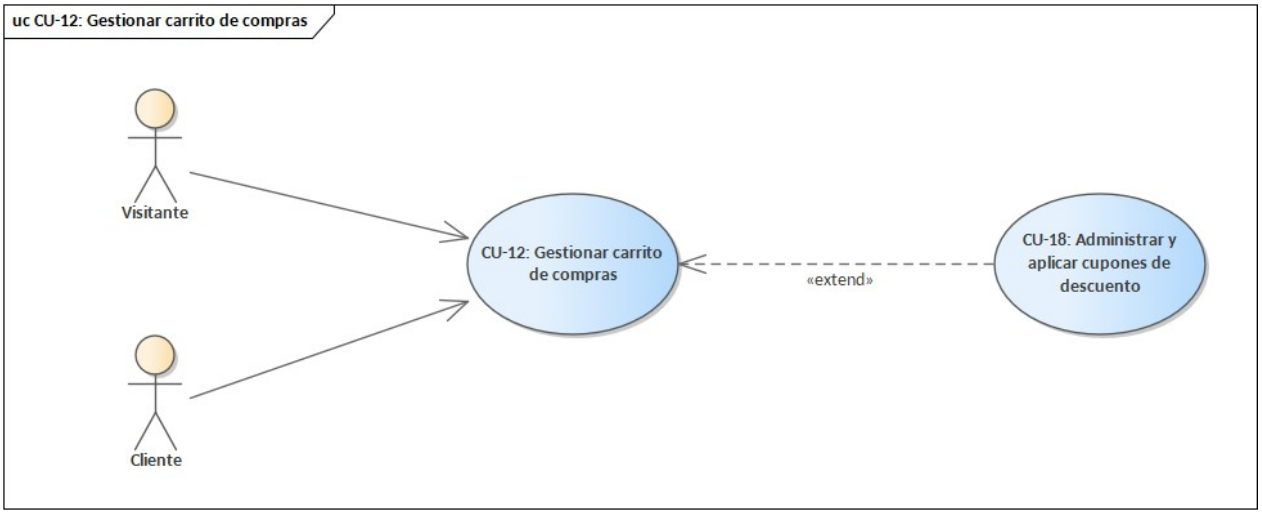
CU11: Gestionar existencias e inventario



Nombre	Gestionar existencias e inventario
Propósito	Controlar stock físico y alertas en almacén.
Descripción	Permite registrar ingresos de mercadería, ajustes y monitorear stock mínimo.
Actor(es)	A1: Administrador, A4: Sistema
Actor iniciador	A1, A4
Pre condición	Permisos sobre el módulo de inventario.

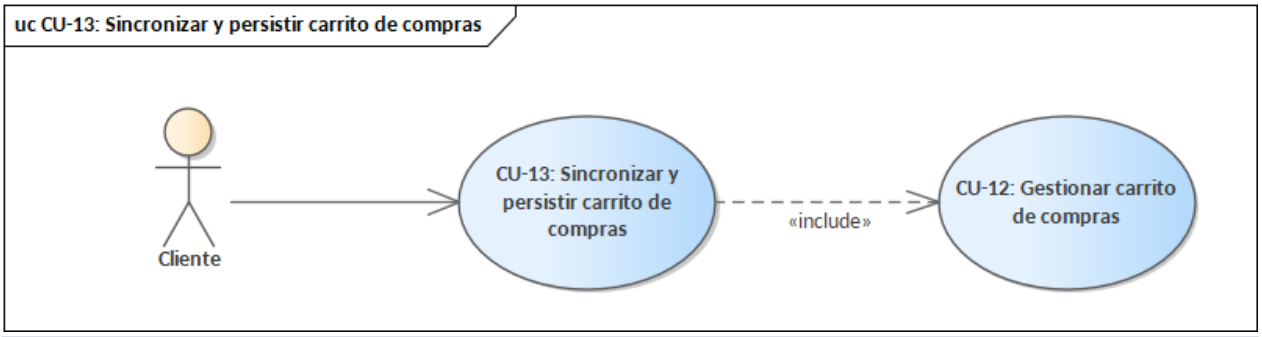
Flujo principal	1. El admin consulta matriz de variantes. 2. Registra ajuste indicando cantidad y motivo. 3. El sistema actualiza `stock` y asienta en kardex.
Post condición	Inventario actualizado y kardex registrado.
Excepción	Ajuste resulta en stock negativo.

CU12: Gestionar carrito de compras



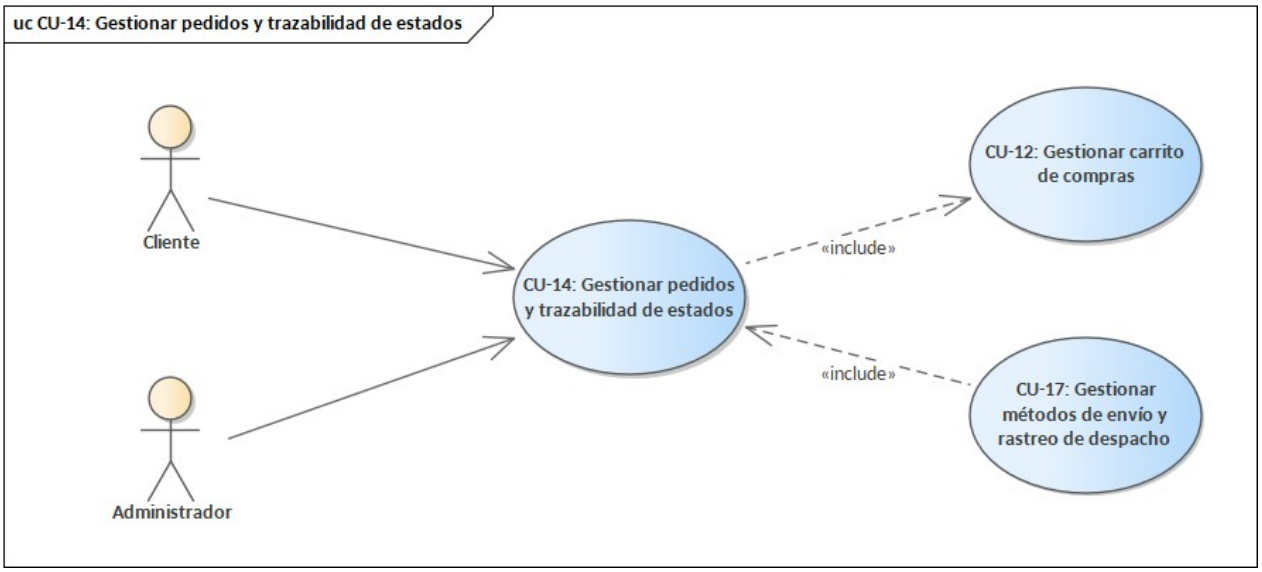
Nombre	Gestionar carrito de compras
Propósito	Administrar bolsa de compras de prendas.
Descripción	Permite añadir variantes (talla/color), alterar cantidades y vaciar el carrito.
Actor(es)	A2: Cliente, A3: Visitante
Actor iniciador	A2, A3
Pre condición	Variante con stock disponible mayor a cero.
Flujo principal	1. El usuario elige talla y color y añade al carrito. 2. El sistema valida disponibilidad de existencias. 3. Agrega ítem y recalcula subtotales y flete.
Post condición	Bolsa de compras actualizada en tiempo real.
Excepción	Variante agotada en almacén.

CU13: Sincronizar y persistir carrito



Nombre	Sincronizar y persistir carrito
Propósito	Garantizar persistencia entre dispositivos.
Descripción	Permite guardar carrito en `localStorage` y fusionarlo con la BD al iniciar sesión.
Actor(es)	A2: Cliente, A4: Sistema
Actor iniciador	A4
Pre condición	Cliente con ítems previos en sesión anónima.
Flujo principal	1. El cliente inicia sesión formalmente. 2. El frontend envía ítems locales al backend. 3. El sistema fusiona cantidades y persiste en BD.
Post condición	Carrito unificado disponible en la cuenta.
Excepción	Stock insuficiente al momento de fusionar.

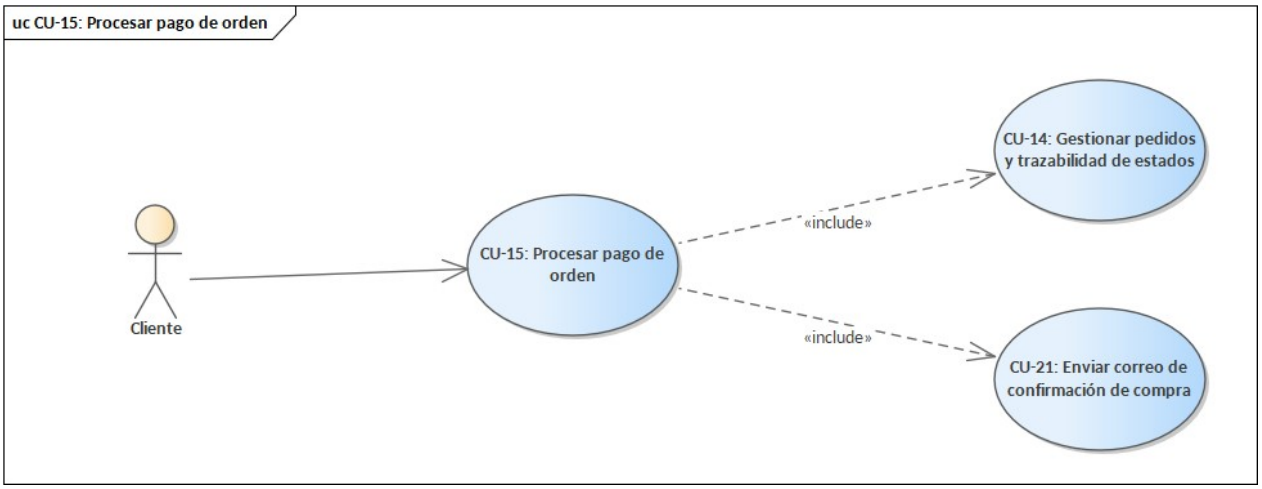
CU14: Gestionar pedidos y estados



Nombre	Gestionar pedidos y estados
Propósito	Trazabilidad completa de órdenes de compra.

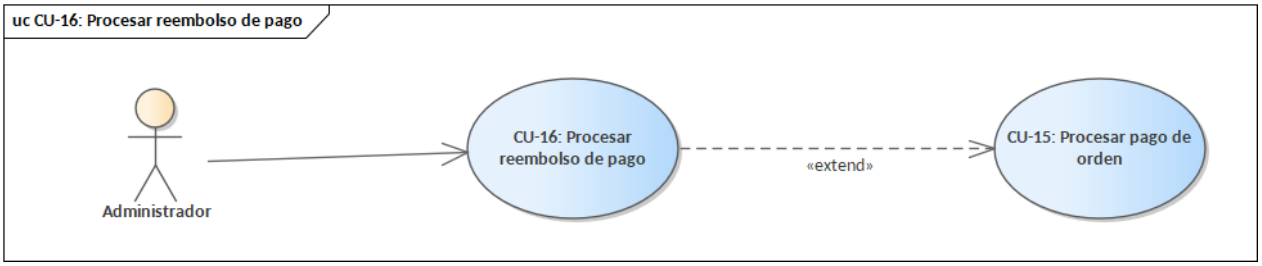
Descripción	Permite consultar pedidos y avanzar estados logísticos (PAGADO ➔ EN_CAMINO ➔ ENTREGADO).
Actor(es)	A1: Administrador, A2: Cliente, A4: Sistema
Actor iniciador	A1, A2
Pre condición	Debe existir una nota de venta registrada.
Flujo principal	1. El cliente o admin consulta el pedido. 2. Almacén prepara paquete y actualiza a EN_CAMINO. 3. Se registra evento en bitácora y se notifica.
Post condición	Pedido con estado actualizado y trazabilidad.
Excepción	Transición de estado inválida.

CU15: Procesar pago con Stripe



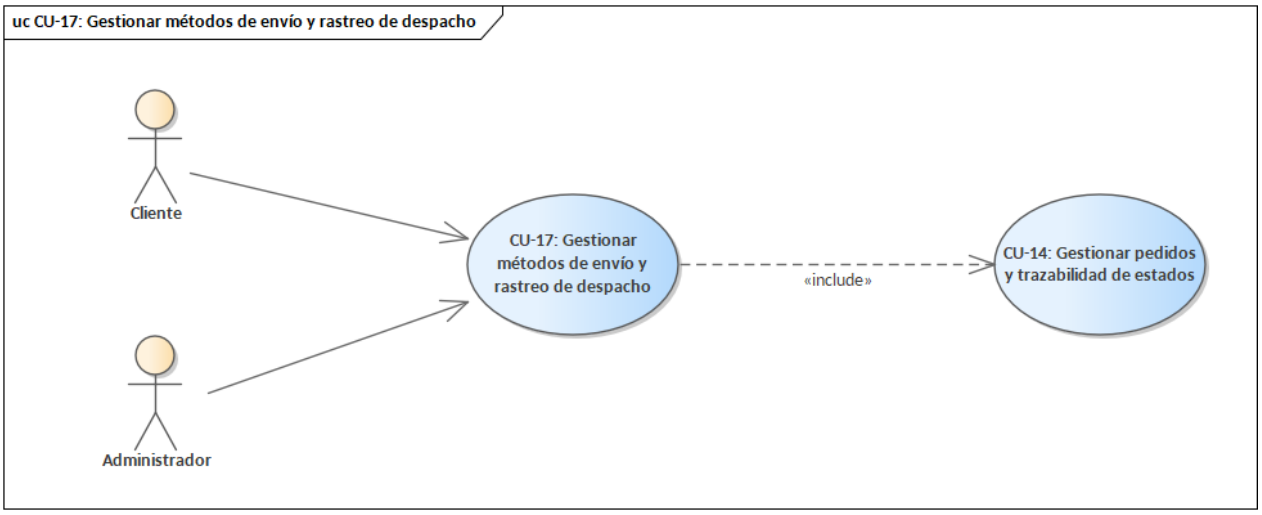
Nombre	Procesar pago con Stripe
Propósito	Cobro electrónico seguro con tarjeta bancaria.
Descripción	Permite procesar pagos con Stripe PaymentIntents y registrar pagos contra entrega.
Actor(es)	A2: Cliente, A4: Sistema
Actor iniciador	A2
Pre condición	Carrito con prendas y dirección seleccionada.
Flujo principal	1. El cliente introduce datos de tarjeta en Stripe. 2. Stripe autoriza la transacción bancaria. 3. Backend descuenta stock de variantes y crea orden.
Post condición	Orden pagada e inventario rebajado en BD.
Excepción	Tarjeta rechazada o fondos insuficientes.

CU16: Procesar reembolso de pago



Nombre	Procesar reembolso de pago
Propósito	Reversión bancaria y reposición de stock.
Descripción	Permite al administrador devolver fondos vía Stripe ante cancelaciones o fallas.
Actor(es)	A1: Administrador, A4: Sistema
Actor iniciador	A1
Pre condición	Orden pagada con transacción Stripe registrada.
Flujo principal	1. El admin ingresa a la orden y solicita reembolso. 2. Invoca a Stripe API (`refunds.create`). 3. Stripe devuelve fondos y sistema repone stock.
Post condición	Fondos restituidos y stock devuelto a almacén.
Excepción	Plazo de disputa expirado en Stripe.

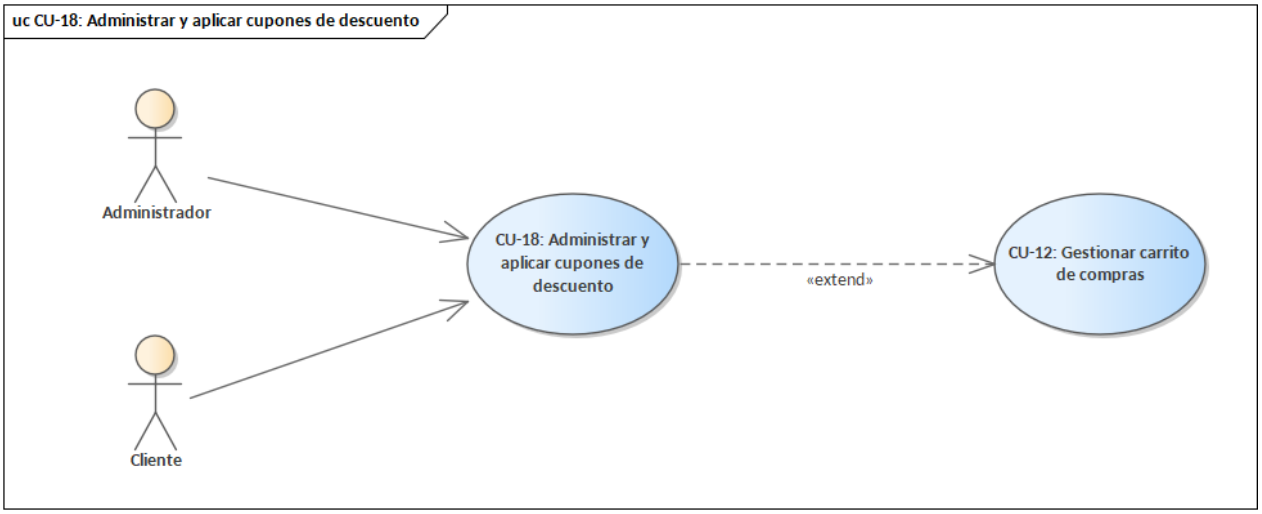
CU17: Gestionar métodos de envío



Nombre	Gestionar métodos de envío
Propósito	Cotizar fletes y seguimiento de entregas.
Descripción	Permite definir tarifas de courier por zona geográfica y asignar guías de transporte.
Actor(es)	A1: Administrador, A2: Cliente, A4: Sistema

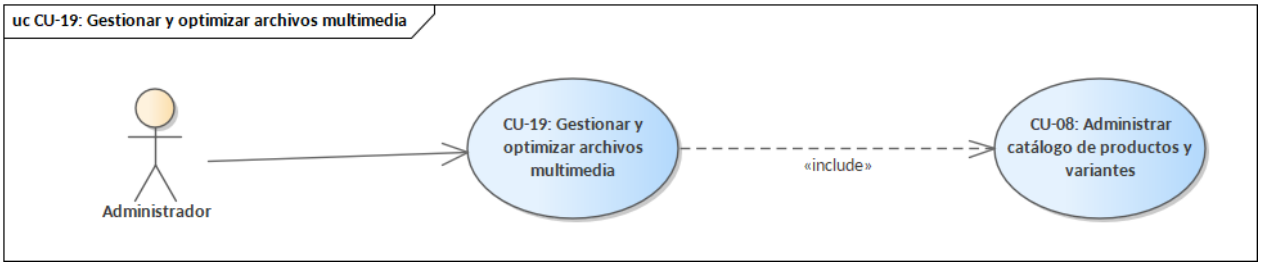
Actor iniciador	A1, A2
Pre condición	Dirección de destino válida seleccionada.
Flujo principal	1. El admin configura tarifas y tiempos por zona. 2. El cliente selecciona modalidad en checkout. 3. El sistema suma el flete al total de la orden.
Post condición	Flete congelado y guía asignada al pedido.
Excepción	Zona fuera de cobertura de entrega.

CU18: Administrar cupones de descuento



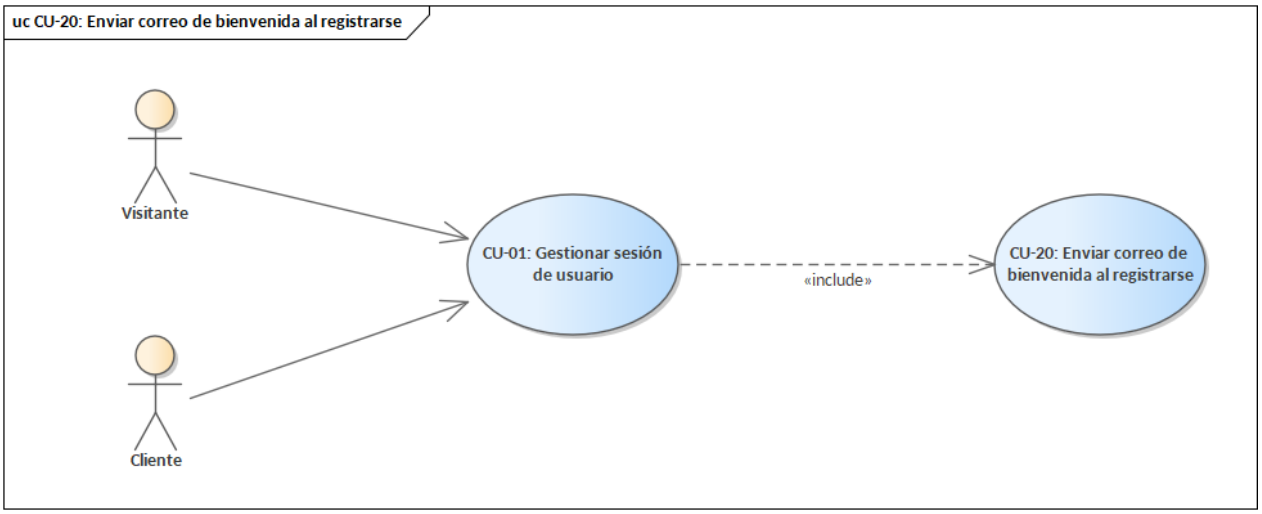
Nombre	Administrar cupones de descuento
Propósito	Crear y canjear promociones comerciales.
Descripción	Permite crear cupones (% o monto fijo) y validarlos en tiempo real en el checkout.
Actor(es)	A1: Administrador, A2: Cliente, A4: Sistema
Actor iniciador	A1, A2
Pre condición	Cupón activo y dentro del plazo de vigencia.
Flujo principal	1. El cliente ingresa código de cupón. 2. El sistema valida vigencia, tope y monto mínimo. 3. Aplica descuento y descuenta un uso del cupón.
Post condición	Descuento aplicado al subtotal del pedido.
Excepción	Cupón vencido o límite de usos alcanzado.

CU19: Gestionar multimedia WebP



Nombre	Gestionar multimedia WebP
Propósito	Optimización y compresión de fotos de ropa.
Descripción	Permite subir múltiples fotos de prendas, comprimirlas a WebP y asociar portadas.
Actor(es)	A1: Administrador, A4: Sistema
Actor iniciador	A1
Pre condición	Archivos de imagen en formato JPG, PNG o WebP.
Flujo principal	1. El admin selecciona fotos de la prenda. 2. El interceptor Multer valida tipos de archivo. 3. Optimiza imagen y guarda en disco con hash único.
Post condición	Imágenes indexadas en tabla `imagen_producto`.
Excepción	Archivo no permitido o mayor a 10 MB.

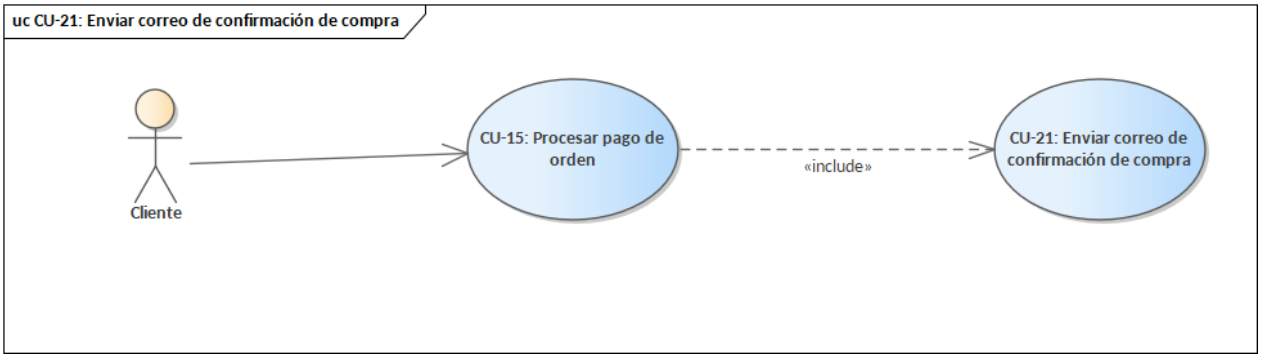
CU20: Enviar correo de bienvenida



Nombre	Enviar correo de bienvenida
Propósito	Notificación de alta de usuario por correo.
Descripción	Despacha automáticamente una plantilla HTML de bienvenida al confirmar el registro.
Actor(es)	A4: Sistema

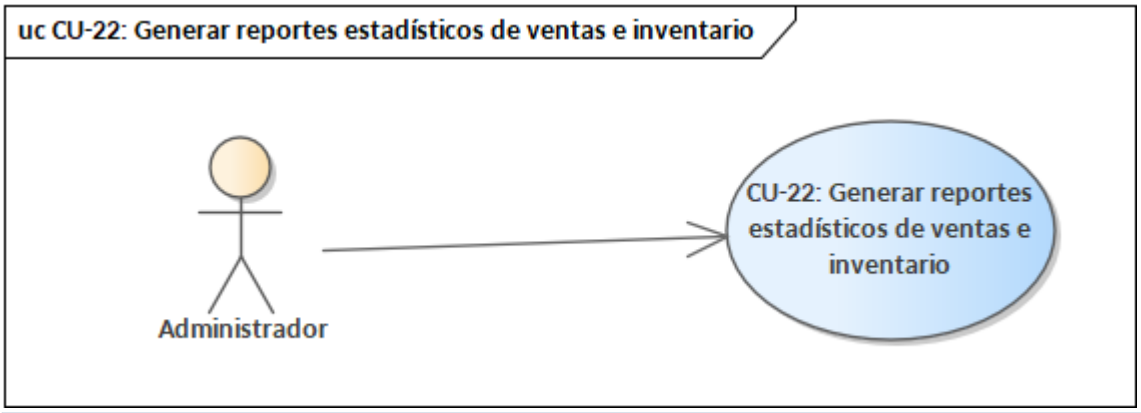
Actor iniciador	A4
Pre condición	Nuevo usuario dado de alta en tabla `usuario`.
Flujo principal	1. Se emite evento `user.registered`. 2. Servicio de correo compila plantilla de AURA. 3. Despacha mensaje mediante servidor SMTP.
Post condición	Cliente recibe confirmación en su bandeja.
Excepción	Falla temporal de conexión SMTP.

CU21: Enviar correo de confirmación de compra



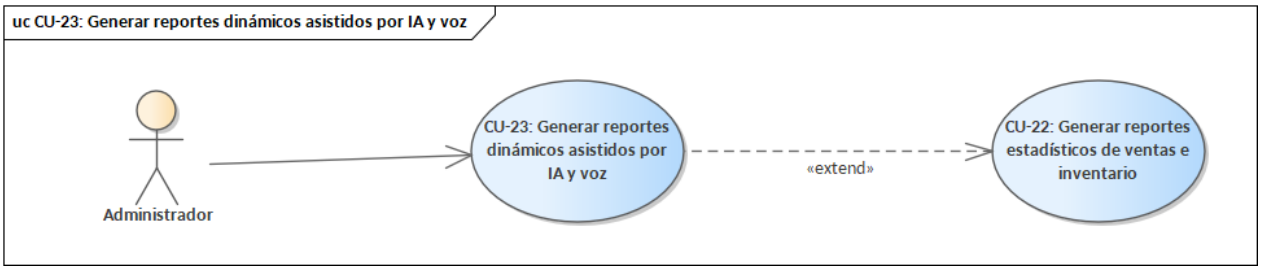
Nombre	Enviar correo de confirmación de compra
Propósito	Despacho de comprobante digital de compra.
Descripción	Envía el recibo formal de venta con lista de prendas, tallas, flete y total.
Actor(es)	A4: Sistema
Actor iniciador	A4
Pre condición	Nota de venta en estado PAGADO.
Flujo principal	1. Al confirmarse el pago se compila el comprobante. 2. Se inserta detalle de ítems y dirección. 3. Se envía correo con número de orden al cliente.
Post condición	Comprobante digital recibido por el cliente.
Excepción	Reintento automático por backoff exponencial.

CU22: Generar reportes estadísticos



Nombre	Generar reportes estadísticos
Propósito	Dashboard gerencial de ingresos e inventario.
Descripción	Permite consultar métricas de venta consolidadas y exportar resúmenes en PDF y CSV.
Actor(es)	A1: Administrador
Actor iniciador	A1
Pre condición	Sesión de administración activa.
Flujo principal	1. El admin define rango de fechas y filtros. 2. El sistema ejecuta consultas de agregación SQL. 3. Despliega gráficas y genera archivo descargable.
Post condición	Reporte analítico generado en pantalla/archivo.
Excepción	Periodo sin transacciones registradas.

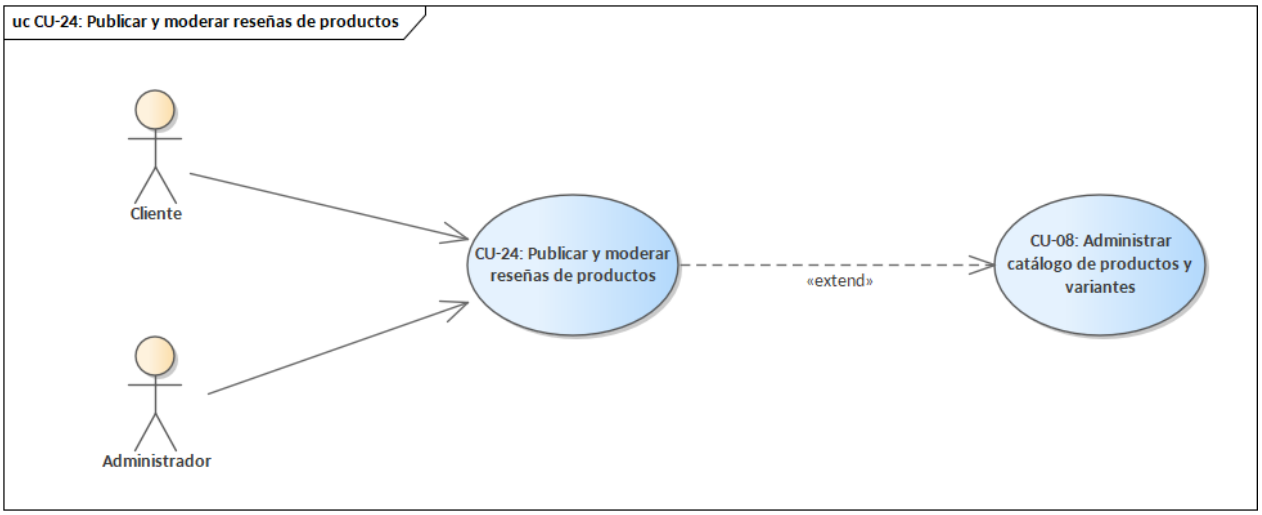
CU23: Generar reportes por voz con IA



Nombre	Generar reportes por voz con IA
Propósito	Consultas analíticas dictadas con lenguaje natural.
Descripción	Permite al directivo hablar por micrófono para que un LLM genere SQL de solo lectura.
Actor(es)	A1: Administrador, A4: Sistema
Actor iniciador	A1
Pre condición	Navegador con micrófono habilitado.

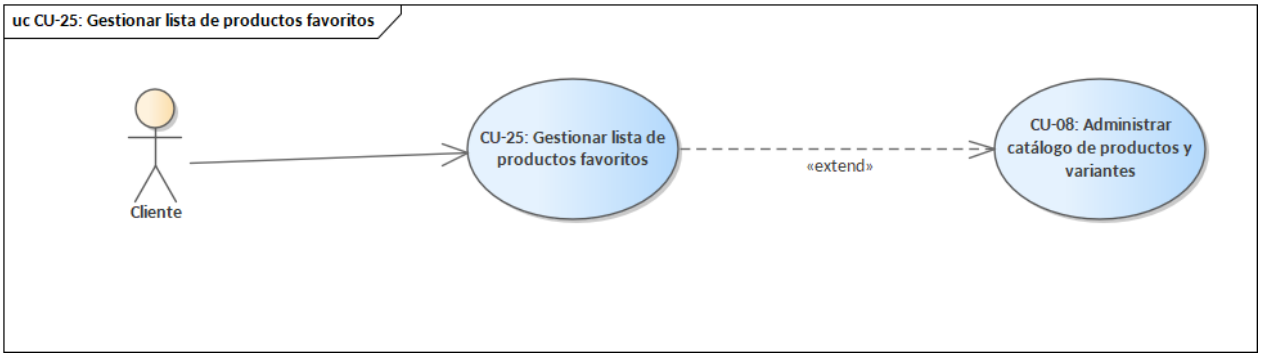
Flujo principal	1. El admin dicta pregunta comercial por voz. 2. Web Speech API transcribe audio a texto. 3. LLM traduce a consulta SQL SELECT sobre PostgreSQL. 4. Se ejecutan datos y se muestran tablas y gráficos.
Post condición	Respuesta analítica visualizada de inmediato.
Excepción	Consulta ambigua o sentencia no-SELECT detectada.

CU24: Publicar y moderar reseñas



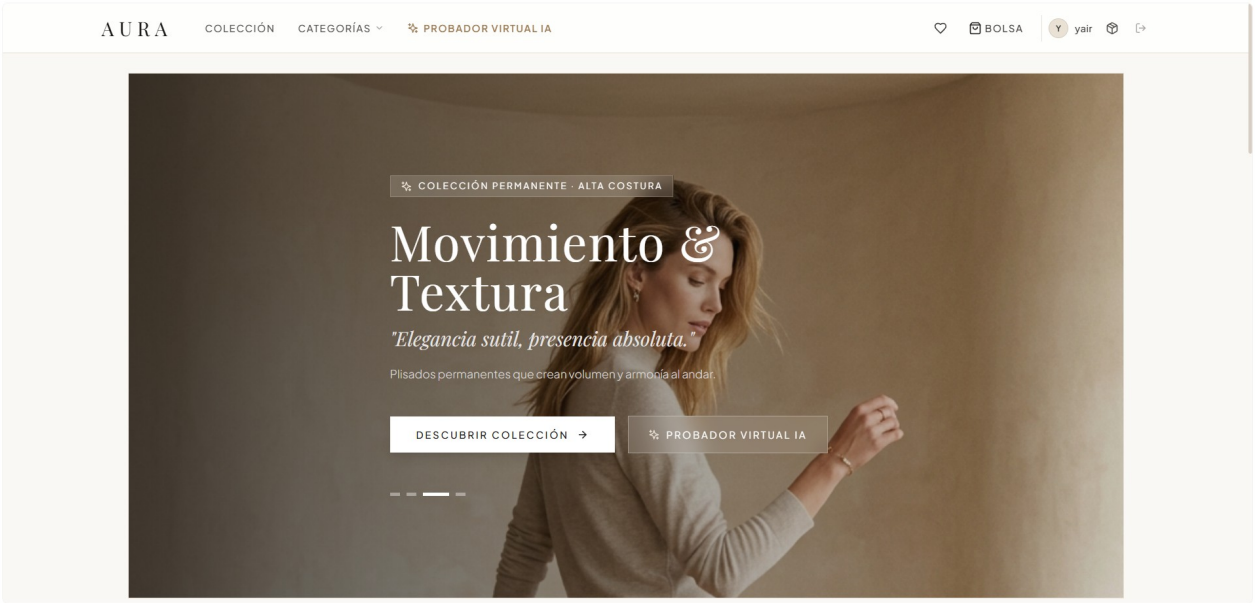
Nombre	Publicar y moderar reseñas
Propósito	Calificación con estrellas de prendas.
Descripción	Permite a clientes con compra entregada valorar de 1 a 5 estrellas y moderar spam.
Actor(es)	A1: Administrador, A2: Cliente
Actor iniciador	A2, A1
Pre condición	Cliente con pedido en estado ENTREGADO.
Flujo principal	1. El cliente entra a la prenda comprada. 2. El sistema valida compra verificada. 3. Cliente califica de 1 a 5 estrellas y comenta. 4. Se actualiza promedio de estrellas en catálogo.
Post condición	Reseña publicada con sello verificado.
Excepción	Cliente no registra compra de dicha prenda.

CU25: Gestionar lista de favoritos



Nombre	Gestionar lista de favoritos
Propósito	Guardar prendas de interés en Wishlist.
Descripción	Permite marcar ropa preferida y migrarla con un clic a la bolsa de compras.
Actor(es)	A2: Cliente
Actor iniciador	A2
Pre condición	Cliente autenticado en la plataforma.
Flujo principal	1. El cliente hace clic en el corazón de la prenda. 2. Se asienta registro en tabla `favorito`. 3. Puede consultar su lista y enviar prendas a compra.
Post condición	Prendas guardadas en la cuenta del cliente.
Excepción	Prenda ya guardada previamente en favoritos.

1.5. Prototipar Interfaz De Usuario



COLECCIÓN PERMANENTE

Colección AURA

Prendas atemporales confeccionadas con materias primas nobles y sastrería de alta precisión.

Buscar por nombre o tejido...

ÁRBOL DE CATEGORÍAS | TODAS | ABRIGOS | CAMISAS | FALDAS | POLERAS | SHORTS | VESTIDOS

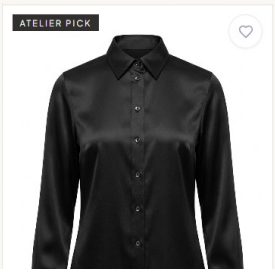
FILTROS | Talla: Todas | Color: Todos | Hasta: $250 | Ordenar por: Más recientes

Mostrando 6 de 6 prendas — Página 1 de 1

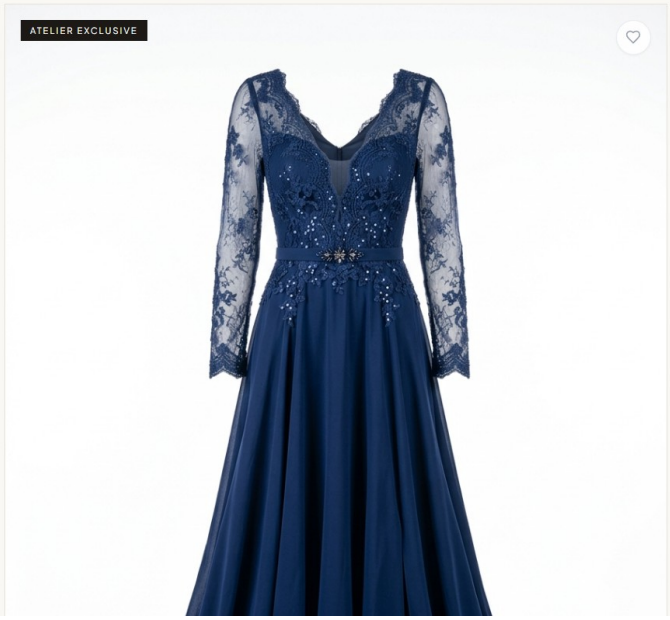








AURA | COLECCIÓN | CATEGORÍAS | PROBADOR VIRTUAL IA | BOLSA | yair



VESTIDOS

Vestido de Noche Elegante

$250.00

USD · Impuestos calculados en checkout

5.0 · 1 opiniones verificadas

COLOR: **AZUL NOCHE**

TALLA — ¿DUDAS CON LA TALLA? PROBAR IA

S | M

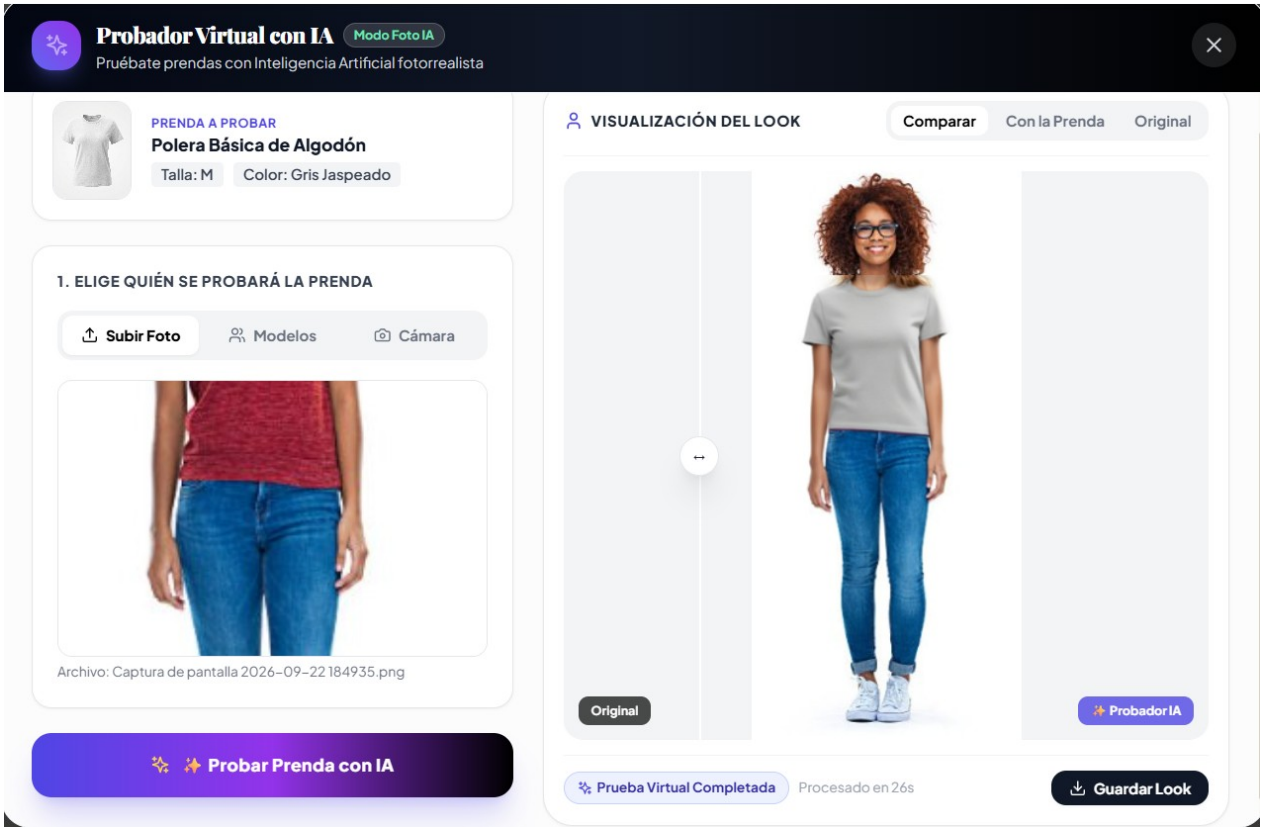
Edición limitada: solo 2 piezas restantes

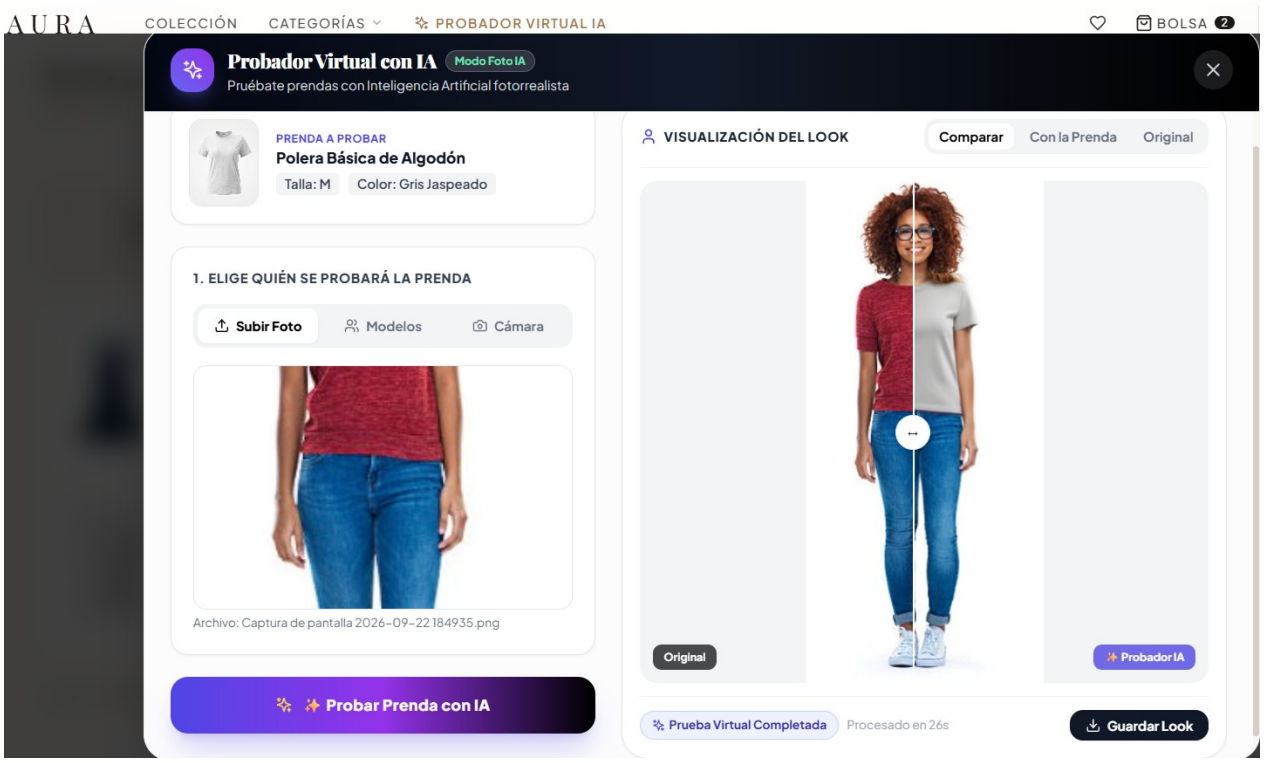
1 | AÑADIR A LA BOLSA — $250.00

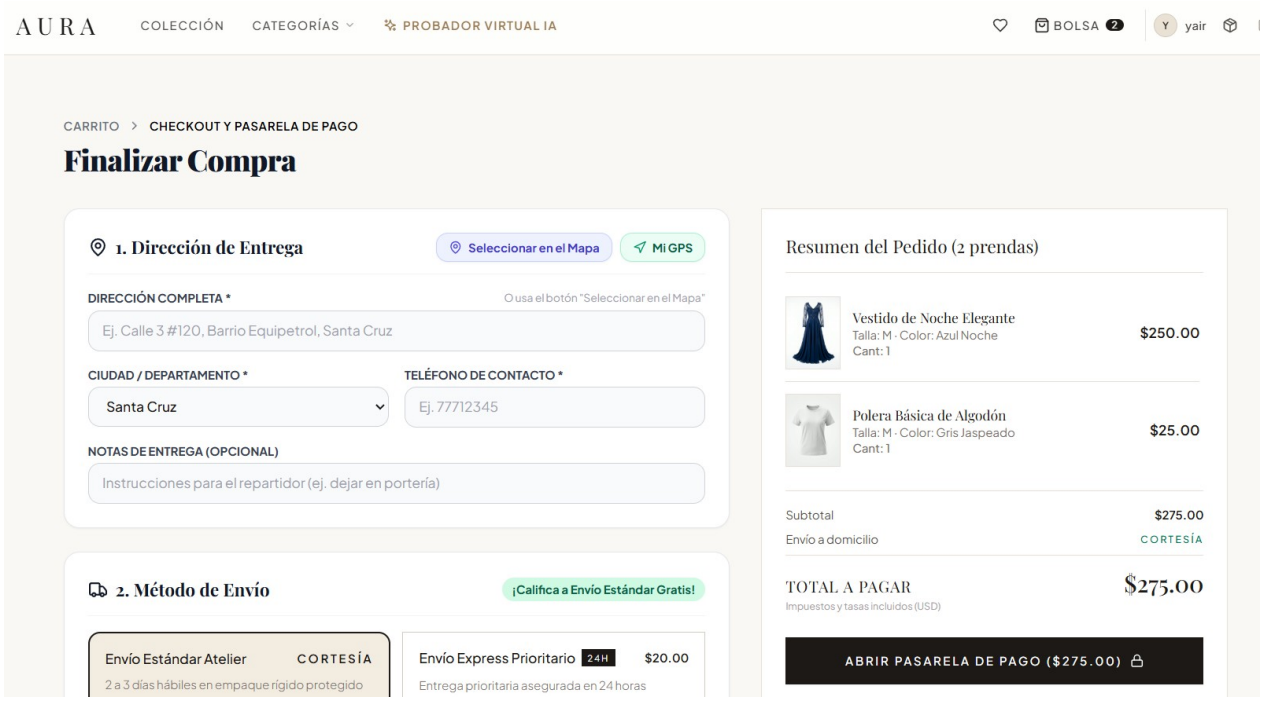
PROBADOR VIRTUAL CON IA — ALTA PRECISIÓN

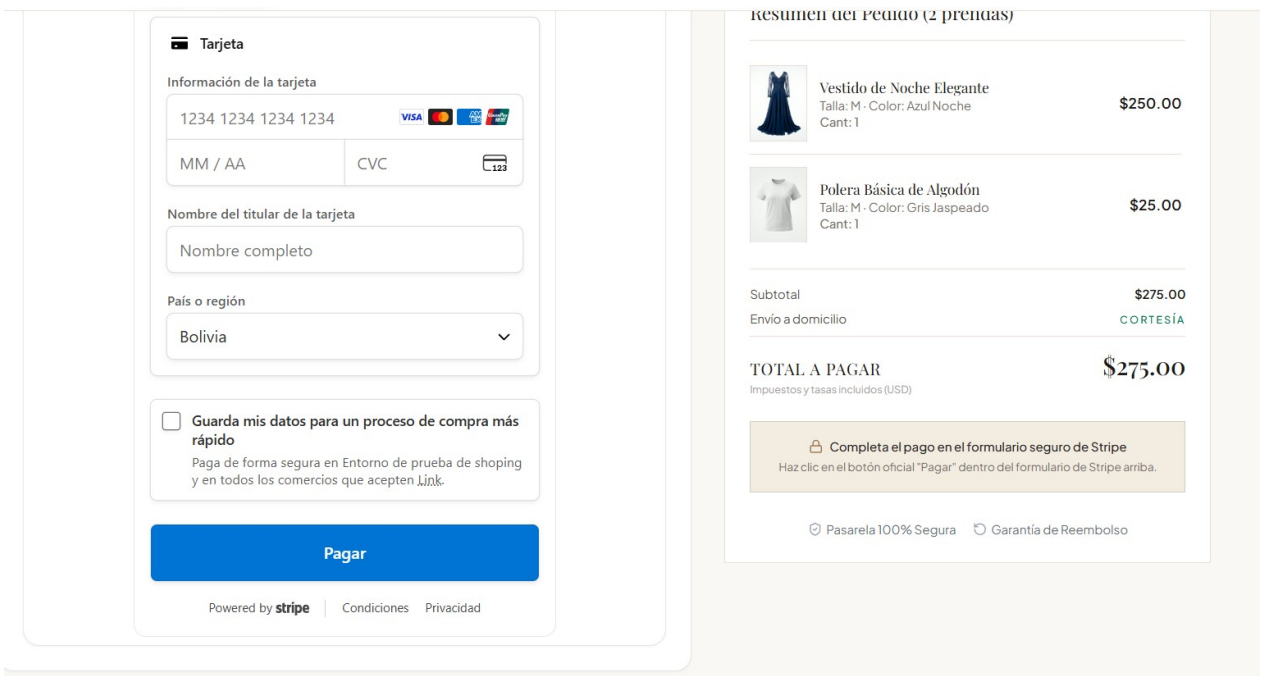
¿Quieres ver cómo te queda antes de ordenar? Sube tu fotografía o elige un modelo para simular la caída exacta de esta pieza en 10 segundos.

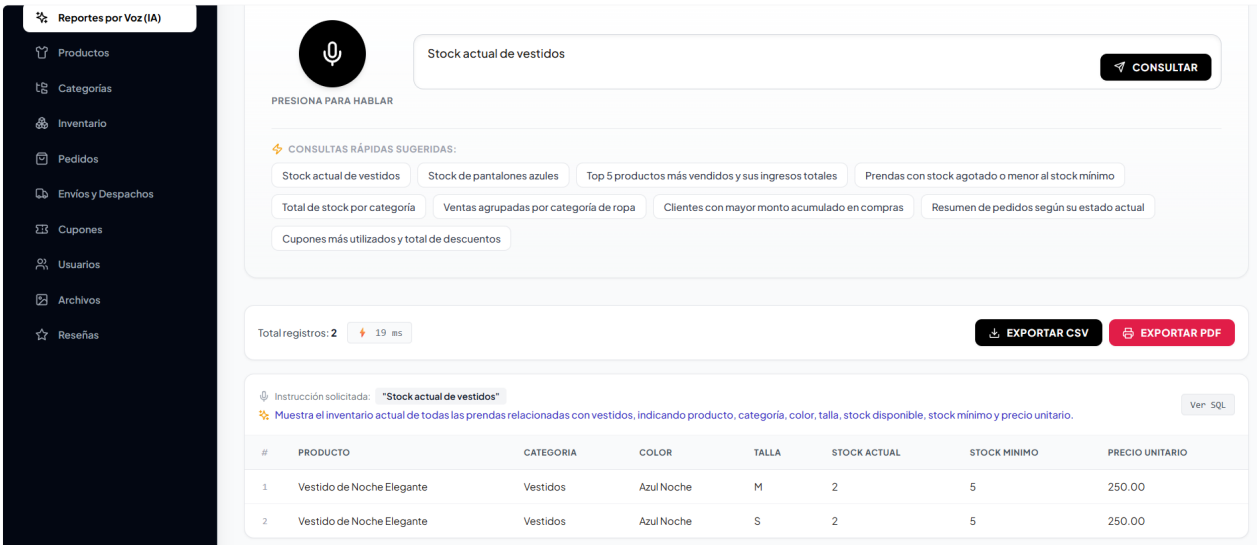
PROBAR EN MI SILUETA


Probador Virtual con IA
Modo Foto IA
Pruébate prendas con Inteligencia Artificial fotorrealista
PRENDA A PROBAR
Polera Básica de Algodón
Talla: M
Color: Gris Jaspeado
1. ELIGE QUIÉN SE PROBARÁ LA PRENDA
Subir Foto
Modelos
Cámara
Archivo: Captura de pantalla 2026-09-22 184935.png
Probar Prenda con IA
VISUALIZACIÓN DEL LOOK
Comparar
Con la Prenda
Original
Original
Probador IA
Prueba Virtual Completada
Procesado en 26s
Guardar Look


AURA
COLECCIÓN
CATEGORÍAS
PROBADOR VIRTUAL IA
BOLSA
2
Probador Virtual con IA
Modo Foto IA
Pruébate prendas con Inteligencia Artificial fotorrealista
PRENDA A PROBAR
Polera Básica de Algodón
Talla: M
Color: Gris Jaspeado
1. ELIGE QUIÉN SE PROBARÁ LA PRENDA
Subir Foto
Modelos
Cámara
Archivo: Captura de pantalla 2026-09-22 184935.png
Probar Prenda con IA
VISUALIZACIÓN DEL LOOK
Comparar
Con la Prenda
Original
Original
Probador IA
Prueba Virtual Completada
Procesado en 26s
Guardar Look


AURA
COLECCIÓN
CATEGORÍAS
PROBADOR VIRTUAL IA
BOLSA 2
yair
CARRITO > CHECKOUT Y PASARELA DE PAGO
Finalizar Compra
1. Dirección de Entrega
Seleccionar en el Mapa
Mi GPS
DIRECCIÓN COMPLETA *
O usa el botón "Seleccionar en el Mapa"
Ej. Calle 3 #120, Barrio Equipetrol, Santa Cruz
CIUDAD / DEPARTAMENTO *
Santa Cruz
TELÉFONO DE CONTACTO *
Ej. 77712345
NOTAS DE ENTREGA (OPCIONAL)
Instrucciones para el repartidor (ej. dejar en portería)
2. Método de Envío
¡Califica a Envío Estándar Gratis!
Envío Estándar Atelier
CORTESÍA
2 a 3 días hábiles en empaque rígido protegido
Envío Express Prioritario
24H
$20.00
Entrega prioritaria asegurada en 24 horas
Resumen del Pedido (2 prendas)
Vestido de Noche Elegante
Talla: M · Color: Azul Noche
Cant: 1
$250.00
Polera Básica de Algodón
Talla: M · Color: Gris Jaspeado
Cant: 1
$25.00
Subtotal
$275.00
Envío a domicilio
CORTESÍA
TOTAL A PAGAR
Impuestos y tasas incluidos (USD)
$275.00
ABRIR PASARELA DE PAGO ($275.00)


Tarjeta
Información de la tarjeta
1234 1234 1234 1234
MM / AA
CVC
Nombre del titular de la tarjeta
Nombre completo
País o región
Bolivia
Guarda mis datos para un proceso de compra más rápido
Paga de forma segura en Entorno de prueba de shoping y en todos los comercios que acepten Link.
Pagar
Powered by stripe
Condiciones
Privacidad
Resumen del Pedido (2 prendas)
Vestido de Noche Elegante
Talla: M · Color: Azul Noche
Cant: 1
$250.00
Polera Básica de Algodón
Talla: M · Color: Gris Jaspeado
Cant: 1
$25.00
Subtotal
$275.00
Envío a domicilio
CORTESÍA
TOTAL A PAGAR
Impuestos y tasas incluidos (USD)
$275.00
Completa el pago en el formulario seguro de Stripe
Haz clic en el botón oficial "Pagar" dentro del formulario de Stripe arriba.
Pasarela 100% Segura
Garantía de Reembolso


Reportes por Voz (IA)
Productos
Categorías
Inventario
Pedidos
Envíos y Despachos
Cupones
Usuarios
Archivos
Reseñas
PRESIONA PARA HABLAR
Stock actual de vestidos
CONSULTAR
CONSULTAS RÁPIDAS SUGERIDAS:
Stock actual de vestidos
Stock de pantalones azules
Top 5 productos más vendidos y sus ingresos totales
Prendas con stock agotado o menor al stock mínimo
Total de stock por categoría
Ventas agrupadas por categoría de ropa
Clientes con mayor monto acumulado en compras
Resumen de pedidos según su estado actual
Cupones más utilizados y total de descuentos
Total registros: 2
19 ms
EXPORTAR CSV
EXPORTAR PDF
Instrucción solicitada:
"Stock actual de vestidos"
Muestra el inventario actual de todas las prendas relacionadas con vestidos, indicando producto, categoría, color, talla, stock disponible, stock mínimo y precio unitario.
Ver SQL
PRODUCTO CATEGORIA COLOR TALLA STOCK ACTUAL STOCK MINIMO PRECIO UNITARIO
1 Vestido de Noche Elegante Vestidos Azul Noche M 2 5 250.00
2 Vestido de Noche Elegante Vestidos Azul Noche S 2 5 250.00

CAPITULO 2: FLUJO DE TRABAJO: ANALISIS

2.1. Análisis de Arquitectura

El Flujo de Análisis descompone el sistema AURA de manera conceptual e independiente de la tecnología mediante la taxonomía BCE (Boundary, Control, Entity).

2.1.1. Identificar paquetes

- Paquete 1: Seguridad y Usuarios (SecurityPkg).
- Paquete 2: Catálogo e Inventario Multi-Variante (CatalogInventoryPkg).
- Paquete 3: Probador Virtual 3D y Realidad Aumentada (VirtualTryOnPkg).
- Paquete 4: Ventas, Carrito y Pasarela Stripe (SalesPaymentPkg).
- Paquete 5: Logística, Reseñas y Analítica Inteligente (LogisticsAnalyticsPkg).

2.1.2. Relacionar paquetes y casos de uso

- SecurityPkg: CU01, CU02, CU03, CU04, CU05, CU06, CU07.
- CatalogInventoryPkg: CU08, CU10, CU11, CU19, CU25.
- VirtualTryOnPkg: CU09.
- SalesPaymentPkg: CU12, CU13, CU14, CU15, CU16.
- LogisticsAnalyticsPkg: CU17, CU18, CU20, CU21, CU22, CU23, CU24.

2.1.3. Vista de casos de uso (encapsulamiento)

Cada paquete expone interfaces de frontera que encapsulan su lógica interna, protegiendo las entidades

contra accesos directos no coordinados.

2.2. Análisis de caso de uso Tablas resumen de los ciclos

2.3. Ciclo #1

ID	Caso de Uso	Descripción del Análisis
CU01	Iniciar sesión	Autentica credenciales en `Usuario`, emite Access Token y Refresh Token con `Ctrl_Auth`.
CU02	Cerrar sesión	Invalida sesión y tokens activos desde la interfaz `UI_Logout`.
CU03	Gestionar usuario	Permite al administrador crear, actualizar, consultar y deshabilitar cuentas en `Usuario`.
CU04	Modificar contraseña	Verifica hash actual y actualiza clave con `Ctrl_Seguridad`.
CU05	Administrar libreta de direcciones	Permite registrar y seleccionar dirección de entrega en entidad `Direccion`.
CU06	Bloquear / Desbloquear cuenta	Actualiza bandera de actividad en `Usuario` y revoca permisos.
CU07	Asignar y gestionar roles	Asigna roles RBAC vinculando `Usuario` con la entidad `Rol`.

2.4. Ciclo #2

ID	Caso de Uso	Descripción del Análisis
CU08	Administrar catálogo y variantes	Permite crear prendas y variantes vinculando `Producto`, `Talla`, `Color` y `ProductoVariante`.
CU10	Administrar categorías jerárquicas	Mantiene árbol de clasificación en la entidad `Categoria` con slugs amigables.
CU11	Gestionar existencias e inventario	Actualiza stock físico atómico en `ProductoVariante` y registra movimientos en `Kardex`.
CU19	Gestionar multimedia WebP	Optimiza y comprime fotos en `Ctrl_Multimedia` asociándolas a `ImagenProducto`.
CU25	Gestionar lista de favoritos	Registra prendas deseadas en la entidad `Favorito` para compra posterior.

2.5. Ciclo #3

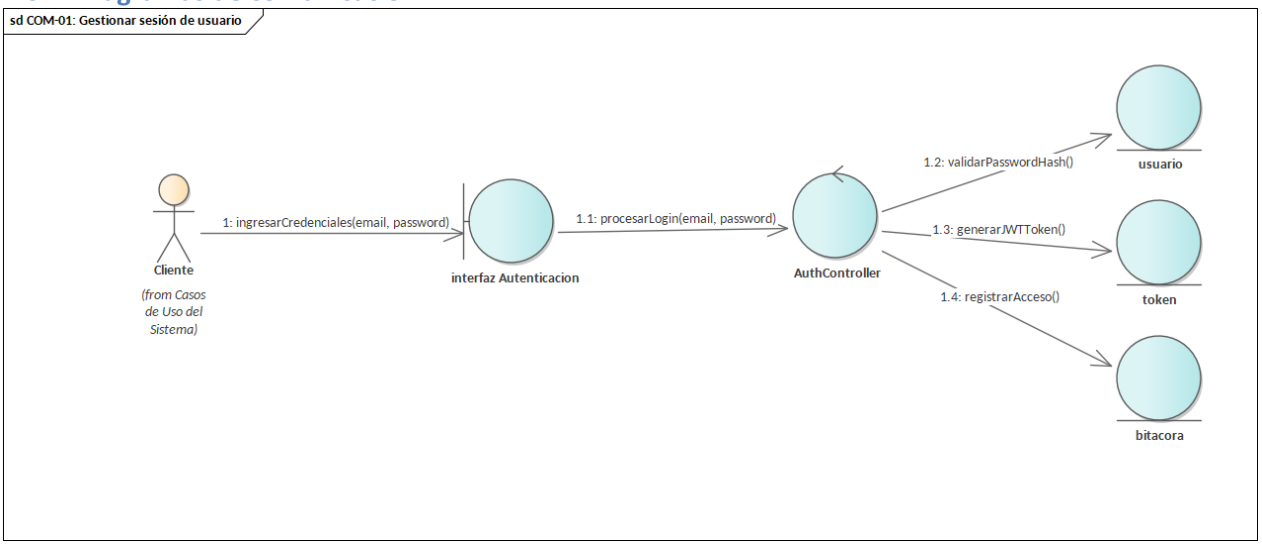
ID	Caso de Uso	Descripción del Análisis
CU09	Probar prendas en probador 3D y AR	Orquesta mallas GLB en `Ctrl_ThreeJs` y tracking corporal en tiempo real con `Ctrl_MediaPipe`.
CU12	Gestionar carrito de compras	Coordina la bolsa de compras en memoria

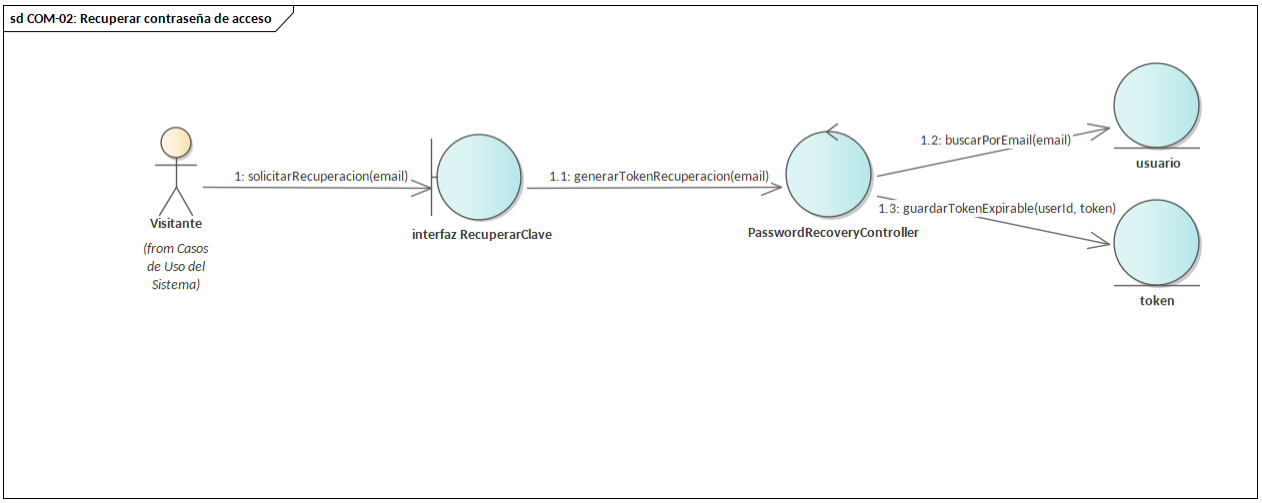
		reactiva con `Ctrl_Carrito`.
CU13	Sincronizar y persistir carrito	Fusiona carrito anónimo en `localStorage` con la entidad `Carrito` en base de datos.
CU24	Publicar y moderar reseñas	Valida compra verificada en `NotaVenta` y publica calificación en `Resena`.

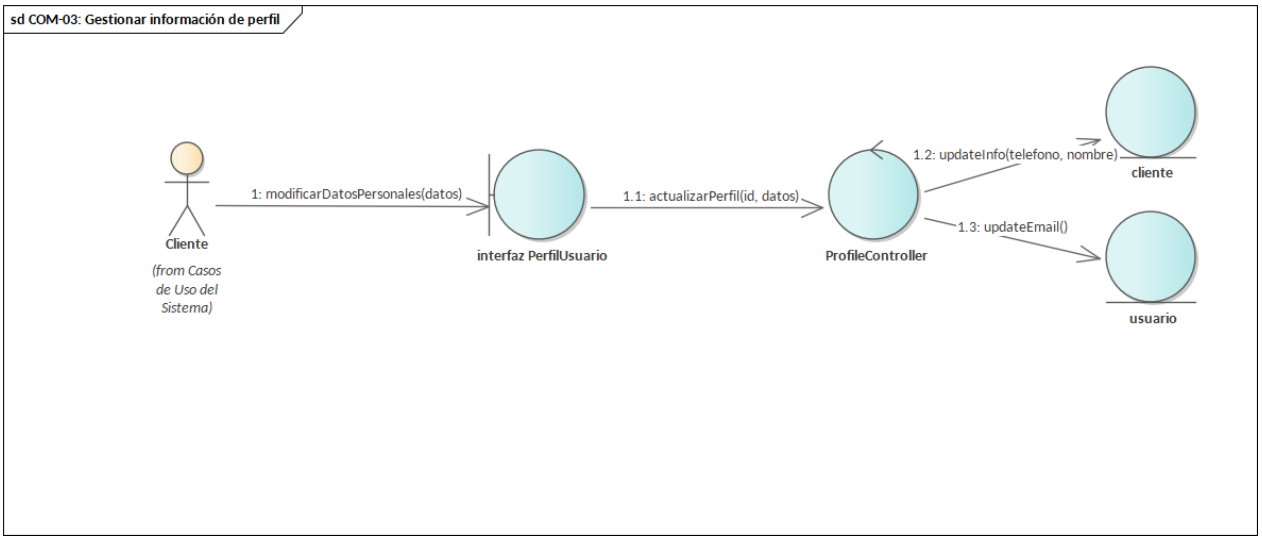
2.6. Ciclo #4

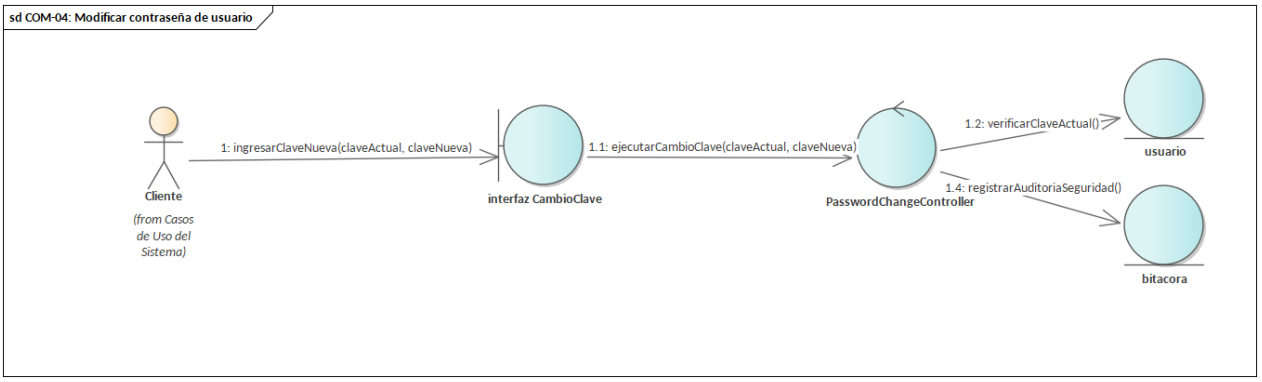
ID	Caso de Uso	Descripción del Análisis
CU14	Gestionar pedidos y estados	Trazabilidad de la orden en `NotaVenta` y avance de estados de despacho.
CU15	Procesar pago con Stripe	Crea `PaymentIntent`, valida tarjeta con Stripe y rebaja inventario atómicamente.
CU16	Procesar reembolso de pago	Revierte fondos mediante Stripe API y restituye existencias a `ProductoVariante`.
CU17	Gestionar métodos de envío	Calcula tarifa de flete en `MetodoEnvio` y asigna número de guía de transporte.
CU18	Administrar cupones de descuento	Valida vigencia y tope en `Cupon` deduciendo descuento sobre el subtotal.
CU20	Enviar correo de bienvenida	Despacha plantilla HTML transaccional de bienvenida vía servidor SMTP.
CU21	Enviar correo de confirmación	Envía recibo formal y detalle de prendas adquiridas al cliente por correo.
CU22	Generar reportes estadísticos	Consolida métricas de ingresos y exporta resúmenes ejecutivos en PDF/CSV.
CU23	Generar reportes por voz con IA	Transcribe voz del directivo y traduce prompt a consulta SQL con `Ctrl_NLP`.

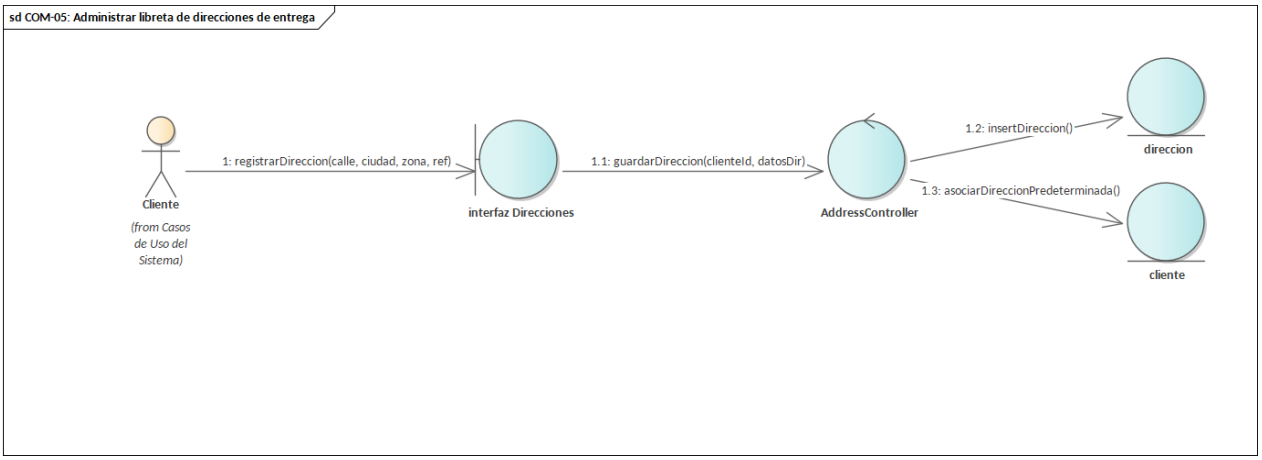
2.6.1. Diagramas de comunicación

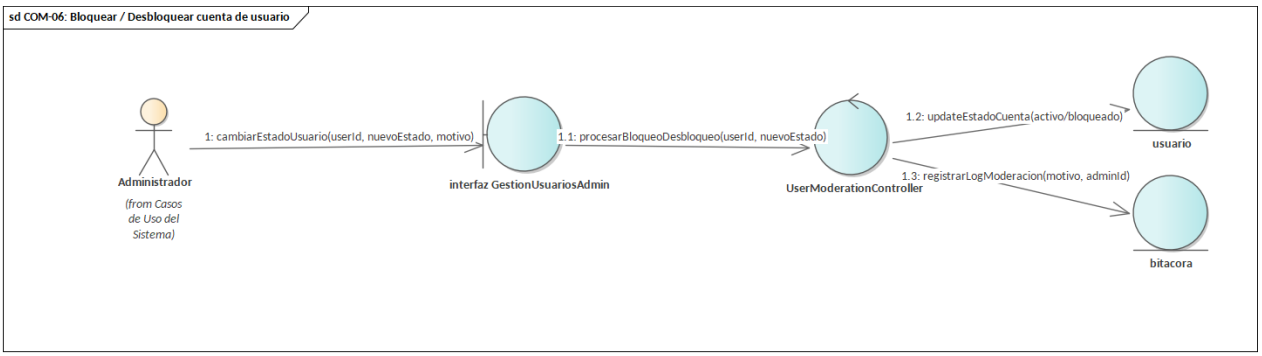


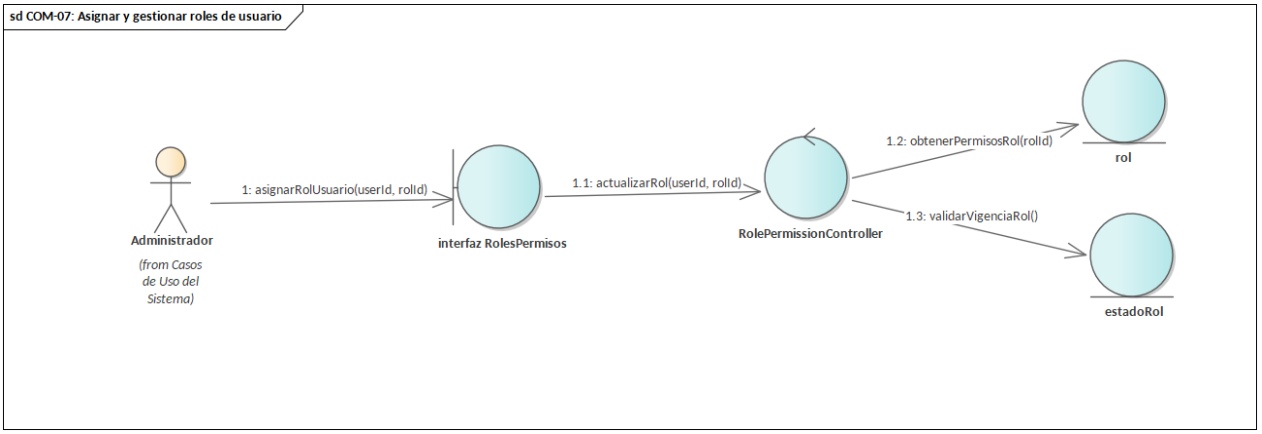

sd COM-02: Recuperar contraseña de acceso
Visitante
(from Casos de Uso del Sistema)
1: solicitarRecuperacion(email)
interfaz RecuperarClave
1.1: generarTokenRecuperacion(email)
PasswordRecoveryController
1.2: buscarPorEmail(email)
usuario
1.3: guardarTokenExpirable(userId, token)
token

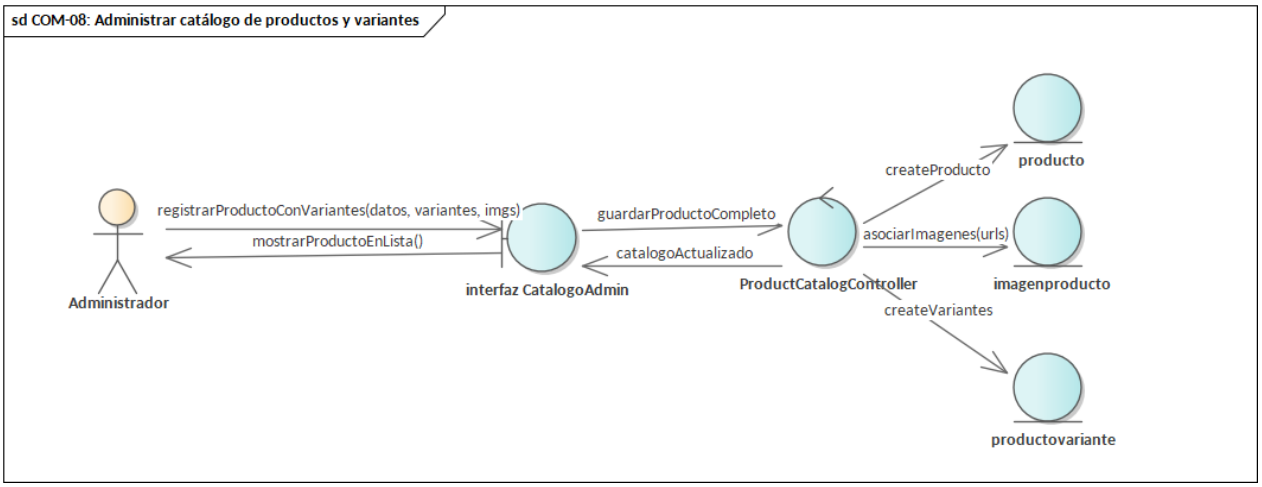

sd COM-03: Gestionar información de perfil
Cliente
(from Casos de Uso del Sistema)
1: modificarDatosPersonales(datos)
interfaz PerfilUsuario
1.1: actualizarPerfil(id, datos)
ProfileController
1.2: updateInfo(telefono, nombre)
cliente
1.3: updateEmail()
usuario

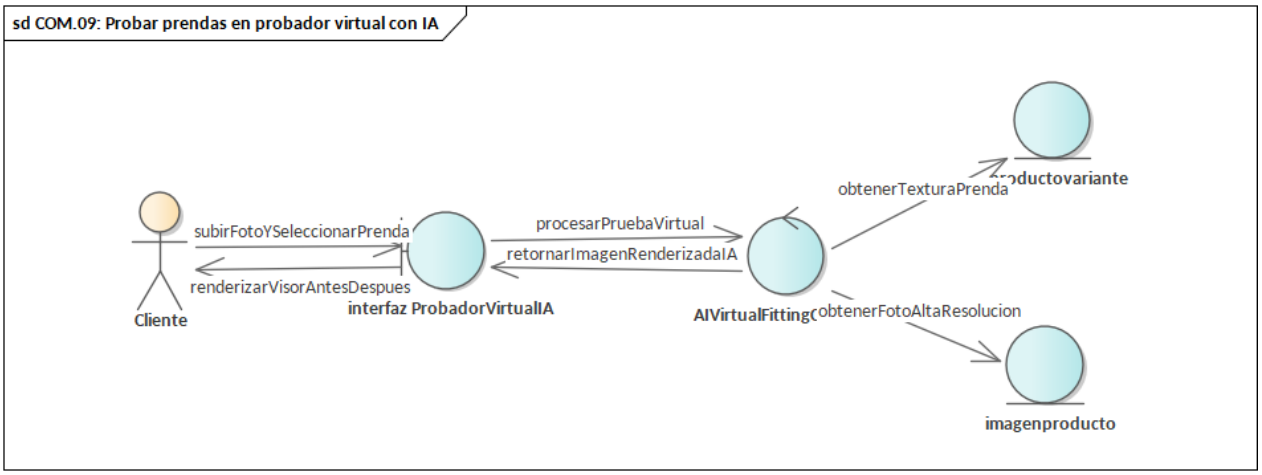

sd COM-04: Modificar contraseña de usuario
Cliente
(from Casos de Uso del Sistema)
1: ingresarClaveNueva(claveActual, claveNueva)
interfaz CambioClave
1.1: ejecutarCambioClave(claveActual, claveNueva)
PasswordChangeController
1.2: verificarClaveActual()
usuario
1.4: registrarAuditoriaSeguridad()
bitacora

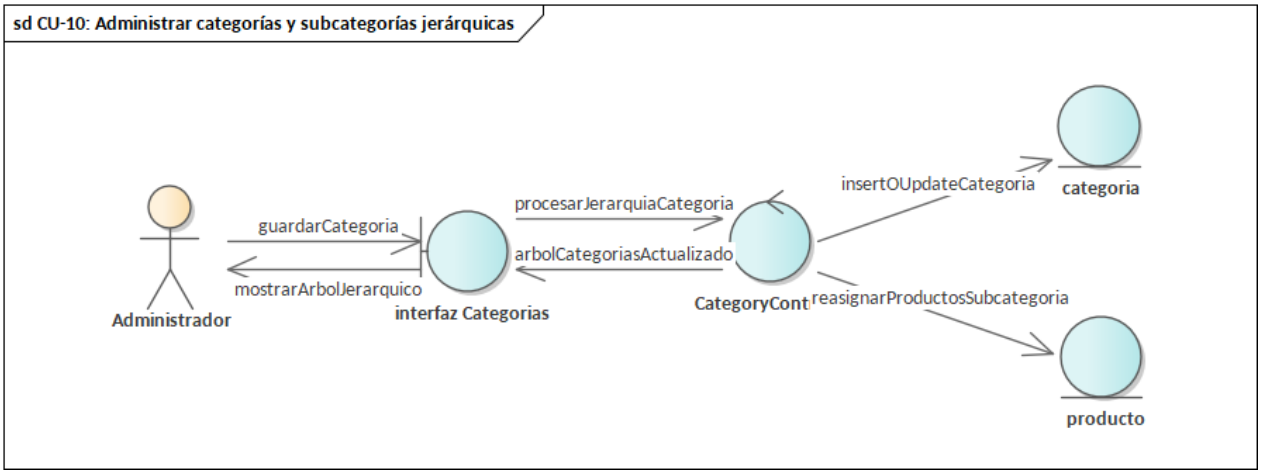

sd COM-05: Administrar libreta de direcciones de entrega
Cliente
(from Casos de Uso del Sistema)
1: registrarDireccion(calle, ciudad, zona, ref)
interfaz Direcciones
1.1: guardarDireccion(clienteId, datosDir)
AddressController
1.2: insertDireccion()
direccion
1.3: asociarDireccionPredeterminada()
cliente

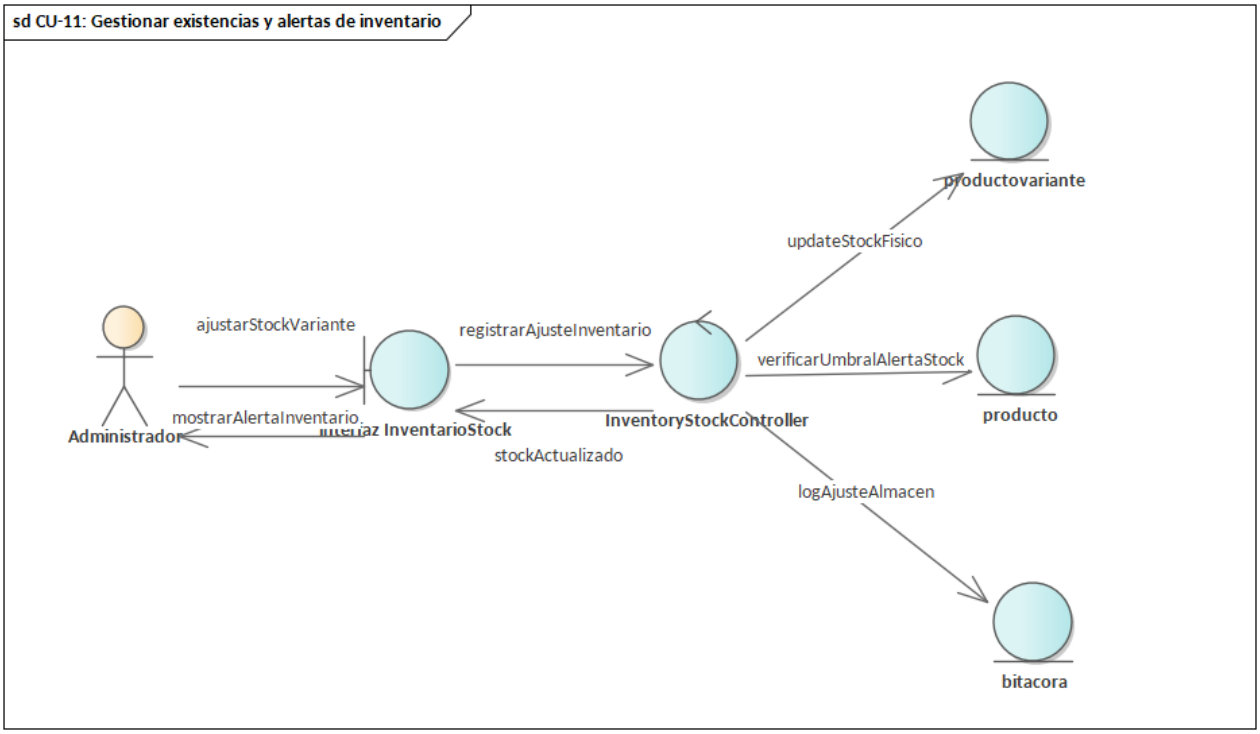

sd COM-06: Bloquear / Desbloquear cuenta de usuario
Administrador
(from Casos de Uso del Sistema)
1: cambiarEstadoUsuario(userId, nuevoEstado, motivo)
interfaz GestionUsuariosAdmin
1.1: procesarBloqueoDesbloqueo(userId, nuevoEstado)
UserModerationController
1.2: updateEstadoCuenta(activo/bloqueado)
usuario
1.3: registrarLogModeracion(motivo, adminId)
bitacora

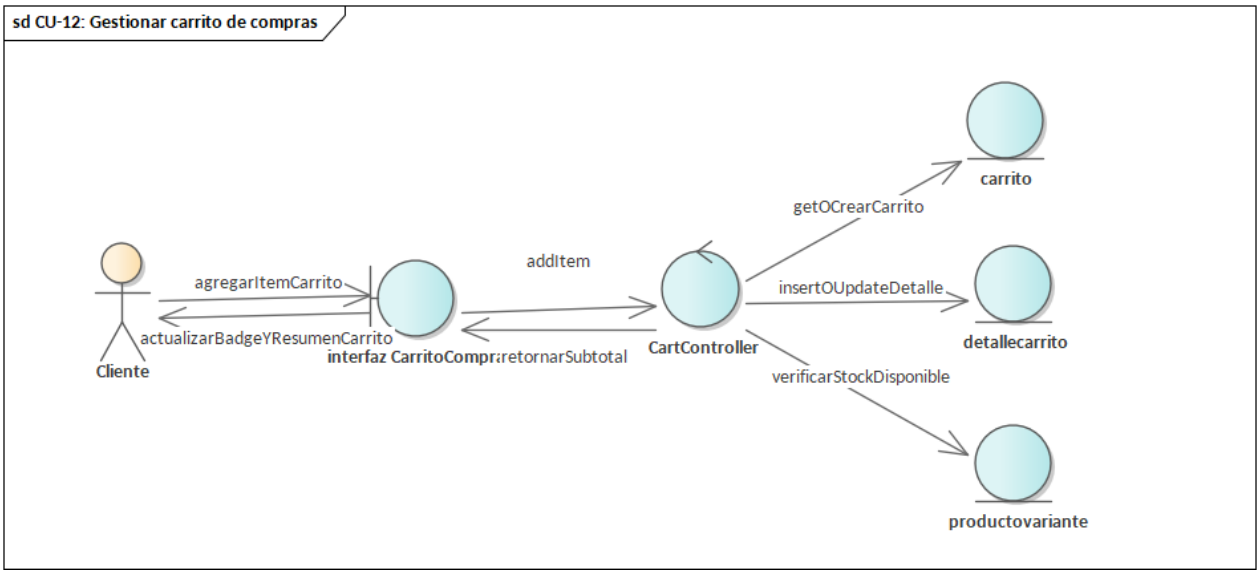

sd COM-07: Asignar y gestionar roles de usuario
Administrador
(from Casos de Uso del Sistema)
1: asignarRolUsuario(userId, rolId)
interfaz RolesPermisos
1.1: actualizarRol(userId, rolId)
RolePermissionController
1.2: obtenerPermisosRol(rolId)
rol
1.3: validarVigenciaRol()
estadoRol

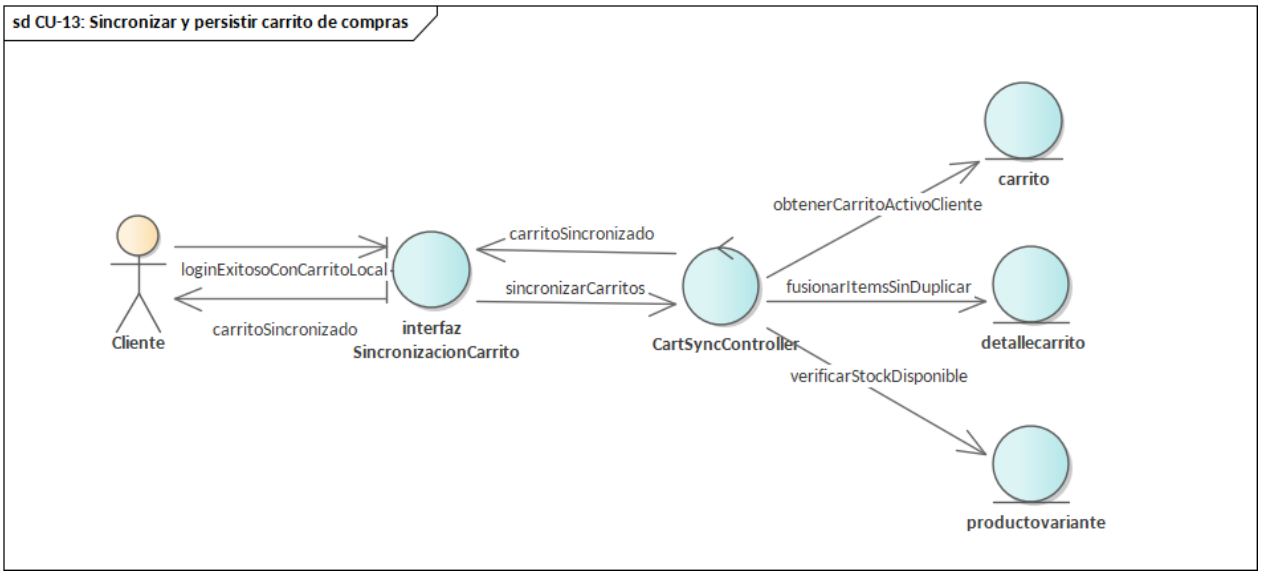

sd COM-08: Administrar catálogo de productos y variantes
Administrador
registrarProductoConVariantes(datos, variantes, imgs)
mostrarProductoEnLista()
interfaz CatalogoAdmin
guardarProductoCompleto
catalogoActualizado
ProductCatalogController
createProducto
producto
asociarImagenes(urls)
imagenproducto
createVariantes
productovariante

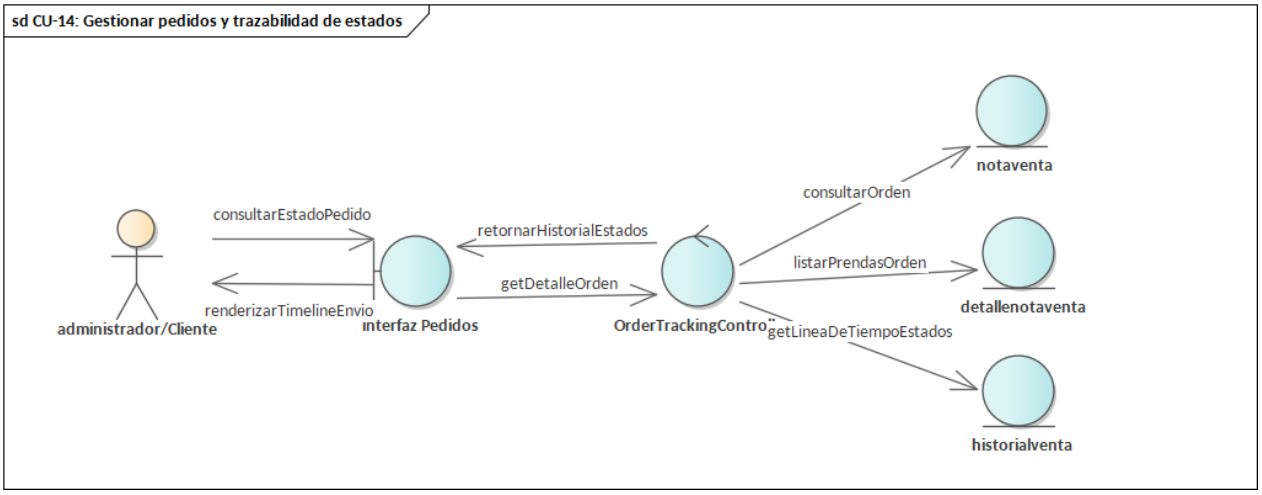

sd COM.09: Probar prendas en probador virtual con IA
Cliente
subirFotoYSeleccionarPrenda
renderizarVisorAntesDespues
interfaz ProbadorVirtualIA
procesarPruebaVirtual
retornarImagenRenderizadaIA
AIVirtualFitting
obtenerTexturaPrenda
productovariante
obtenerFotoAltaResolucion
imagenproducto

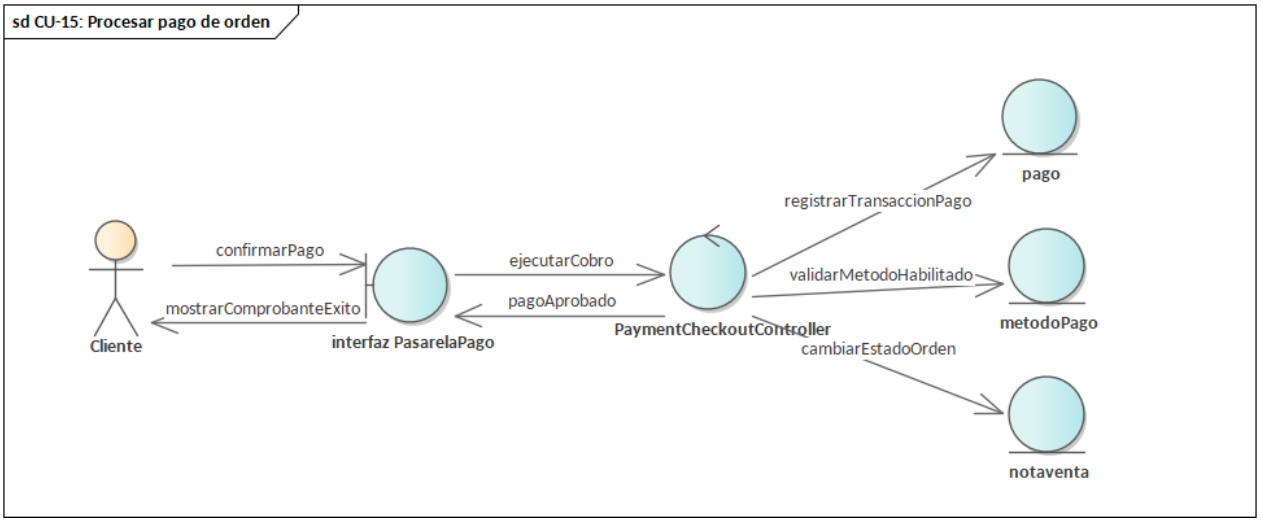

sd CU-10: Administrar categorías y subcategorías jerárquicas
Administrador
guardarCategoria
mostrarArbolJerarquico
interfaz Categorias
procesarJerarquiaCategoria
arbolCategoriasActualizado
CategoryCont
insertOUpdateCategoria
categoria
reasignarProductosSubcategoria
producto

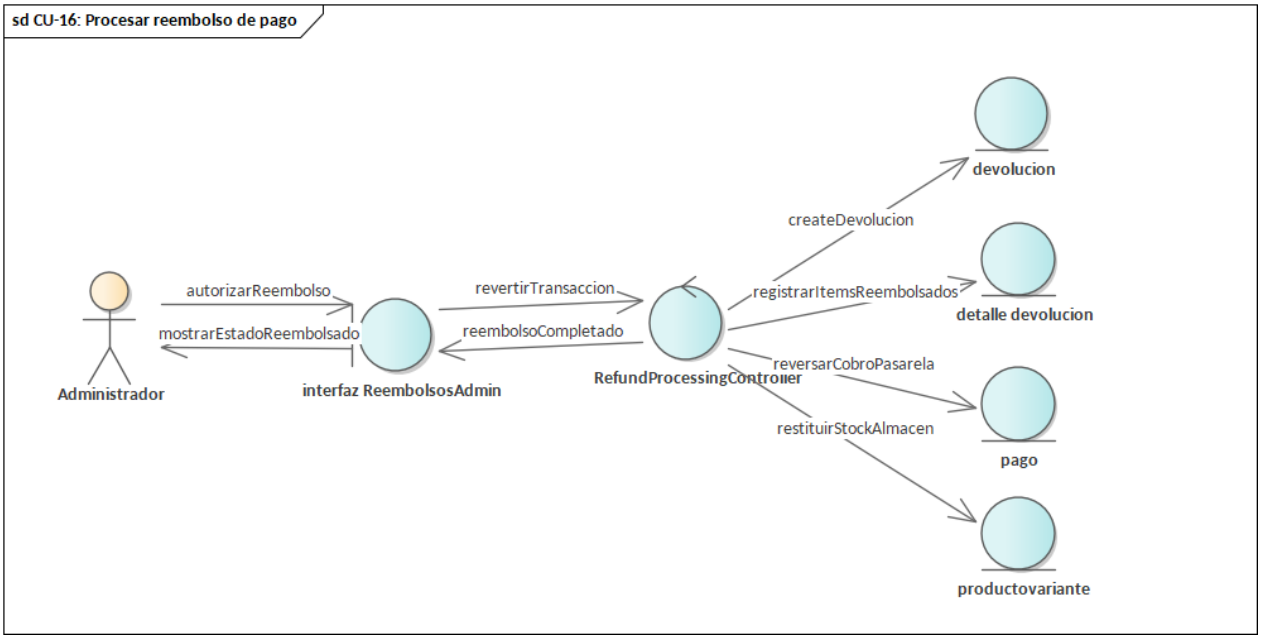

sd CU-11: Gestionar existencias y alertas de inventario
productovariante
updateStockFisico
ajustarStockVariante
registrarAjusteInventario
verificarUmbralAlertaStock
producto
Administrador
mostrarAlertaInventario
InventoryStockController
stockActualizado
logAjusteAlmacen
bitacora

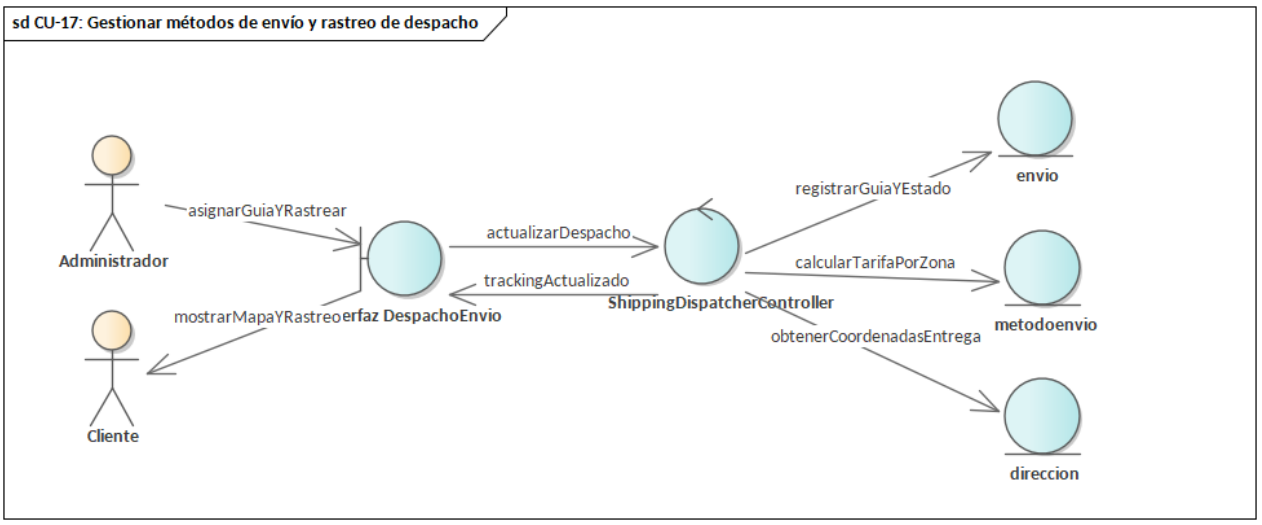

sd CU-12: Gestionar carrito de compras
carrito
getOCrearCarrito
addItem
agregarItemCarrito
insertOUpdateDetalle
detallecarrito
actualizarBadgeYResumenCarrito
CartController
Cliente
retornarSubtotal
verificarStockDisponible
productovariante

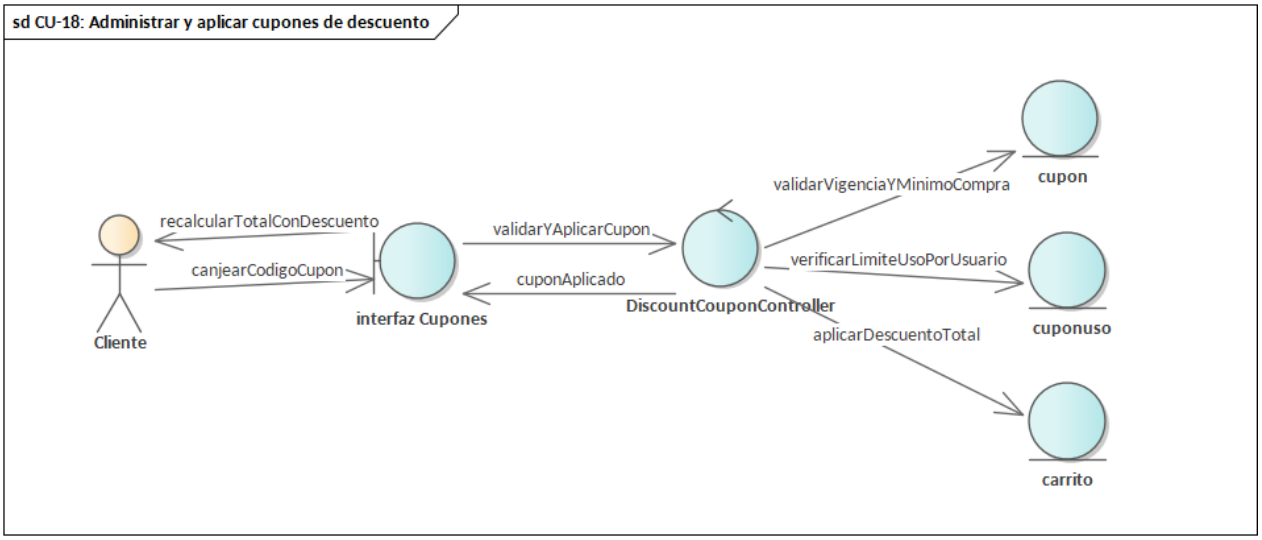

sd CU-13: Sincronizar y persistir carrito de compras
Cliente
loginExitosoConCarritoLocal
carritoSincronizado
interfaz SincronizacionCarrito
carritoSincronizado
sincronizarCarritos
CartSyncController
obtenerCarritoActivoCliente
carrito
fusionarItemsSinDuplicar
detallecarrito
verificarStockDisponible
productovariante

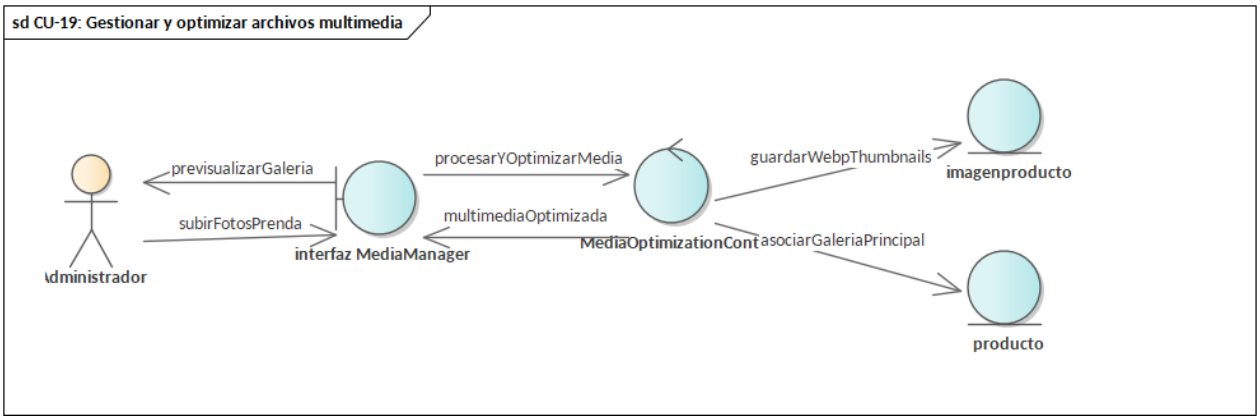

sd CU-14: Gestionar pedidos y trazabilidad de estados
administrador/Cliente
consultarEstadoPedido
renderizarTimelineEnvio
interfaz Pedidos
retornarHistorialEstados
getDetalleOrden
OrderTrackingContro...
consultarOrden
notaventa
listarPrendasOrden
detallenotaventa
getLineaDeTiempoEstados
historialventa

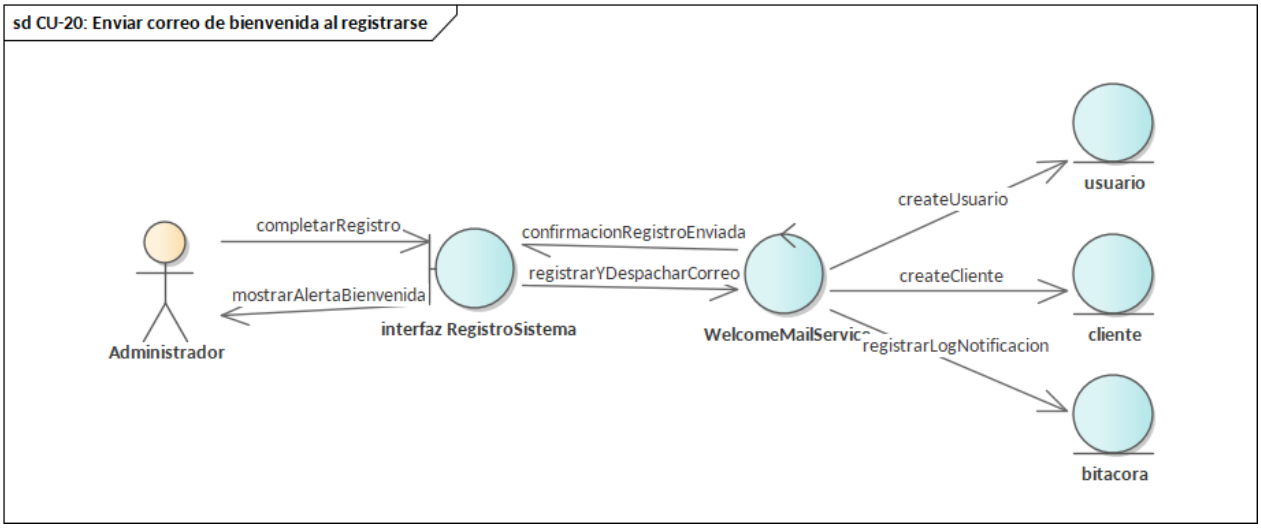

sd CU-15: Procesar pago de orden
Cliente
confirmarPago
mostrarComprobanteExito
interfaz PasarelaPago
ejecutarCobro
pagoAprobado
PaymentCheckoutController
registrarTransaccionPago
pago
validarMetodoHabilitado
metodoPago
cambiarEstadoOrden
notaventa

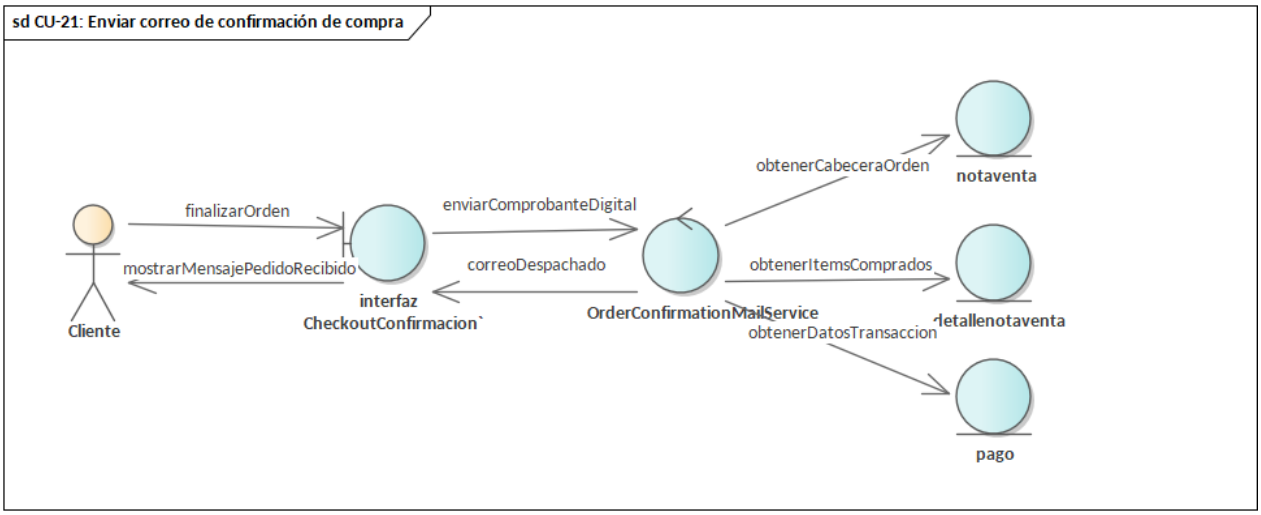

sd CU-16: Procesar reembolso de pago
devolucion
createDevolucion
detalle devolucion
registrarItemsReembolsados
autorizarReembolso
revertirTransaccion
mostrarEstadoReembolsado
reembolsoCompletado
reversarCobroPasarela
Administrador
interfaz ReembolsosAdmin
RefundProcessingController
pago
restituirStockAlmacen
productovariante

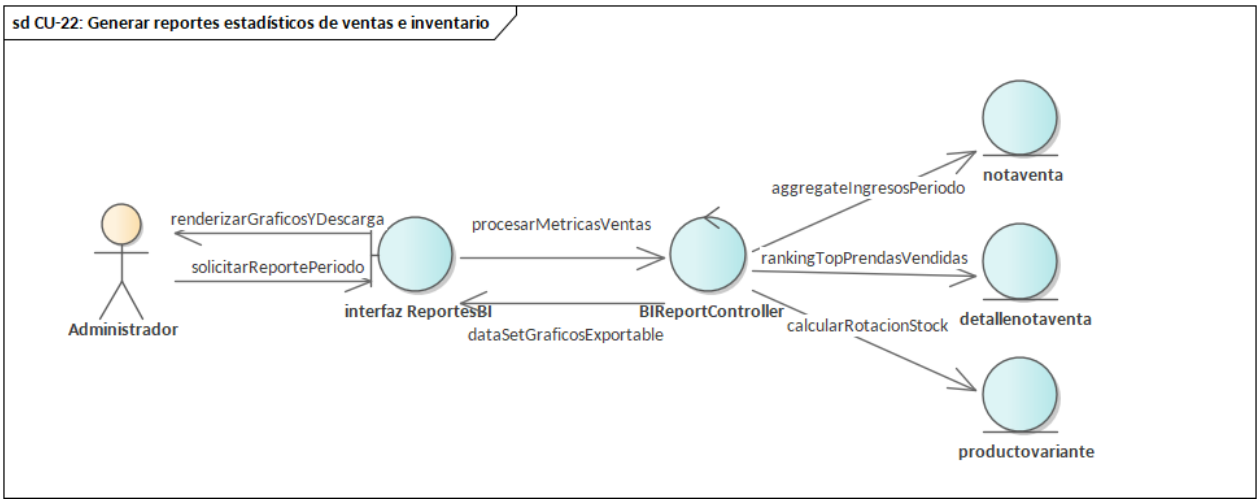

sd CU-17: Gestionar métodos de envío y rastreo de despacho
envio
registrarGuiaYEstado
asignarGuiaYRastrear
actualizarDespacho
calcularTarifaPorZona
Administrador
trackingActualizado
ShippingDispatcherController
metodoenvio
mostrarMapaYRastreo
erfaz DespachoEnvio
obtenerCoordenadasEntrega
Cliente
direccion

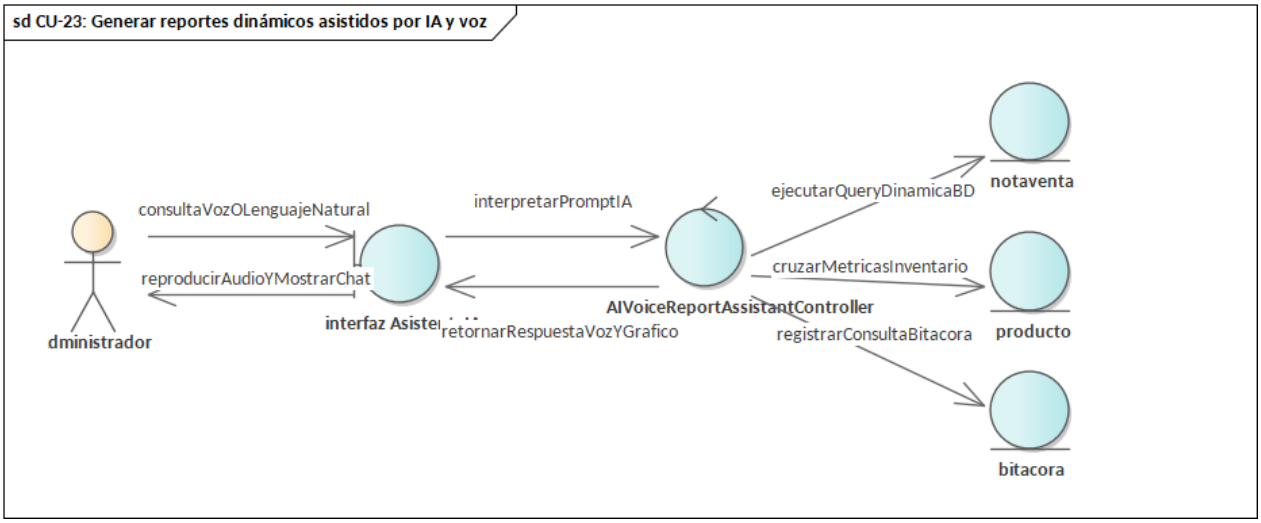

sd CU-18: Administrar y aplicar cupones de descuento
cupon
validarVigenciaYMinimoCompra
recalcularTotalConDescuento
validarYAplicarCupon
verificarLimiteUsoPorUsuario
canjearCodigoCupon
cuponAplicado
cuponuso
Cliente
interfaz Cupones
DiscountCouponController
aplicarDescuentoTotal
carrito

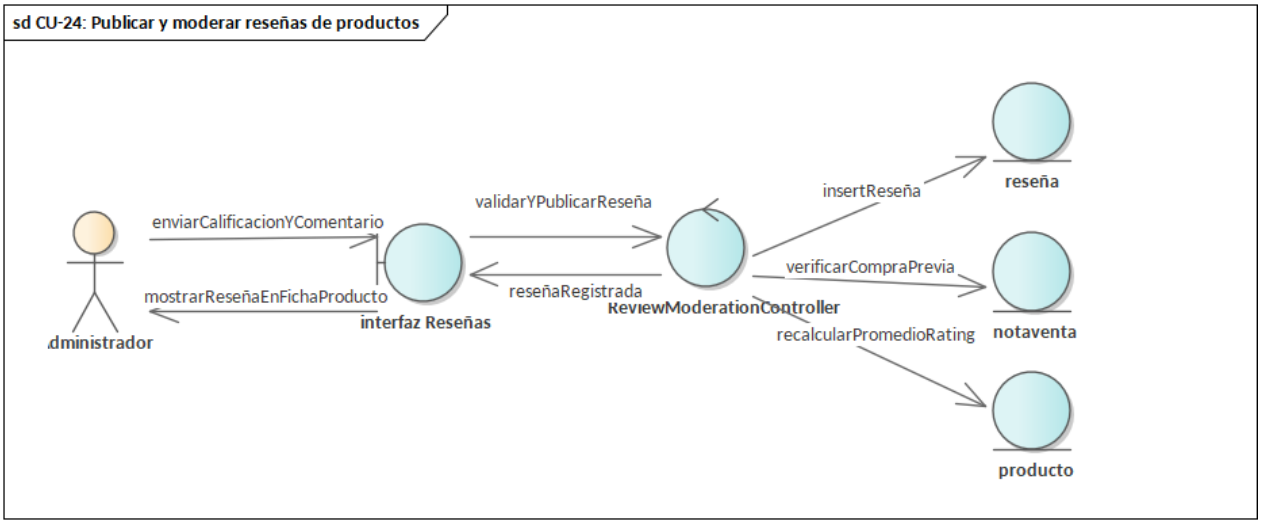

sd CU-19: Gestionar y optimizar archivos multimedia
previsualizarGaleria
procesarYOptimizarMedia
guardarWebpThumbnails
imagenproducto
subirFotosPrenda
multimediaOptimizada
asociarGaleriaPrincipal
producto
interfaz MediaManager
MediaOptimizationCont

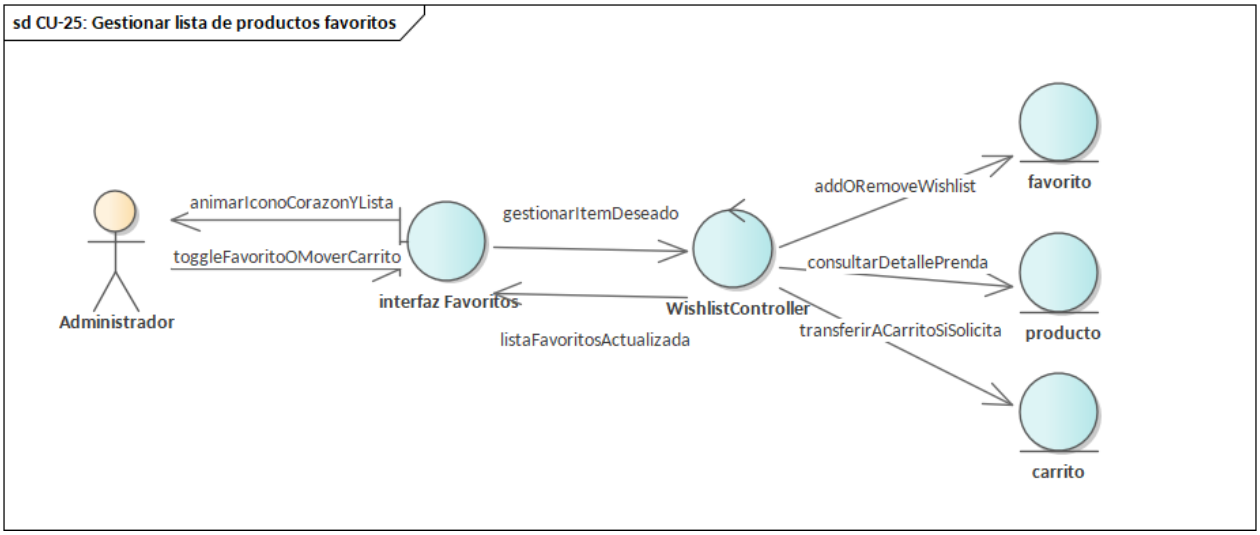

sd CU-20: Enviar correo de bienvenida al registrarse
completarRegistro
confirmacionRegistroEnviada
createUsuario
usuario
registrarYDespacharCorreo
createCliente
cliente
mostrarAlertaBienvenida
registrarLogNotificacion
Administrador
interfaz RegistroSistema
bitacora


sd CU-21: Enviar correo de confirmación de compra
obtenerCabeceraOrden
notaventa
finalizarOrden
enviarComprobanteDigital
mostrarMensajePedidoRecibido
correoDespachado
obtenerItemsComprados
obtenerDatosTransaccion
pago
Cliente
interfaz CheckoutConfirmacion`
OrderConfirmationMailService

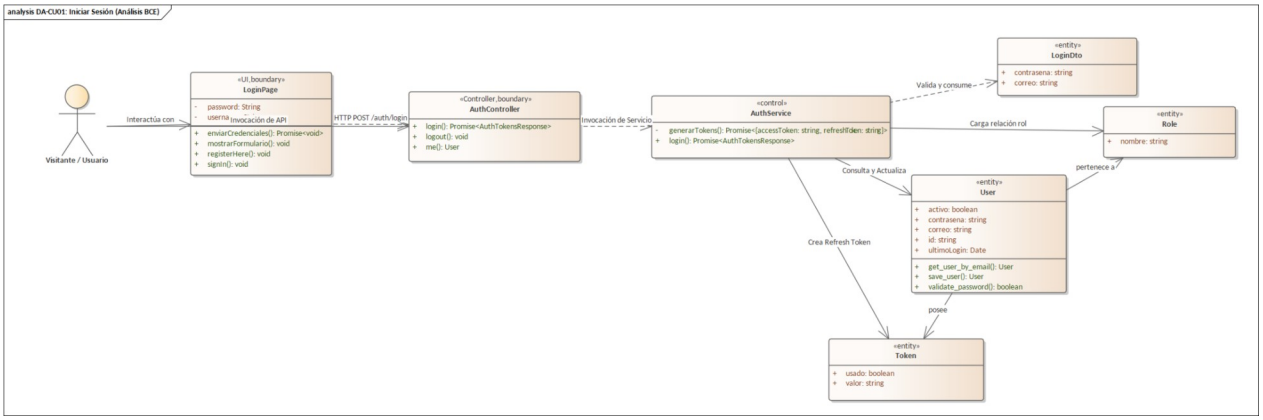

sd CU-22: Generar reportes estadísticos de ventas e inventario
Administrador
renderizarGraficosYDescarga
solicitarReportePeriodo
interfaz ReportesBI
procesarMetricasVentas
dataSetGraficosExportable
BIReportController
aggregateIngresosPeriodo
rankingTopPrendasVendidas
calcularRotacionStock
notaventa
detallenotaventa
productovariante

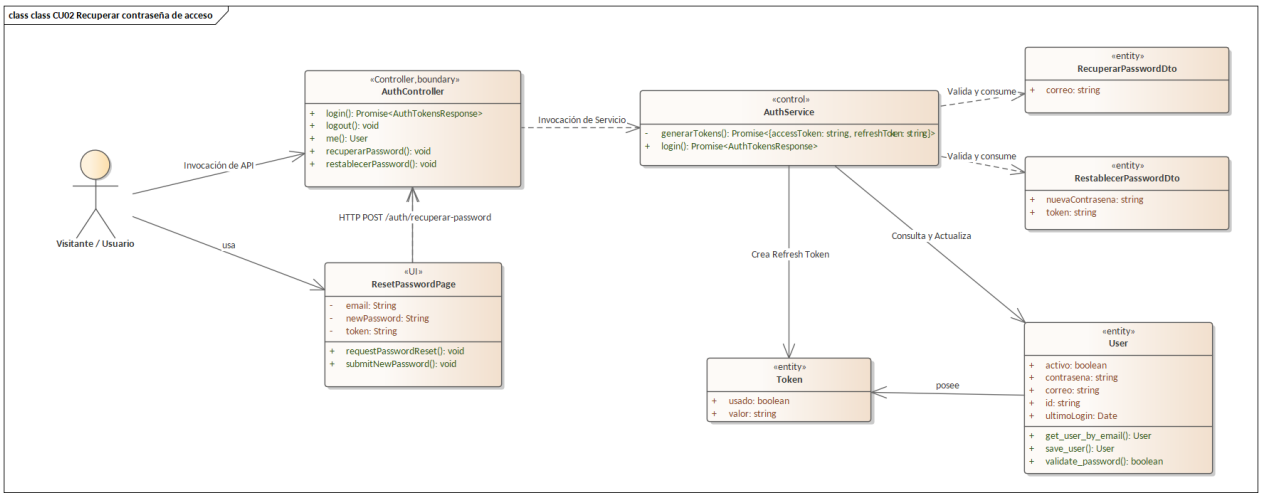

sd CU-23: Generar reportes dinámicos asistidos por IA y voz
dministrador
consultaVozOLenguajeNatural
reproducirAudioYMostrarChat
interfaz Asiste
interpretarPromptIA
retornarRespuestaVozYGrafico
AIVoiceReportAssistantController
ejecutarQueryDinamicaBD
cruzarMetricasInventario
registrarConsultaBitacora
notaventa
producto
bitacora

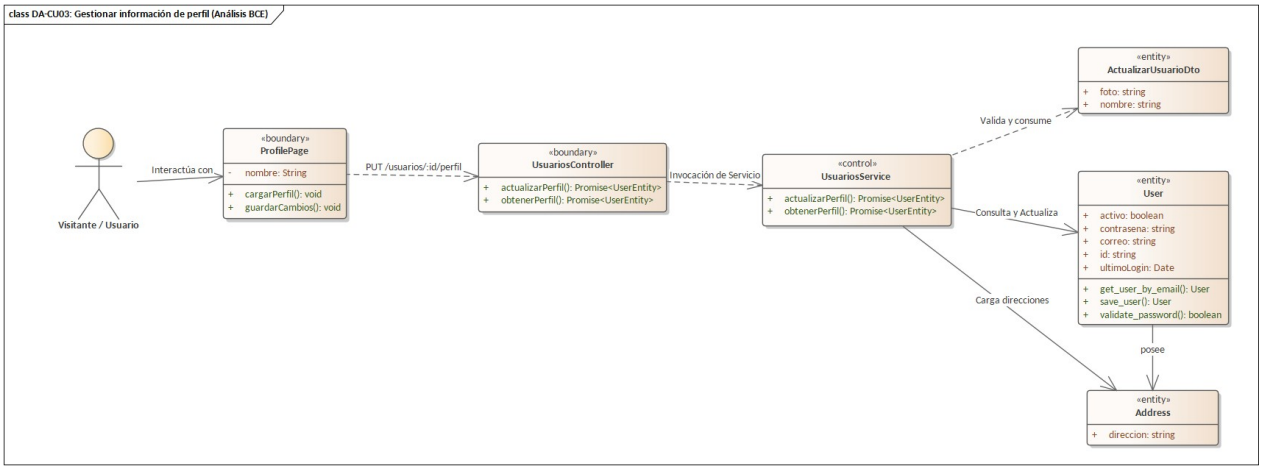

sd CU-24: Publicar y moderar reseñas de productos
dministrador
enviarCalificacionYComentario
mostrarReseñaEnFichaProducto
interfaz Reseñas
validarYPublicarReseña
reseñaRegistrada
ReviewModerationController
insertReseña
verificarCompraPrevia
recalcularPromedioRating
reseña
notaventa
producto

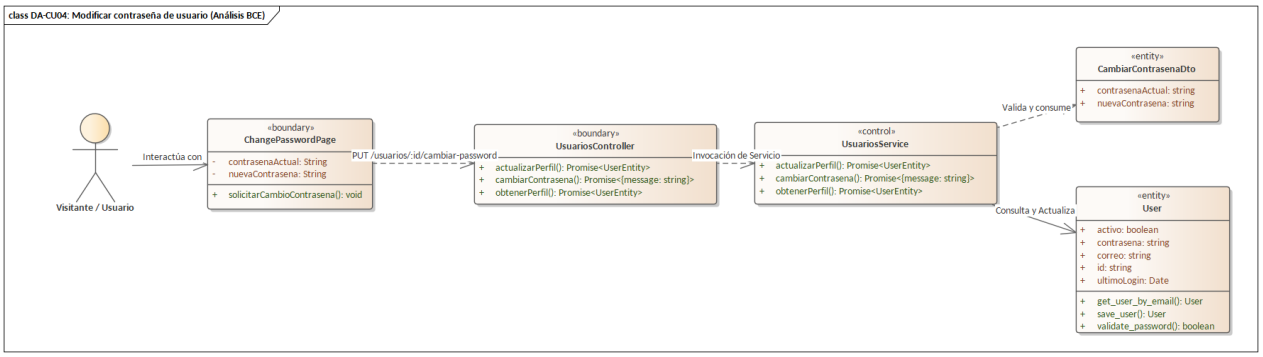


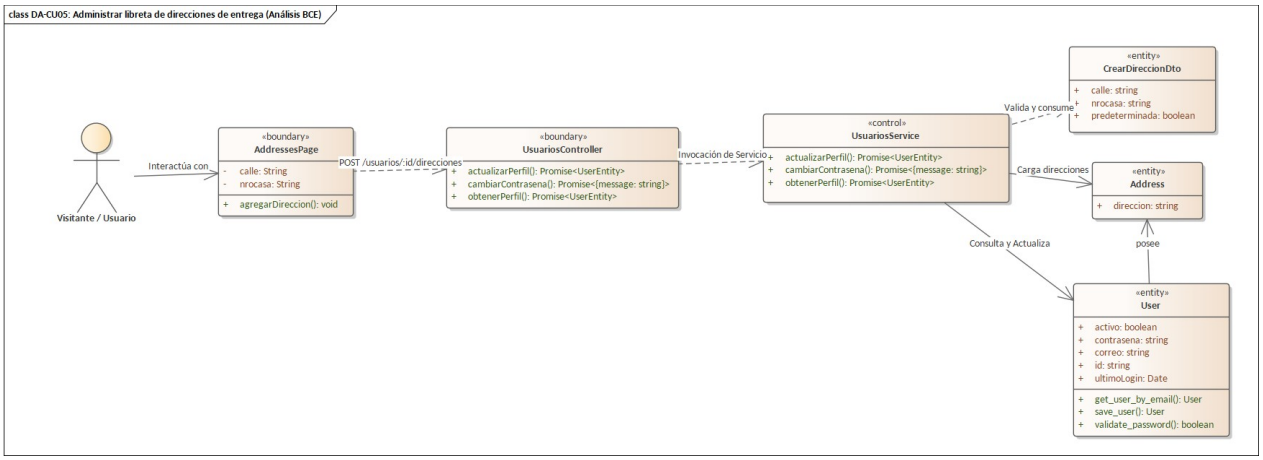
2.7. Análisis de clase

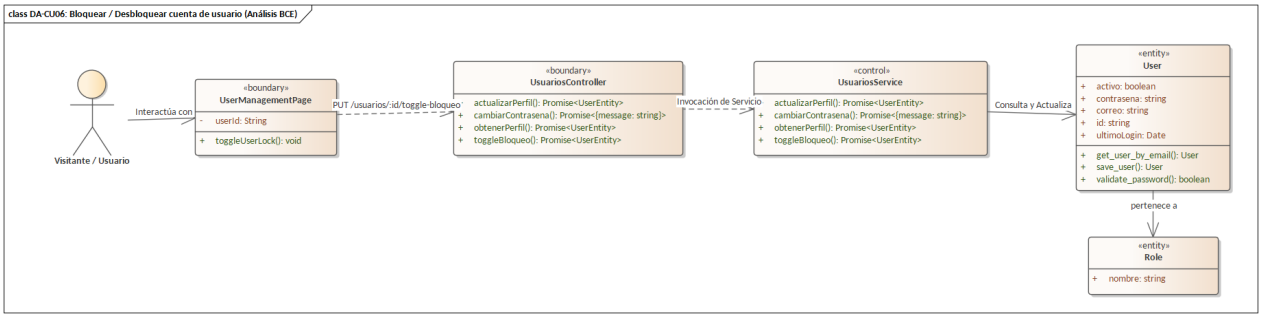


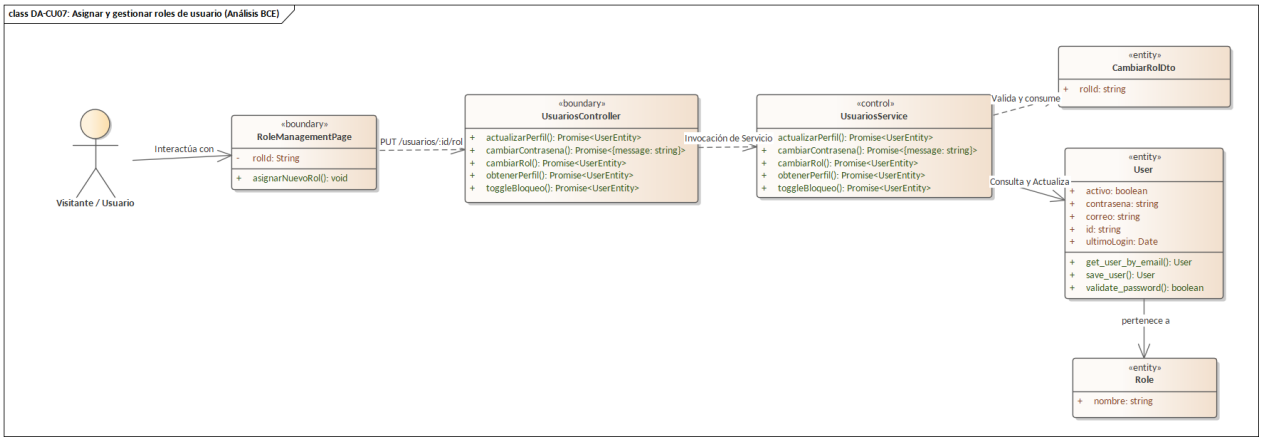

class class CU02 Recuperar contraseña de acceso
«Controller,boundary»
AuthController
+ login(): Promise<AuthTokensResponse>
+ logout(): void
+ me(): User
+ recuperarPassword(): void
+ restablecerPassword(): void
«control»
AuthService
- generarTokens(): Promise<{accessToken: string, refreshToken: string}>
+ login(): Promise<AuthTokensResponse>
«entity»
RecuperarPasswordDto
+ correo: string
«entity»
RestablecerPasswordDto
+ nuevaContrasena: string
+ token: string
«UI»
ResetPasswordPage
- email: String
- newPassword: String
- token: String
+ requestPasswordReset(): void
+ submitNewPassword(): void
«entity»
Token
+ usado: boolean
+ valor: string
«entity»
User
+ activo: boolean
+ contrasena: string
+ correo: string
+ id: string
+ ultimoLogin: Date
+ get_user_by_email(): User
+ save_user(): User
+ validate_password(): boolean
Visitante / Usuario
Invocación de API
usa
HTTP POST /auth/recuperar-password
Invocación de Servicio
Valida y consume
Valida y consume
Crea Refresh Token
Consulta y Actualiza
posee

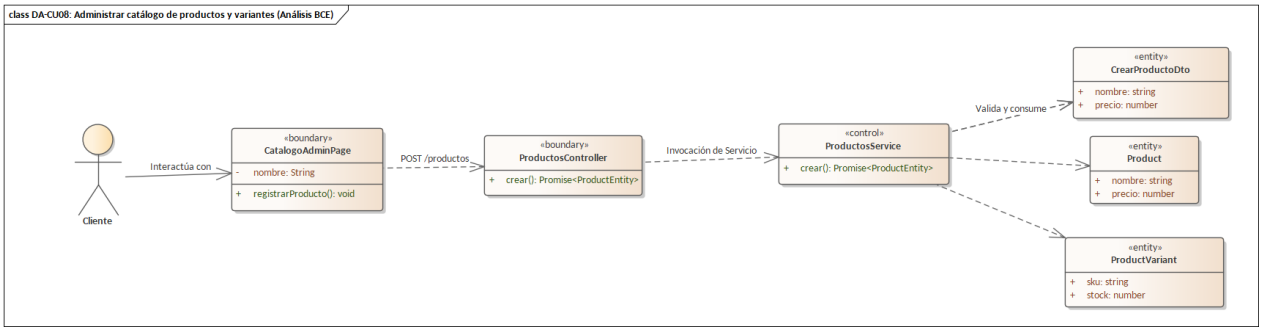

class DA-CU03: Gestionar información de perfil (Análisis BCE)
«boundary»
ProfilePage
- nombre: String
+ cargarPerfil(): void
+ guardarCambios(): void
«boundary»
UsuariosController
+ actualizarPerfil(): Promise<UserEntity>
+ obtenerPerfil(): Promise<UserEntity>
«control»
UsuariosService
+ actualizarPerfil(): Promise<UserEntity>
+ obtenerPerfil(): Promise<UserEntity>
«entity»
ActualizarUsuarioDto
+ foto: string
+ nombre: string
«entity»
User
+ activo: boolean
+ contrasena: string
+ correo: string
+ id: string
+ ultimoLogin: Date
+ get_user_by_email(): User
+ save_user(): User
+ validate_password(): boolean
«entity»
Address
+ direccion: string
Visitante / Usuario
Interactúa con
PUT /usuarios/:id/perfil
Invocación de Servicio
Valida y consume
Consulta y Actualiza
Carga direcciones
posee

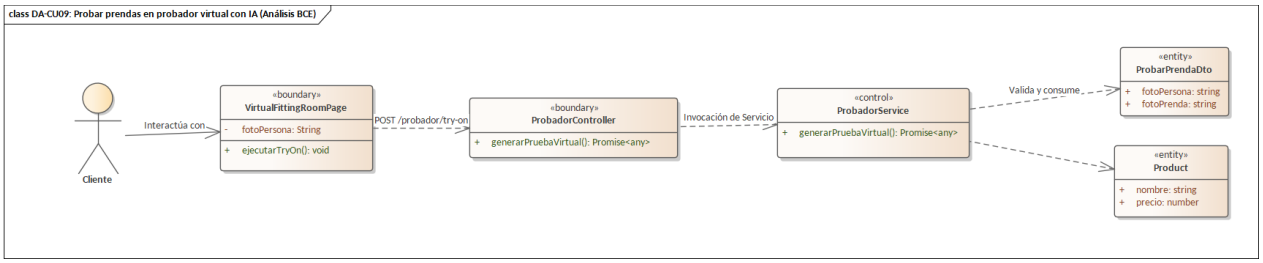

class DA-CU04: Modificar contraseña de usuario (Análisis BCE)
«boundary»
ChangePasswordPage
- contrasenaActual: String
- nuevaContrasena: String
+ solicitarCambioContrasena(): void
«boundary»
UsuariosController
+ actualizarPerfil(): Promise<UserEntity>
+ cambiarContrasena(): Promise<{message: string}>
+ obtenerPerfil(): Promise<UserEntity>
«control»
UsuariosService
+ actualizarPerfil(): Promise<UserEntity>
+ cambiarContrasena(): Promise<{message: string}>
+ obtenerPerfil(): Promise<UserEntity>
«entity»
CambiarContrasenaDto
+ contrasenaActual: string
+ nuevaContrasena: string
«entity»
User
+ activo: boolean
+ contrasena: string
+ correo: string
+ id: string
+ ultimoLogin: Date
+ get_user_by_email(): User
+ save_user(): User
+ validate_password(): boolean
Visitante / Usuario
Interactúa con
PUT /usuarios/:id/cambiar-password
Invocación de Servicio
Valida y consume
Consulta y Actualiza

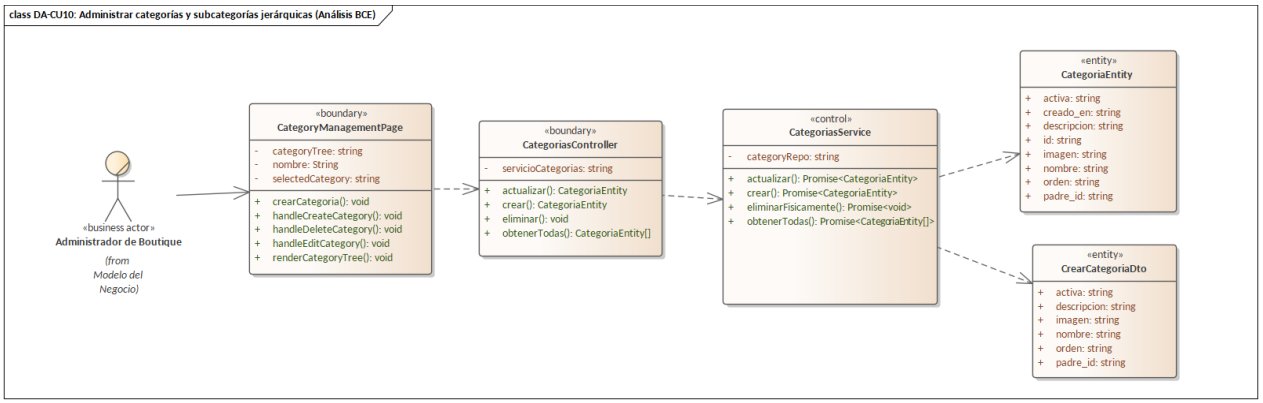

class DA-CU05: Administrar libreta de direcciones de entrega (Análisis BCE)
Visitante / Usuario
Interactúa con
«boundary»
AddressesPage
- calle: String
- nrocasa: String
+ agregarDireccion(): void
POST /usuarios/:id/direcciones
«boundary»
UsuariosController
+ actualizarPerfil(): Promise<UserEntity>
+ cambiarContrasena(): Promise<{message: string}>
+ obtenerPerfil(): Promise<UserEntity>
Invocación de Servicio
«control»
UsuariosService
+ actualizarPerfil(): Promise<UserEntity>
+ cambiarContrasena(): Promise<{message: string}>
+ obtenerPerfil(): Promise<UserEntity>
Valida y consume
«entity»
CrearDireccionDto
+ calle: string
+ nrocasa: string
+ predeterminada: boolean
Carga direcciones
«entity»
Address
+ direccion: string
Consulta y Actualiza
posee
«entity»
User
+ activo: boolean
+ contrasena: string
+ correo: string
+ id: string
+ ultimoLogin: Date
+ get_user_by_email(): User
+ save_user(): User
+ validate_password(): boolean

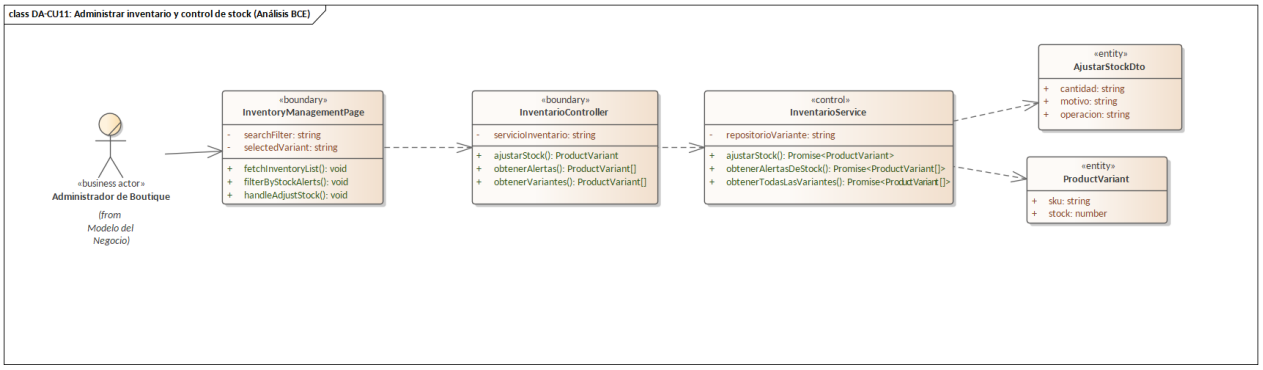

class DA-CU06: Bloquear / Desbloquear cuenta de usuario (Análisis BCE)
Visitante / Usuario
Interactúa con
«boundary»
UserManagementPage
- userId: String
+ toggleUserLock(): void
PUT /usuarios/:id/toggle-bloqueo
«boundary»
UsuariosController
+ actualizarPerfil(): Promise<UserEntity>
+ cambiarContrasena(): Promise<{message: string}>
+ obtenerPerfil(): Promise<UserEntity>
+ toggleBloqueo(): Promise<UserEntity>
Invocación de Servicio
«control»
UsuariosService
+ actualizarPerfil(): Promise<UserEntity>
+ cambiarContrasena(): Promise<{message: string}>
+ obtenerPerfil(): Promise<UserEntity>
+ toggleBloqueo(): Promise<UserEntity>
Consulta y Actualiza
«entity»
User
+ activo: boolean
+ contrasena: string
+ correo: string
+ id: string
+ ultimoLogin: Date
+ get_user_by_email(): User
+ save_user(): User
+ validate_password(): boolean
pertenece a
«entity»
Role
+ nombre: string

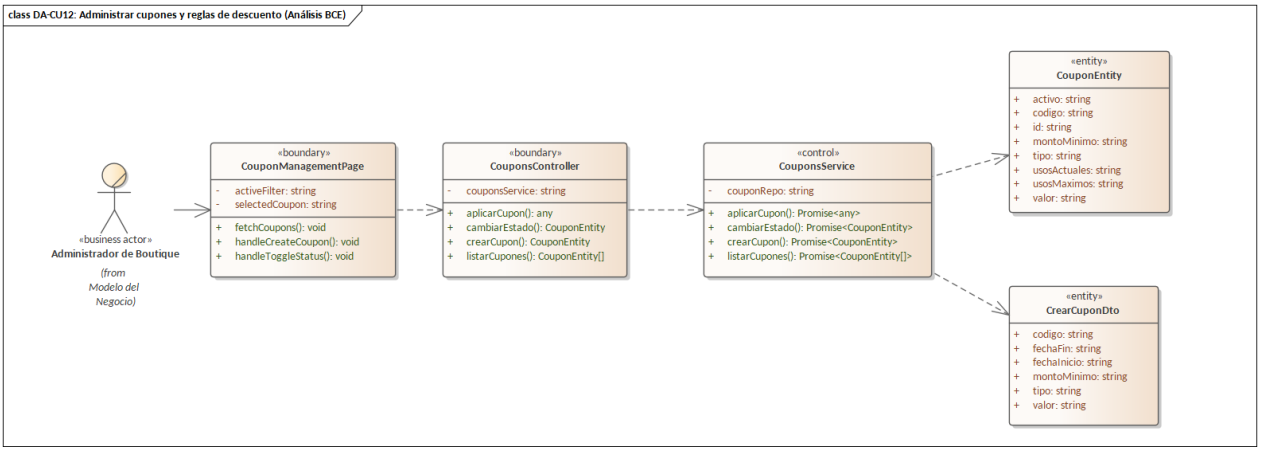

class DA-CU07: Asignar y gestionar roles de usuario (Análisis BCE)
Visitante / Usuario
Interactúa con
«boundary»
RoleManagementPage
- rolId: String
+ asignarNuevoRol(): void
PUT /usuarios/:id/rol
«boundary»
UsuariosController
+ actualizarPerfil(): Promise<UserEntity>
+ cambiarContrasena(): Promise<{message: string}>
+ cambiarRol(): Promise<UserEntity>
+ obtenerPerfil(): Promise<UserEntity>
+ toggleBloqueo(): Promise<UserEntity>
Invocación de Servicio
«control»
UsuariosService
+ actualizarPerfil(): Promise<UserEntity>
+ cambiarContrasena(): Promise<{message: string}>
+ cambiarRol(): Promise<UserEntity>
+ obtenerPerfil(): Promise<UserEntity>
+ toggleBloqueo(): Promise<UserEntity>
Valida y consume
«entity»
CambiarRolDto
+ rolId: string
Consulta y Actualiza
«entity»
User
+ activo: boolean
+ contrasena: string
+ correo: string
+ id: string
+ ultimoLogin: Date
+ get_user_by_email(): User
+ save_user(): User
+ validate_password(): boolean
pertenece a
«entity»
Role
+ nombre: string

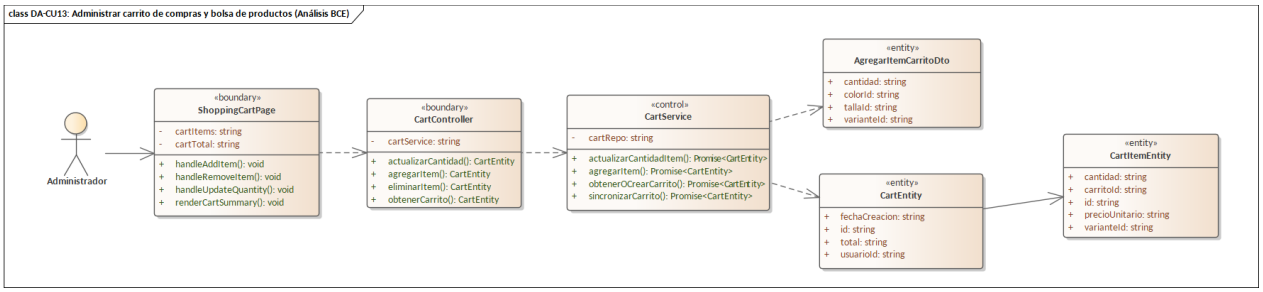

class DA-CU08: Administrar catálogo de productos y variantes (Análisis BCE)
Cliente
Interactúa con
«boundary»
CatalogoAdminPage
- nombre: String
+ registrarProducto(): void
POST /productos
«boundary»
ProductosController
+ crear(): Promise<ProductEntity>
Invocación de Servicio
«control»
ProductosService
+ crear(): Promise<ProductEntity>
Valida y consume
«entity»
CrearProductoDto
+ nombre: string
+ precio: number
«entity»
Product
+ nombre: string
+ precio: number
«entity»
ProductVariant
+ sku: string
+ stock: number

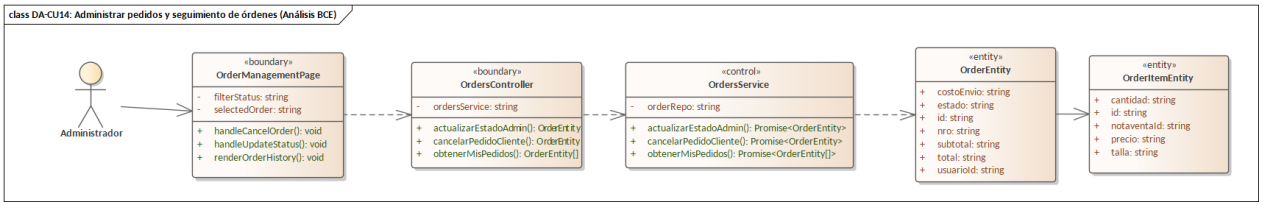

class DA-CU09: Probar prendas en probador virtual con IA (Análisis BCE)
Cliente
Interactúa con
«boundary»
VirtualFittingRoomPage
- fotoPersona: String
+ ejecutarTryOn(): void
POST /probador/try-on
«boundary»
ProbadorController
+ generarPruebaVirtual(): Promise<any>
Invocación de Servicio
«control»
ProbadorService
+ generarPruebaVirtual(): Promise<any>
Valida y consume
«entity»
ProbarPrendaDto
+ fotoPersona: string
+ fotoPrenda: string
«entity»
Product
+ nombre: string
+ precio: number

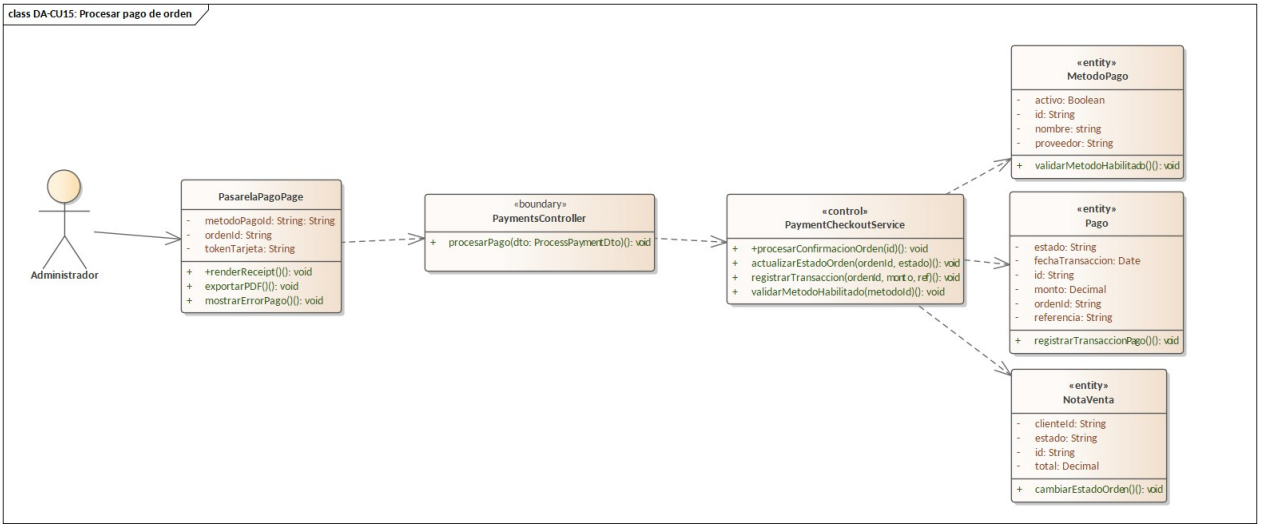

class DA-CU10: Administrar categorías y subcategorías jerárquicas (Análisis BCE)
«business actor»
Administrador de Boutique
(from Modelo del Negocio)
«boundary»
CategoryManagementPage
- categoryTree: string
- nombre: String
- selectedCategory: string
+ crearCategoria(): void
+ handleCreateCategory(): void
+ handleDeleteCategory(): void
+ handleEditCategory(): void
+ renderCategoryTree(): void
«boundary»
CategoriasController
- servicioCategorias: string
+ actualizar(): CategoriaEntity
+ crear(): CategoriaEntity
+ eliminar(): void
+ obtenerTodas(): CategoriaEntity[]
«control»
CategoriasService
- categoryRepo: string
+ actualizar(): Promise<CategoriaEntity>
+ crear(): Promise<CategoriaEntity>
+ eliminarFisicamente(): Promise<void>
+ obtenerTodas(): Promise<CategoriaEntity[]>
«entity»
CategoriaEntity
+ activa: string
+ creado_en: string
+ descripcion: string
+ id: string
+ imagen: string
+ nombre: string
+ orden: string
+ padre_id: string
«entity»
CrearCategoriaDto
+ activa: string
+ descripcion: string
+ imagen: string
+ nombre: string
+ orden: string
+ padre_id: string

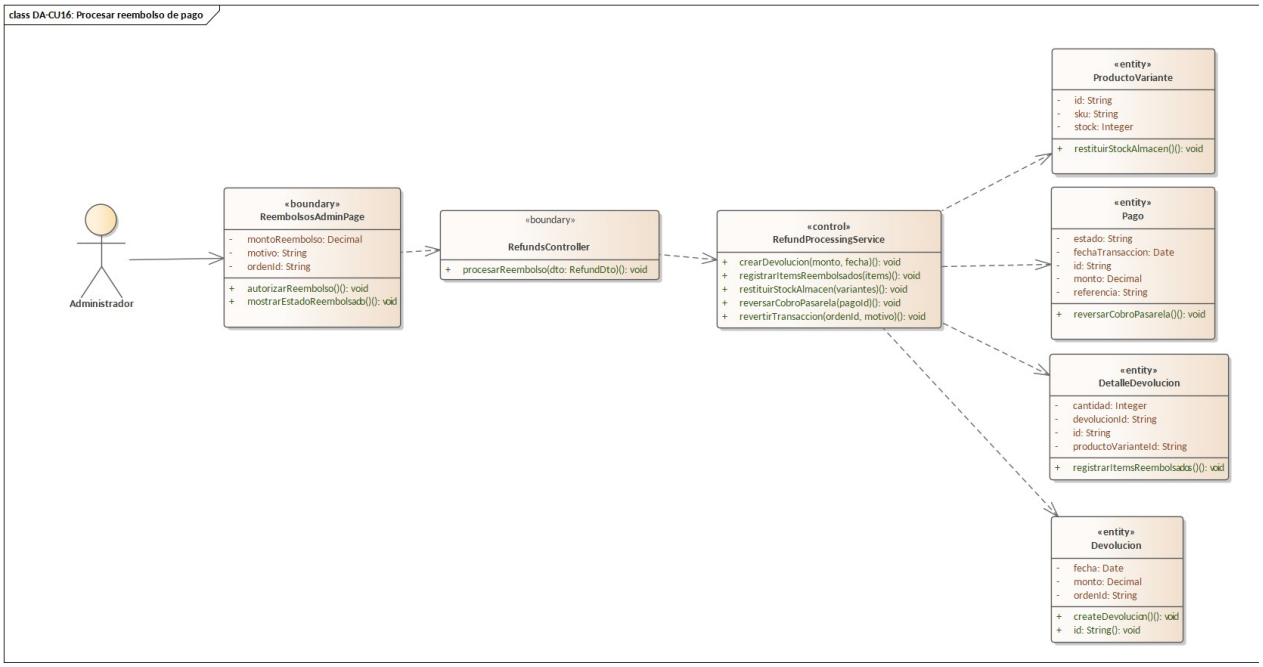

class DA-CU11: Administrar inventario y control de stock (Análisis BCE)
«business actor»
Administrador de Boutique
(from Modelo del Negocio)
«boundary»
InventoryManagementPage
- searchFilter: string
- selectedVariant: string
+ fetchInventoryList(): void
+ filterByStockAlerts(): void
+ handleAdjustStock(): void
«boundary»
InventarioController
- servicioInventario: string
+ ajustarStock(): ProductVariant
+ obtenerAlertas(): ProductVariant[]
+ obtenerVariantes(): ProductVariant[]
«control»
InventarioService
- repositorioVariante: string
+ ajustarStock(): Promise<ProductVariant>
+ obtenerAlertasDeStock(): Promise<ProductVariant[]>
+ obtenerTodasLasVariantes(): Promise<ProductVariant[]>
«entity»
AjustarStockDto
+ cantidad: string
+ motivo: string
+ operacion: string
«entity»
ProductVariant
+ sku: string
+ stock: number

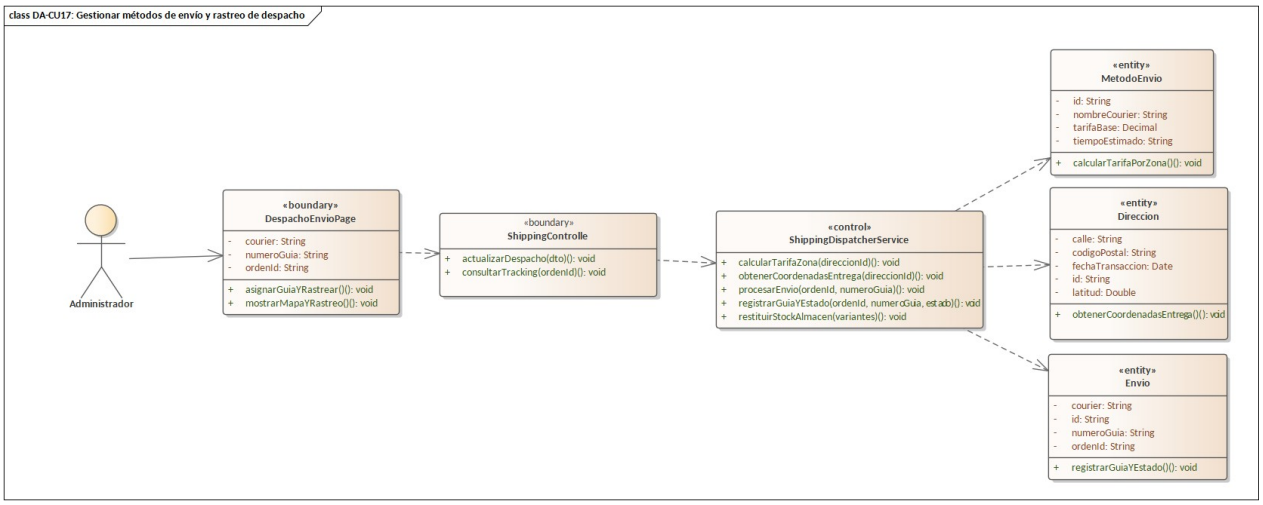

class DA-CU12: Administrar cupones y reglas de descuento (Análisis BCE)
«business actor»
Administrador de Boutique
(from Modelo del Negocio)
«boundary»
CouponManagementPage
- activeFilter: string
- selectedCoupon: string
+ fetchCoupons(): void
+ handleCreateCoupon(): void
+ handleToggleStatus(): void
«boundary»
CouponsController
- couponsService: string
+ aplicarCupon(): any
+ cambiarEstado(): CouponEntity
+ crearCupon(): CouponEntity
+ listarCupones(): CouponEntity[]
«control»
CouponsService
- couponRepo: string
+ aplicarCupon(): Promise<any>
+ cambiarEstado(): Promise<CouponEntity>
+ crearCupon(): Promise<CouponEntity>
+ listarCupones(): Promise<CouponEntity[]>
«entity»
CouponEntity
+ activo: string
+ codigo: string
+ id: string
+ montoMinimo: string
+ tipo: string
+ usosActuales: string
+ usosMaximos: string
+ valor: string
«entity»
CrearCuponDto
+ codigo: string
+ fechaFin: string
+ fechaInicio: string
+ montoMinimo: string
+ tipo: string
+ valor: string

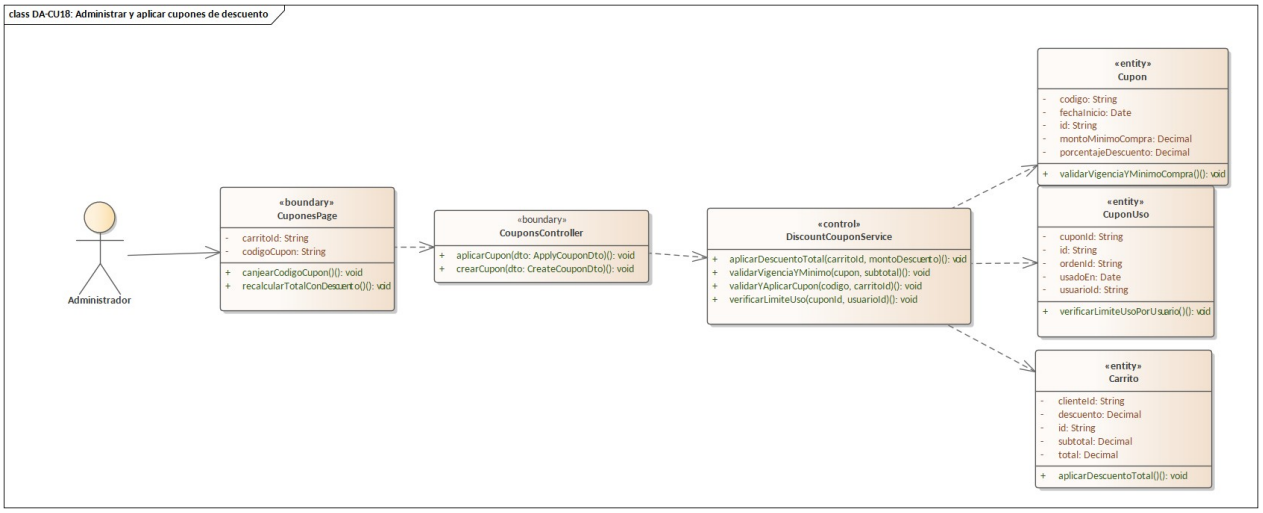

class DA-CU13: Administrar carrito de compras y bolsa de productos (Análisis BCE)
Administrador
«boundary»
ShoppingCartPage
- cartItems: string
- cartTotal: string
+ handleAddItem(): void
+ handleRemoveItem(): void
+ handleUpdateQuantity(): void
+ renderCartSummary(): void
«boundary»
CartController
- cartService: string
+ actualizarCantidad(): CartEntity
+ agregarItem(): CartEntity
+ eliminarItem(): CartEntity
+ obtenerCarrito(): CartEntity
«control»
CartService
- cartRepo: string
+ actualizarCantidadItem(): Promise<CartEntity>
+ agregarItem(): Promise<CartEntity>
+ obtenerOCrearCarrito(): Promise<CartEntity>
+ sincronizarCarrito(): Promise<CartEntity>
«entity»
AgregarItemCarritoDto
+ cantidad: string
+ colorId: string
+ tallaId: string
+ varianteId: string
«entity»
CartEntity
+ fechaCreacion: string
+ id: string
+ total: string
+ usuarioId: string
«entity»
CartItemEntity
+ cantidad: string
+ carritoId: string
+ id: string
+ precioUnitario: string
+ varianteId: string

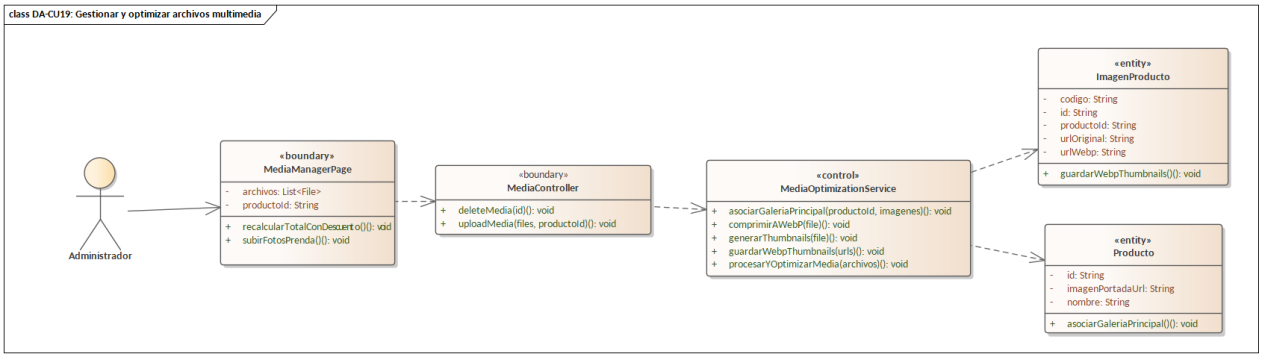

class DA-CU14: Administrar pedidos y seguimiento de órdenes (Análisis BCE)
Administrador
«boundary»
OrderManagementPage
- filterStatus: string
- selectedOrder: string
+ handleCancelOrder(): void
+ handleUpdateStatus(): void
+ renderOrderHistory(): void
«boundary»
OrdersController
- ordersService: string
+ actualizarEstadoAdmin(): OrderEntity
+ cancelarPedidoCliente(): OrderEntity
+ obtenerMisPedidos(): OrderEntity[]
«control»
OrdersService
- orderRepo: string
+ actualizarEstadoAdmin(): Promise<OrderEntity>
+ cancelarPedidoCliente(): Promise<OrderEntity>
+ obtenerMisPedidos(): Promise<OrderEntity[]>
«entity»
OrderEntity
+ costoEnvio: string
+ estado: string
+ id: string
+ nro: string
+ subtotal: string
+ total: string
+ usuarioId: string
«entity»
OrderItemEntity
+ cantidad: string
+ id: string
+ notaventaId: string
+ precio: string
+ talla: string

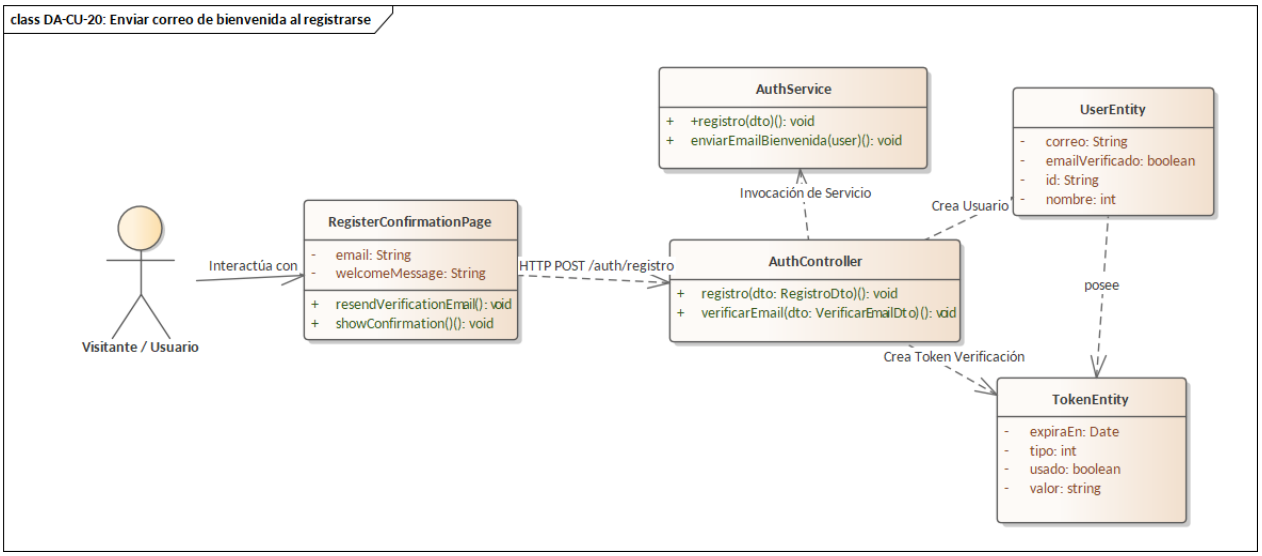

class DA-CU15: Procesar pago de orden
Administrador
PasarelaPagoPage
- metodoPagoId: String: String
- ordenId: String
- tokenTarjeta: String
+ +renderReceipt(): void
+ exportarPDF(): void
+ mostrarErrorPago(): void
«boundary»
PaymentsController
+ procesarPago(dto: ProcessPaymentDto)(): void
«control»
PaymentCheckoutService
+ +procesarConfirmacionOrden(id)(): void
+ actualizarEstadoOrden(ordenId, estado)(): void
+ registrarTransaccion(ordenId, monto, ref)(): void
+ validarMetodoHabilitado(metodoId)(): void
«entity»
MetodoPago
- activo: Boolean
- id: String
- nombre: string
- proveedor: String
+ validarMetodoHabilitado()(): void
«entity»
Pago
- estado: String
- fechaTransaccion: Date
- id: String
- monto: Decimal
- ordenId: String
- referencia: String
+ registrarTransaccionPago()(): void
«entity»
NotaVenta
- clienteId: String
- estado: String
- id: String
- total: Decimal
+ cambiarEstadoOrden()(): void

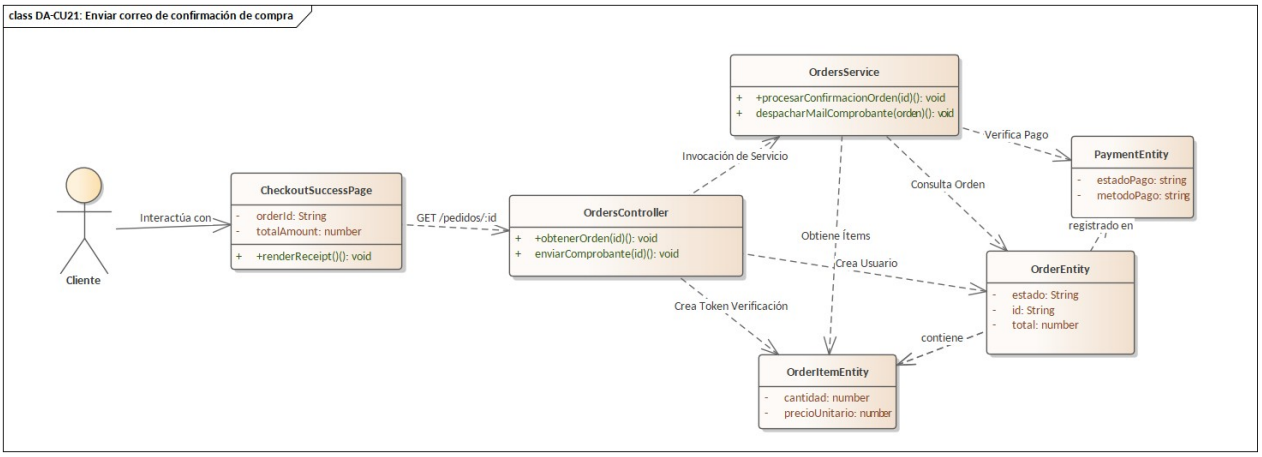

class DA-CU16: Procesar reembolso de pago
Administrador
«boundary»
ReembolsosAdminPage
- montoReembolso: Decimal
- motivo: String
- ordenId: String
+ autorizarReembolso()(): void
+ mostrarEstadoReembolsado()(): void
«boundary»
RefundsController
+ procesarReembolso(dto: RefundDto)(): void
«control»
RefundProcessingService
+ crearDevolucion(monto, fecha)(): void
+ registrarItemsReembolsados(items)(): void
+ restituirStockAlmacen(variantes)(): void
+ reversarCobroPasarela(pagoId)(): void
+ revertirTransaccion(ordenId, motivo)(): void
«entity»
ProductoVariante
- id: String
- sku: String
- stock: Integer
+ restituirStockAlmacen()(): void
«entity»
Pago
- estado: String
- fechaTransaccion: Date
- id: String
- monto: Decimal
- referencia: String
+ reversarCobroPasarela()(): void
«entity»
DetalleDevolucion
- cantidad: Integer
- devolucionId: String
- id: String
- productoVarianteId: String
+ registrarItemsReembolsados()(): void
«entity»
Devolucion
- fecha: Date
- monto: Decimal
- ordenId: String
+ createDevolucion()(): void
+ id: String(): void

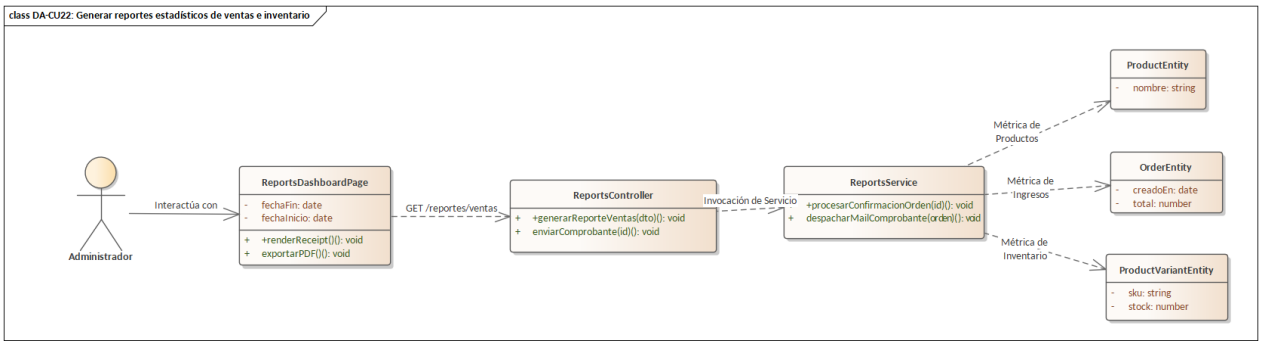

class DA-CU17: Gestionar métodos de envío y rastreo de despacho
Administrador
«boundary»
DespachoEnvioPage
- courier: String
- numeroGuia: String
- ordenId: String
+ asignarGuiaYRastrear(): void
+ mostrarMapaYRastreo(): void
«boundary»
ShippingControlle
+ actualizarDespacho(dto): void
+ consultarTracking(ordenId): void
«control»
ShippingDispatcherService
+ calcularTarifaZona(direccionId): void
+ obtenerCoordenadasEntrega(direccionId): void
+ procesarEnvio(ordenId, numeroGuia): void
+ registrarGuiaYEstado(ordenId, numeroGuia, estado): void
+ restituirStockAlmacen(variantes): void
«entity»
MetodoEnvio
- id: String
- nombreCourier: String
- tarifaBase: Decimal
- tiempoEstimado: String
+ calcularTarifaPorZona(): void
«entity»
Direccion
- calle: String
- codigoPostal: String
- fechaTransaccion: Date
- id: String
- latitud: Double
+ obtenerCoordenadasEntrega(): void
«entity»
Envio
- courier: String
- id: String
- numeroGuia: String
- ordenId: String
+ registrarGuiaYEstado(): void

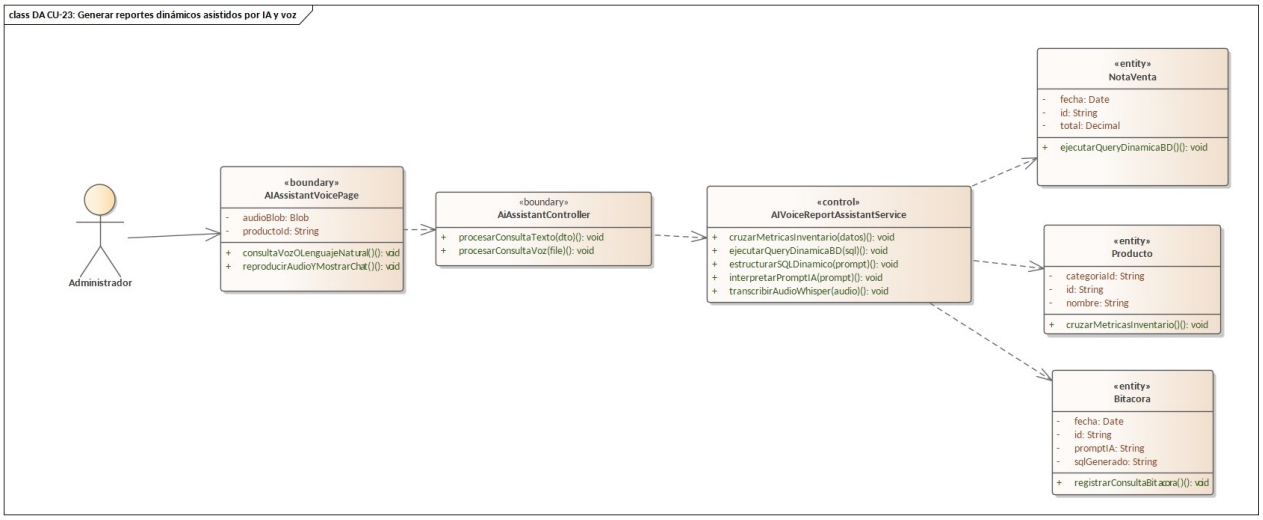

class DA-CU18: Administrar y aplicar cupones de descuento
Administrador
«boundary»
CuponesPage
- carritoId: String
- codigoCupon: String
+ canjearCodigoCupon(): void
+ recalcularTotalConDescuento(): void
«boundary»
CouponsController
+ aplicarCupon(dto: ApplyCouponDto): void
+ crearCupon(dto: CreateCouponDto): void
«control»
DiscountCouponService
+ aplicarDescuentoTotal(carritoId, montoDescuento): void
+ validarVigenciaYMinimo(cupon, subtotal): void
+ validarYAplicarCupon(codigo, carritoId): void
+ verificarLimiteUso(cuponId, usuarioId): void
«entity»
Cupon
- codigo: String
- fechaInicio: Date
- id: String
- montoMinimoCompra: Decimal
- porcentajeDescuento: Decimal
+ validarVigenciaYMinimoCompra(): void
«entity»
CuponUso
- cuponId: String
- id: String
- ordenId: String
- usadoEn: Date
- usuarioId: String
+ verificarLimiteUsoPorUsuario(): void
«entity»
Carrito
- clienteId: String
- descuento: Decimal
- id: String
- subtotal: Decimal
- total: Decimal
+ aplicarDescuentoTotal(): void

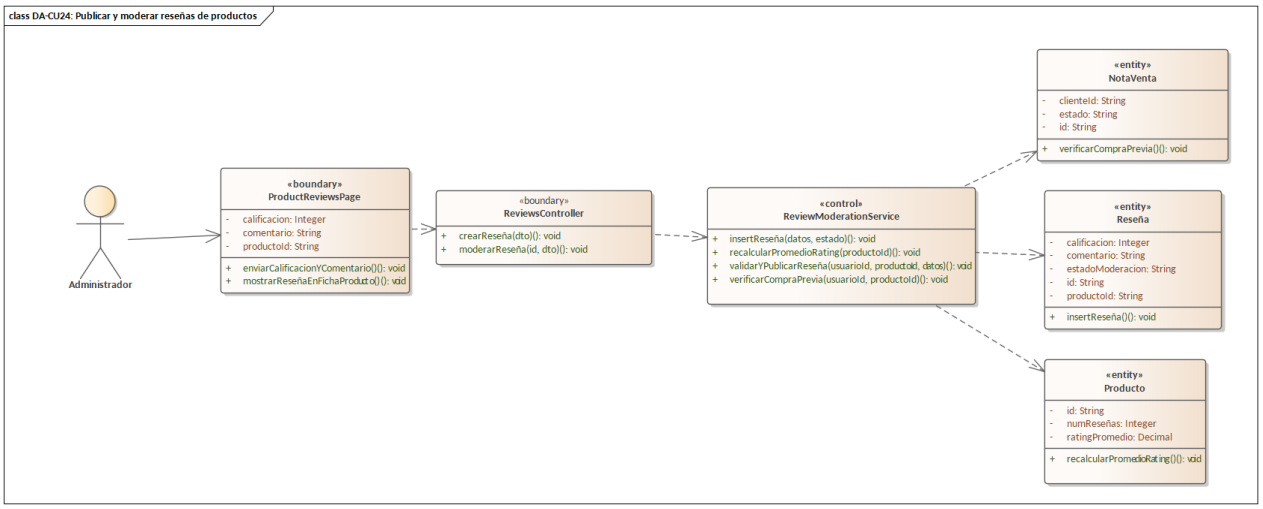

class DA-CU19: Gestionar y optimizar archivos multimedia
Administrador
«boundary»
MediaManagerPage
- archivos: List<File>
- productoId: String
+ recalcularTotalConDescuento(): void
+ subirFotosPrenda(): void
«boundary»
MediaController
+ deleteMedia(id): void
+ uploadMedia(files, productoId): void
«control»
MediaOptimizationService
+ asociarGaleriaPrincipal(productoId, imagenes): void
+ comprimirAWebP(file): void
+ generarThumbnails(file): void
+ guardarWebpThumbnails(urls): void
+ procesarYOptimizarMedia(archivos): void
«entity»
ImagenProducto
- codigo: String
- id: String
- productoId: String
- urlOriginal: String
- urlWebp: String
+ guardarWebpThumbnails(): void
«entity»
Producto
- id: String
- imagenPortadaUrl: String
- nombre: String
+ asociarGaleriaPrincipal(): void

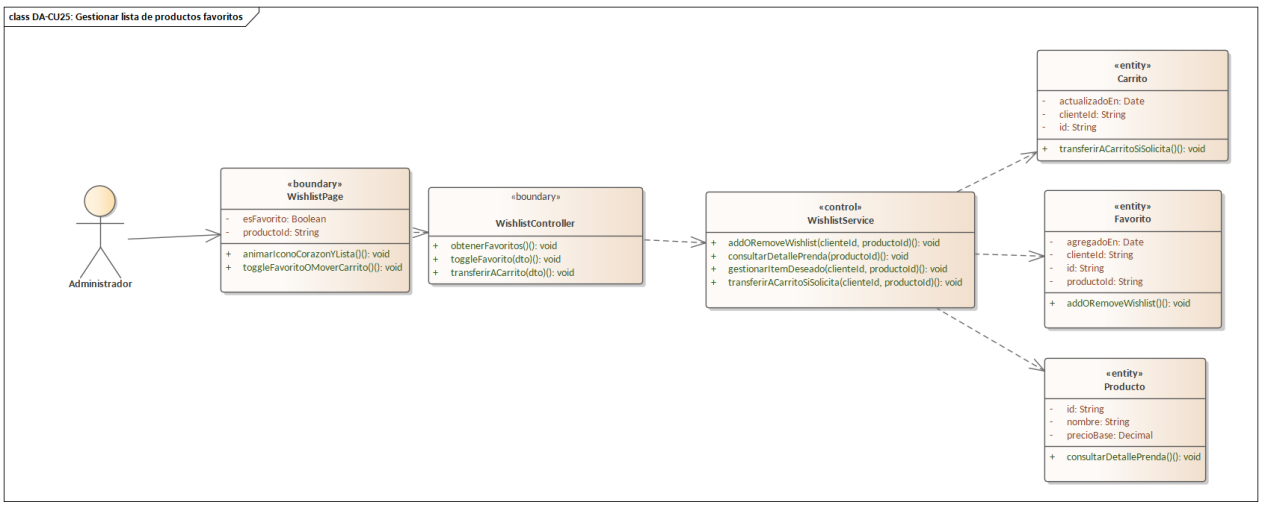

class DA-CU-20: Enviar correo de bienvenida al registrarse
AuthService
+ +registro(dto)(): void
+ enviarEmailBienvenida(user)(): void
UserEntity
- correo: String
- emailVerificado: boolean
- id: String
- nombre: int
Invocación de Servicio
Crea Usuario
Visitante / Usuario
Interactúa con
RegisterConfirmationPage
- email: String
- welcomeMessage: String
+ resendVerificationEmail(): void
+ showConfirmation()(): void
HTTP POST /auth/registro
AuthController
+ registro(dto: RegistroDto)(): void
+ verificarEmail(dto: VerificarEmailDto)(): void
posee
Crea Token Verificación
TokenEntity
- expiraEn: Date
- tipo: int
- usado: boolean
- valor: string


class DA-CU21: Enviar correo de confirmación de compra
OrdersService
+ +procesarConfirmacionOrden(id)(): void
+ despacharMailComprobante(orden)(): void
Verifica Pago
PaymentEntity
- estadoPago: string
- metodoPago: string
Invocación de Servicio
Consulta Orden
registrado en
Cliente
Interactúa con
CheckoutSuccessPage
- orderId: String
- totalAmount: number
+ +renderReceipt()(): void
GET /pedidos/:id
OrdersController
+ +obtenerOrden(id)(): void
+ enviarComprobante(id)(): void
Obtiene Ítems
Crea Usuario
OrderEntity
- estado: String
- id: String
- total: number
Crea Token Verificación
contiene
OrderItemEntity
- cantidad: number
- precioUnitario: number

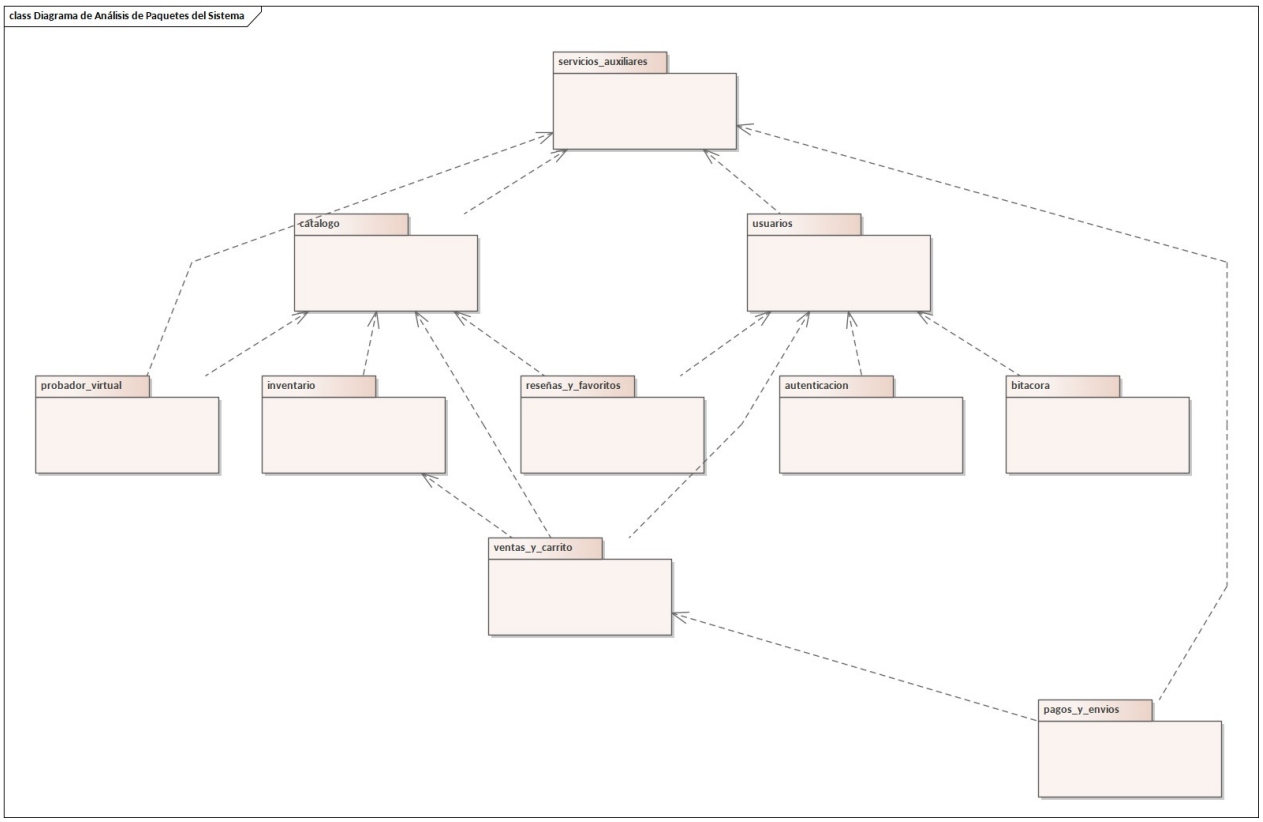

class DA-CU22: Generar reportes estadísticos de ventas e inventario
ProductEntity
- nombre: string
Métrica de Productos
Administrador
Interactúa con
ReportsDashboardPage
- fechaFin: date
- fechaInicio: date
+ +renderReceipt()(): void
+ exportarPDF()(): void
GET /reportes/ventas
ReportsController
+ +generarReporteVentas(dto)(): void
+ enviarComprobante(id)(): void
Invocación de Servicio
ReportsService
+ +procesarConfirmacionOrden(id)(): void
+ despacharMailComprobante(orden)(): void
Métrica de Ingresos
OrderEntity
- creadoEn: date
- total: number
Métrica de Inventario
ProductVariantEntity
- sku: string
- stock: number


class DA CU-23: Generar reportes dinámicos asistidos por IA y voz
Administrador
«boundary»
AIAssistantVoicePage
- audioBlob: Blob
- productoId: String
+ consultaVozOLenguajeNatural(): void
+ reproducirAudioYMostrarChat(): void
«boundary»
AiAssistantController
+ procesarConsultaTexto(dto): void
+ procesarConsultaVoz(file): void
«control»
AIVoiceReportAssistantService
+ cruzarMetricasInventario(datos): void
+ ejecutarQueryDinamicaBD(sql): void
+ estructurarSQLDinamico(prompt): void
+ interpretarPromptIA(prompt): void
+ transcribirAudioWhisper(audio): void
«entity»
NotaVenta
- fecha: Date
- id: String
- total: Decimal
+ ejecutarQueryDinamicaBD(): void
«entity»
Producto
- categoriaId: String
- id: String
- nombre: String
+ cruzarMetricasInventario(): void
«entity»
Bitacora
- fecha: Date
- id: String
- promptIA: String
- sqlGenerado: String
+ registrarConsultaBitacora(): void


class DA-CU24: Publicar y moderar reseñas de productos
Administrador
«boundary»
ProductReviewsPage
- calificacion: Integer
- comentario: String
- productoId: String
+ enviarCalificacionYComentario(): void
+ mostrarReseñaEnFichaProducto(): void
«boundary»
ReviewsController
+ crearReseña(dto): void
+ moderarReseña(id, dto): void
«control»
ReviewModerationService
+ insertReseña(datos, estado): void
+ recalcularPromedioRating(productoId): void
+ validarYPublicarReseña(usuarioId, productoId, datos): void
+ verificarCompraPrevia(usuarioId, productoId): void
«entity»
NotaVenta
- clienteId: String
- estado: String
- id: String
+ verificarCompraPrevia(): void
«entity»
Reseña
- calificacion: Integer
- comentario: String
- estadoModeracion: String
- id: String
- productoId: String
+ insertReseña(): void
«entity»
Producto
- id: String
- numReseñas: Integer
- ratingPromedio: Decimal
+ recalcularPromedioRating(): void


class DA-CU25: Gestionar lista de productos favoritos
Administrador
«boundary»
WishlistPage
- esFavorito: Boolean
- productoId: String
+ animarIconoCorazonYLista(): void
+ toggleFavoritoOMoverCarrito(): void
«boundary»
WishlistController
+ obtenerFavoritos(): void
+ toggleFavorito(dto): void
+ transferirACarrito(dto): void
«control»
WishlistService
+ addORemoveWishlist(clienteId, productoId): void
+ consultarDetallePrenda(productoId): void
+ gestionarItemDeseado(clienteId, productoId): void
+ transferirACarritoSiSolicita(clienteId, productoId): void
«entity»
Carrito
- actualizadoEn: Date
- clienteId: String
- id: String
+ transferirACarritoSiSolicita(): void
«entity»
Favorito
- agregadoEn: Date
- clienteId: String
- id: String
- productoId: String
+ addORemoveWishlist(): void
«entity»
Producto
- id: String
- nombre: String
- precioBase: Decimal
+ consultarDetallePrenda(): void

2.8. Análisis de paquete



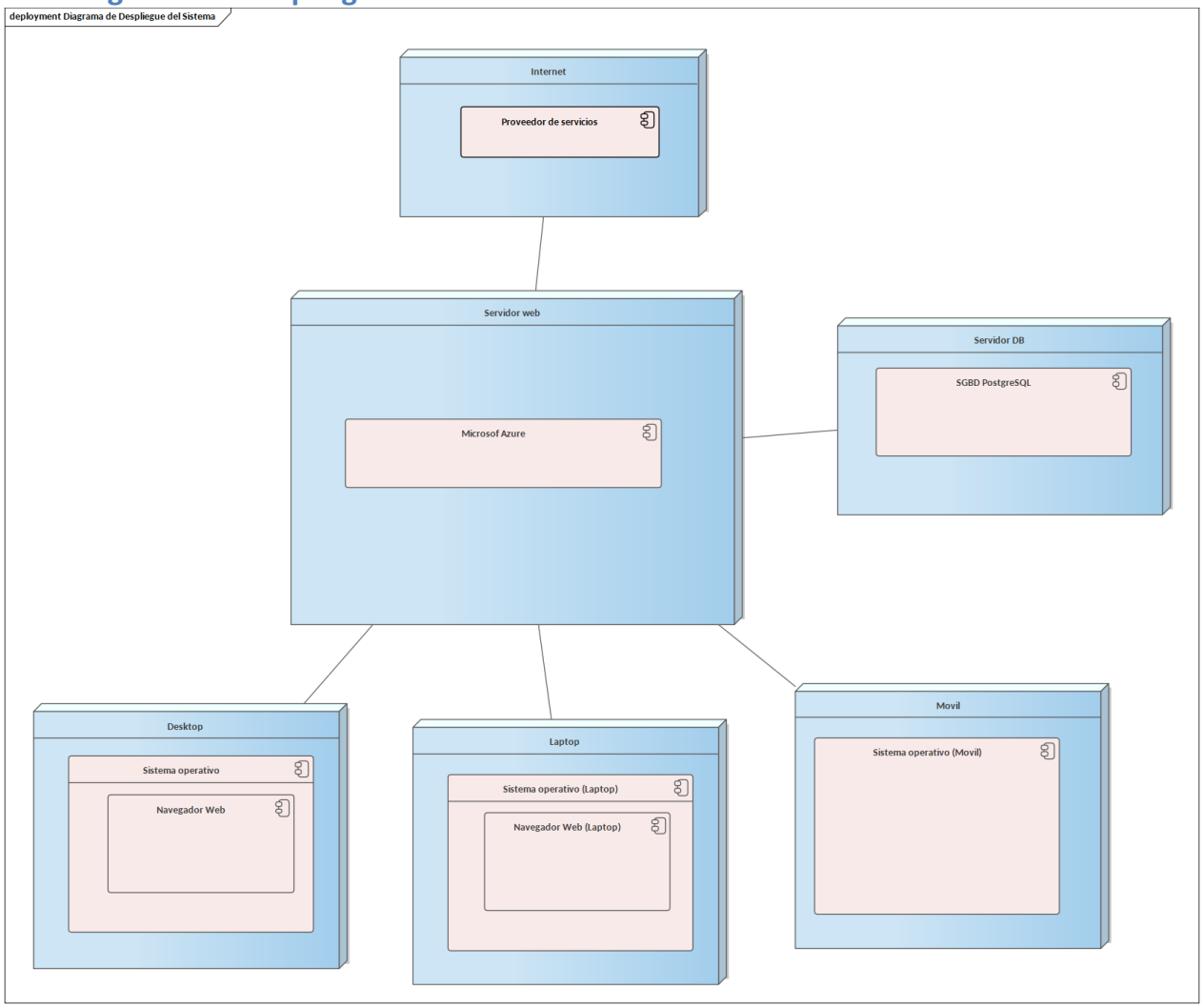
CAPITULO 3: FLUJO DE TRABAJO: DISEÑO

3.1. Diseño de Arquitectura

Patrón en capas de AURA: Presentación (React SPA), Aplicación (NestJS REST API), Dominio (Servicios),

Persistencia (TypeORM + PostgreSQL 16) e Integración Externa (Stripe, Hugging Face, SMTP).

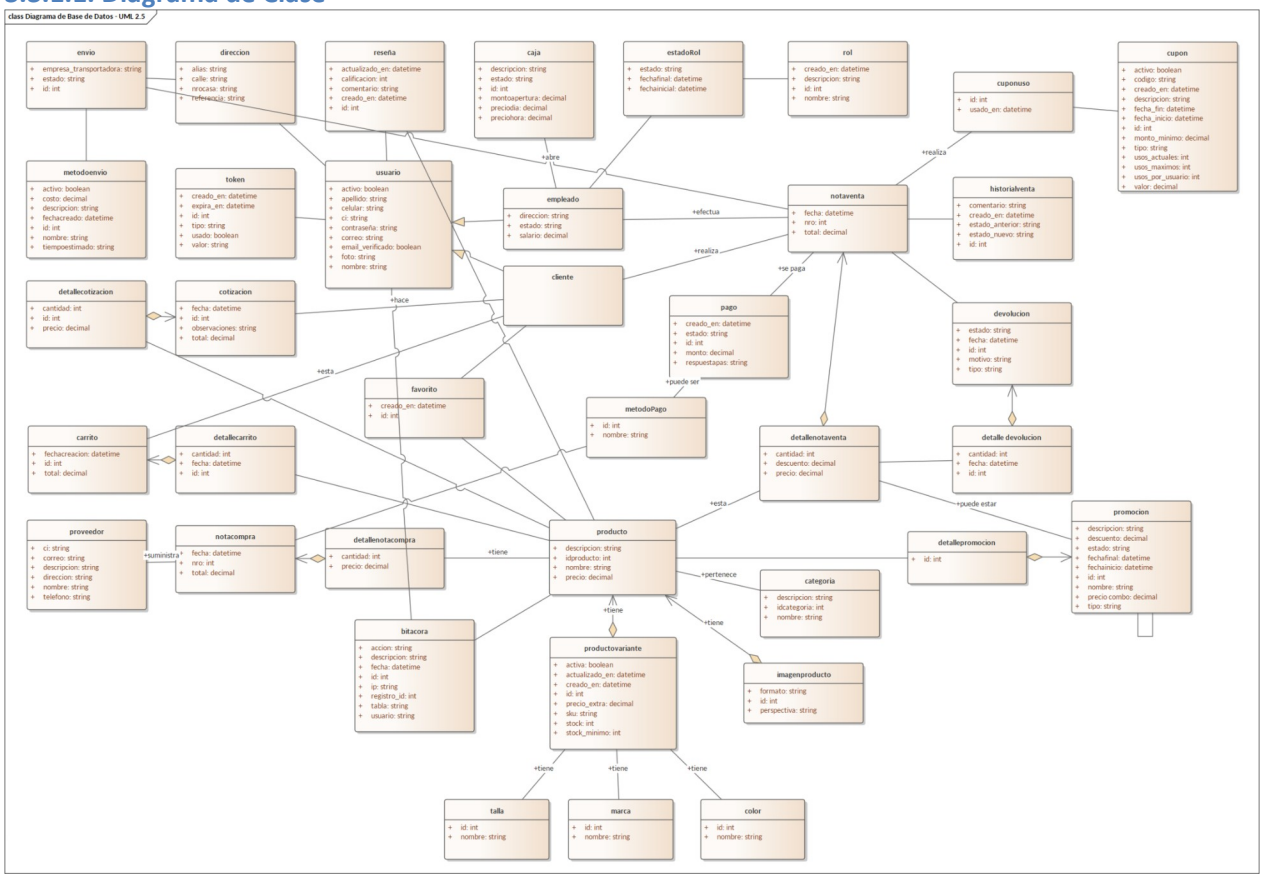
3.2. Diagrama de despliegue



3.3. Diseño de datos

3.3.1. Diseño de datos lógico

3.3.1.1. Diagrama de Clase



3.3.1.2. Mapeo

Mapeo relacional formal con PKs y FKs de las tablas núcleo:

Tabla: usuario
PK: id (UUID) | FK: rol_id → rol(id)
Atributos: nombre, email, password, activo, creado_en

Tabla: cliente
PK: id (UUID) | FK: usuario_id → usuario(id)
Atributos: ci_nit, telefono, avatar_url

Tabla: direccion
PK: id (UUID) | FK: cliente_id → cliente(id)
Atributos: departamento, ciudad, calle, referencia, es_predeterminada

Tabla: categoria

PK: id (UUID) | FK: categoria_padre_id → categoria(id)
Atributos: nombre, slug, descripcion, activa

Tabla: producto

PK: id (UUID) | FK: categoria_id → categoria(id), marca_id → marca(id)
Atributos: nombre, descripcion, precio, activo, destacado

Tabla: producto_variante

PK: id (UUID) | FK: producto_id → producto(id), talla_id → talla(id), color_id → color(id)
Atributos: sku, stock, stock_minimo, precio_extra, activa

Tabla: carrito

PK: id (UUID) | FK: usuario_id → usuario(id)
Atributos: actualizado_en, total_estimado

Tabla: detalle_carrito

PK: id (UUID) | FK: carrito_id → carrito(id), variante_id → producto_variante(id)
Atributos: cantidad

Tabla: nota_venta

PK: id (UUID) | FK: usuario_id → usuario(id), cliente_id → cliente(id), direccion_id → direccion(id), cupon_id → cupon(id)
Atributos: nro, fecha, subtotal, descuento, costo_envio, total, estado

Tabla: detalle_nota_venta

PK: id (UUID) | FK: notaventa_id → nota_venta(id), producto_id → producto(id), variante_id → producto_variante(id)
Atributos: cantidad, precio_unitario, subtotal

Tabla: pago

PK: id (UUID) | FK: notaventa_id → nota_venta(id), metodopago_id → metodo_pago(id)
Atributos: monto, transaccion_id, estado, fecha

Tabla: envio

PK: id (UUID) | FK: notaventa_id → nota_venta(id), metodoenvio_id → metodo_envio(id)
Atributos: costo, guia_transporte, estado, fecha_despacho

3.3.1.3. Normalización

Normalización en 1FN, 2FN y 3FN para garantizar integridad referencial y erradicar redundancias anómalas.

3.3.2. Diseño de datos físico

3.3.2.1. Tabla de volumen

Tabla / Entidad	Tuplas Iniciales	Crecimiento Mensual	Volumen Estimado (1 Año)
usuario y cliente	150	+ 500 usuarios	~ 6,150 registros
producto	40	+ 15 prendas	~ 220 productos
producto_variante	160	+ 60 variantes	~ 880 variantes (SKUs)
nota_venta	20	+ 450 pedidos	~ 5,420 órdenes
detalle_nota_venta	45	+ 1,200 ítems	~ 14,445 registros
pago	20	+ 450 cobros	~ 5,420 transacciones
bitacora (auditoría)	500	+ 4,000 eventos	~ 48,500 registros

3.3.2.2. Script

Script oficial DDL en PostgreSQL 16:

```
-- DDL OFICIAL EN POSTGRESQL 16 (ecommerce_db)
CREATE EXTENSION IF NOT EXISTS "uuid-ossp";

CREATE TABLE rol (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    nombre VARCHAR(50) UNIQUE NOT NULL,
    descripcion VARCHAR(200)
);

CREATE TABLE categoria (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    categoria_padre_id UUID REFERENCES categoria(id) ON DELETE SET NULL,
    nombre VARCHAR(100) NOT NULL,
    slug VARCHAR(120) UNIQUE NOT NULL,
    descripcion TEXT,
    activa BOOLEAN NOT NULL DEFAULT true
);

CREATE TABLE talla (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    nombre VARCHAR(20) UNIQUE NOT NULL,
    descripcion VARCHAR(50)
);
```

```
CREATE TABLE color (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    nombre VARCHAR(50) UNIQUE NOT NULL,
    hex VARCHAR(10)
);

CREATE TABLE metodo_pago (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    nombre VARCHAR(50) UNIQUE NOT NULL,
    activo BOOLEAN NOT NULL DEFAULT true
);

CREATE TABLE metodo_envio (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    nombre VARCHAR(100) NOT NULL,
    descripcion TEXT,
    costo NUMERIC(10,2) NOT NULL DEFAULT 0,
    tiempo_estimado VARCHAR(50),
    activo BOOLEAN NOT NULL DEFAULT true
);

CREATE TABLE cupon (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    codigo VARCHAR(50) UNIQUE NOT NULL,
    tipo VARCHAR(20) NOT NULL,
    valor NUMERIC(10,2) NOT NULL,
    limite_usos INTEGER DEFAULT 100,
    usos_actuales INTEGER DEFAULT 0,
    fecha_expiracion TIMESTAMP,
    activo BOOLEAN NOT NULL DEFAULT true
);

CREATE TABLE usuario (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    rol_id UUID NOT NULL REFERENCES rol(id) ON DELETE RESTRICT,
    nombre VARCHAR(100) NOT NULL,
    email VARCHAR(150) UNIQUE NOT NULL,
    password VARCHAR(255) NOT NULL,
    activo BOOLEAN NOT NULL DEFAULT true,
    creado_en TIMESTAMP NOT NULL DEFAULT now(),
    actualizado_en TIMESTAMP NOT NULL DEFAULT now()
);

CREATE TABLE cliente (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    usuario_id UUID NOT NULL UNIQUE REFERENCES usuario(id) ON DELETE CASCADE,
    ci_nit VARCHAR(30),
    telefono VARCHAR(30),
    avatar_url VARCHAR(255)
);

CREATE TABLE direccion (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    cliente_id UUID NOT NULL REFERENCES cliente(id) ON DELETE CASCADE,
    departamento VARCHAR(50) NOT NULL,
    ciudad VARCHAR(50) NOT NULL,
    calle VARCHAR(150) NOT NULL,
    referencia VARCHAR(200),
    es_predeterminada BOOLEAN NOT NULL DEFAULT false
);

CREATE TABLE producto (
```

```
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    categoria_id UUID NOT NULL REFERENCES categoria(id) ON DELETE RESTRICT,
    nombre VARCHAR(150) NOT NULL,
    descripcion TEXT,
    precio NUMERIC(10,2) NOT NULL DEFAULT 0,
    activo BOOLEAN NOT NULL DEFAULT true,
    destacado BOOLEAN NOT NULL DEFAULT false,
    creado_en TIMESTAMP NOT NULL DEFAULT now(),
    actualizado_en TIMESTAMP NOT NULL DEFAULT now()
);

CREATE TABLE producto_variante (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    producto_id UUID NOT NULL REFERENCES producto(id) ON DELETE CASCADE,
    talla_id UUID REFERENCES talla(id) ON DELETE SET NULL,
    color_id UUID REFERENCES color(id) ON DELETE SET NULL,
    sku VARCHAR(50) UNIQUE NOT NULL,
    stock INTEGER NOT NULL DEFAULT 0 CHECK (stock >= 0),
    stock_minimo INTEGER NOT NULL DEFAULT 5,
    precio_extra NUMERIC(10,2) NOT NULL DEFAULT 0,
    activa BOOLEAN NOT NULL DEFAULT true,
    CONSTRAINT uq_variante UNIQUE(producto_id, talla_id, color_id)
);

CREATE TABLE imagen_producto (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    producto_id UUID NOT NULL REFERENCES producto(id) ON DELETE CASCADE,
    url VARCHAR(255) NOT NULL,
    es_portada BOOLEAN NOT NULL DEFAULT false,
    orden INTEGER NOT NULL DEFAULT 0
);

CREATE TABLE carrito (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    usuario_id UUID REFERENCES usuario(id) ON DELETE CASCADE,
    actualizado_en TIMESTAMP NOT NULL DEFAULT now()
);

CREATE TABLE detalle_carrito (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    carrito_id UUID NOT NULL REFERENCES carrito(id) ON DELETE CASCADE,
    variante_id UUID NOT NULL REFERENCES producto_variante(id) ON DELETE CASCADE,
    cantidad INTEGER NOT NULL DEFAULT 1 CHECK (cantidad > 0),
    CONSTRAINT uq_carrito_variante UNIQUE(carrito_id, variante_id)
);

CREATE TABLE nota_venta (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    nro VARCHAR(20) UNIQUE NOT NULL,
    usuario_id UUID NOT NULL REFERENCES usuario(id) ON DELETE RESTRICT,
    cliente_id UUID REFERENCES cliente(id) ON DELETE SET NULL,
    direccion_id UUID REFERENCES direccion(id) ON DELETE SET NULL,
    cupon_id UUID REFERENCES cupon(id) ON DELETE SET NULL,
    fecha TIMESTAMP NOT NULL DEFAULT now(),
    subtotal NUMERIC(10,2) NOT NULL DEFAULT 0,
    descuento NUMERIC(10,2) NOT NULL DEFAULT 0,
    costo_envio NUMERIC(10,2) NOT NULL DEFAULT 0,
    total NUMERIC(10,2) NOT NULL DEFAULT 0,
    estado VARCHAR(30) NOT NULL DEFAULT 'PENDIENTE'
);

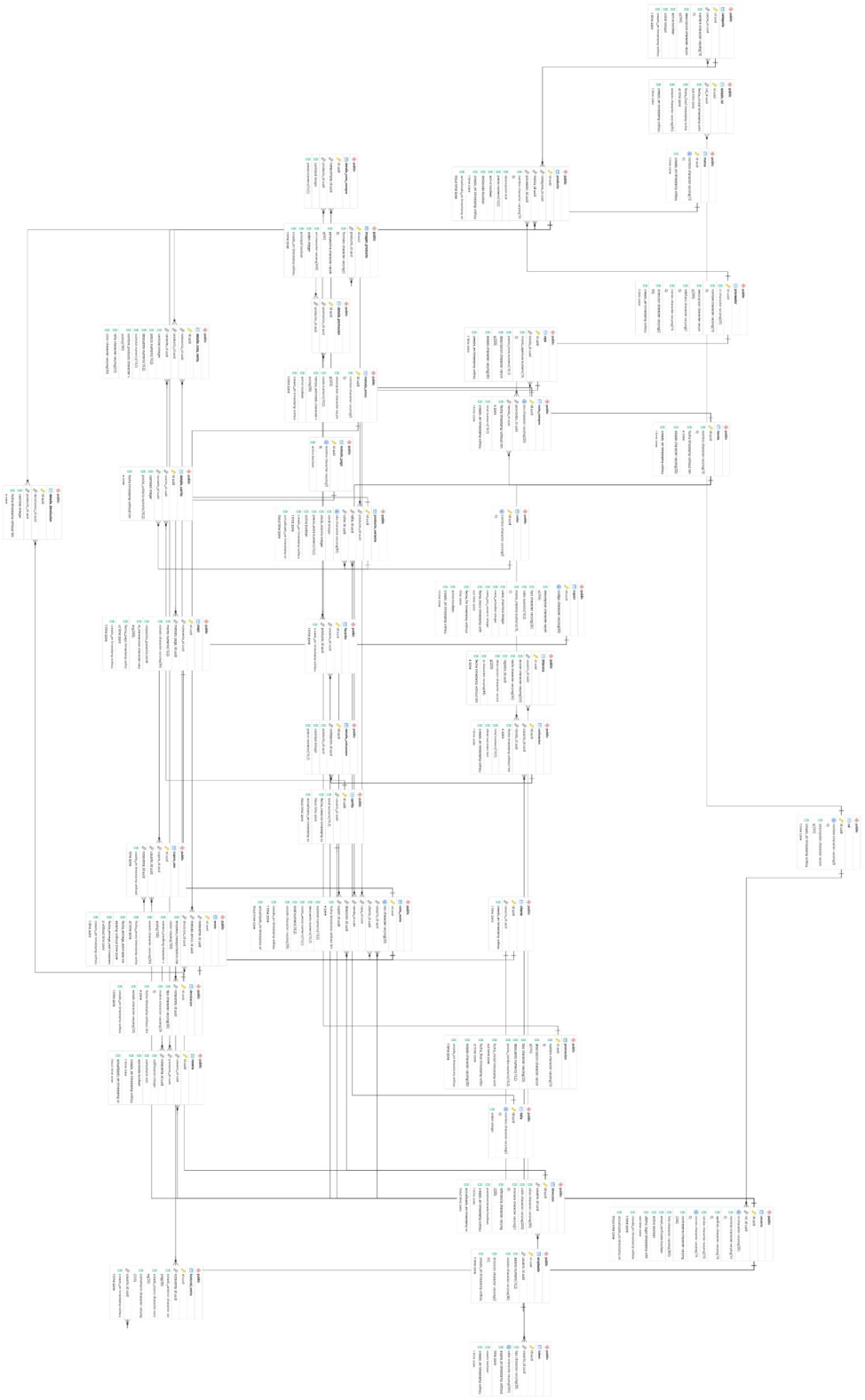
CREATE TABLE detalle_nota_venta (
```

```
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    notaventa_id UUID NOT NULL REFERENCES nota_venta(id) ON DELETE CASCADE,
    producto_id UUID NOT NULL REFERENCES producto(id) ON DELETE RESTRICT,
    variante_id UUID REFERENCES producto_variante(id) ON DELETE SET NULL,
    cantidad INTEGER NOT NULL CHECK (cantidad > 0),
    precio_unitario NUMERIC(10,2) NOT NULL DEFAULT 0,
    subtotal NUMERIC(10,2) NOT NULL DEFAULT 0
);

CREATE TABLE pago (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    notaventa_id UUID NOT NULL REFERENCES nota_venta(id) ON DELETE CASCADE,
    metodopago_id UUID NOT NULL REFERENCES metodo_pago(id),
    monto NUMERIC(10,2) NOT NULL,
    transaccion_id VARCHAR(100),
    estado VARCHAR(30) NOT NULL DEFAULT 'COMPLETADO',
    fecha TIMESTAMP NOT NULL DEFAULT now()
);

CREATE TABLE bitacora (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    usuario_id UUID REFERENCES usuario(id) ON DELETE SET NULL,
    fecha_hora TIMESTAMP NOT NULL DEFAULT now(),
    modulo VARCHAR(50) NOT NULL,
    descripcion VARCHAR(255) NOT NULL,
    ip VARCHAR(45) NOT NULL
);
```

3.3.2.3. Diagrama Relacional



3.3.2.4. Actualización de Tuplas

1.1.1.1.1 Insert()

```
-- INSERCIÓN REAL DE DATOS DE ALTA COSTURA PARA AURA
INSERT INTO rol (id, nombre, descripcion) VALUES
('a0000000-0000-0000-0000-000000000001', 'ADMIN', 'Administrador general del sistema y
directivos'),
('a0000000-0000-0000-0000-000000000002', 'CLIENTE', 'Cliente comprador en la tienda
virtual'),
('a0000000-0000-0000-0000-000000000003', 'EMPLEADO', 'Personal operativo de almacén e
inventario');

INSERT INTO categoria (id, nombre, slug, descripcion) VALUES
('c0000000-0000-0000-0000-000000000001', 'Vestidos de Gala', 'vestidos-de-gala',
'Vestidos largos de seda y corte sirena'),
('c0000000-0000-0000-0000-000000000002', 'Prendas Superiores', 'prendas-superiores',
'Blusas de lino y camisas de popelina'),
('c0000000-0000-0000-0000-000000000003', 'Faldas y Conjuntos', 'faldas-conjuntos',
'Faldas plisadas y satén perla');

INSERT INTO talla (id, nombre, descripcion) VALUES
('t0000000-0000-0000-0000-000000000001', 'XS', 'Extra Pequeño'),
('t0000000-0000-0000-0000-000000000002', 'S', 'Pequeño'),
('t0000000-0000-0000-0000-000000000003', 'M', 'Mediano'),
('t0000000-0000-0000-0000-000000000004', 'L', 'Grande');

INSERT INTO color (id, nombre, hex) VALUES
('k0000000-0000-0000-0000-000000000001', 'Blanco Crudo', '#FDFBF7'),
('k0000000-0000-0000-0000-000000000002', 'Verde Oliva', '#556B2F'),
('k0000000-0000-0000-0000-000000000003', 'Rojo Gala', '#8B0000'),
('k0000000-0000-0000-0000-000000000004', 'Camel Arena', '#C19A6B');

INSERT INTO producto (id, categoria_id, nombre, descripcion, precio, activo, destacado)
VALUES
('p0000000-0000-0000-0000-000000000001', 'c0000000-0000-0000-0000-000000000001', 'Vestido
Seda Oliva Gala', 'Vestido de seda natural con caída fluida', 180.00, true, true),
('p0000000-0000-0000-0000-000000000002', 'c0000000-0000-0000-0000-000000000001', 'Vestido
Lino Blanco Verano', 'Vestido midi en lino puro europeo', 140.00, true, true),
('p0000000-0000-0000-0000-000000000003', 'c0000000-0000-0000-0000-000000000002', 'Camisa
Popelina Blanca Atelier', 'Camisa clásica sastre con botones de nácar', 95.00, true,
false),
('p0000000-0000-0000-0000-000000000004', 'c0000000-0000-0000-0000-000000000003', 'Falda
Plisada Camel Lujo', 'Falda plisada de cintura alta', 110.00, true, true);

INSERT INTO producto_variante (id, producto_id, talla_id, color_id, sku, stock,
stock_minimo) VALUES
('v0000000-0000-0000-0000-000000000001', 'p0000000-0000-0000-0000-000000000001',
't0000000-0000-0000-0000-000000000002', 'k0000000-0000-0000-0000-000000000002', 'VEST-
OLI-S', 12, 3),
('v0000000-0000-0000-0000-000000000002', 'p0000000-0000-0000-0000-000000000001',
't0000000-0000-0000-0000-000000000003', 'k0000000-0000-0000-0000-000000000002', 'VEST-
OLI-M', 8, 3),
('v0000000-0000-0000-0000-000000000003', 'p0000000-0000-0000-0000-000000000002',
't0000000-0000-0000-0000-000000000002', 'k0000000-0000-0000-0000-000000000001', 'VEST-
BLA-S', 15, 4),
('v0000000-0000-0000-0000-000000000004', 'p0000000-0000-0000-0000-000000000003',
't0000000-0000-0000-0000-000000000003', 'k0000000-0000-0000-0000-000000000001', 'CAM-POP-
M', 20, 5),
('v0000000-0000-0000-0000-000000000005', 'p0000000-0000-0000-0000-000000000004',
't0000000-0000-0000-0000-000000000002', 'k0000000-0000-0000-0000-000000000004', 'FAL-CAM-
S', 6, 2);
```

1.1.1.1.2 Update()

```
UPDATE producto_variante SET stock = stock - 1 WHERE id = 'v0000000-0000-0000-0000-
000000000001' AND stock >= 1;
UPDATE nota_venta SET estado = 'EN_CAMINO' WHERE nro = 'NV-2026-0001' AND estado =
'PAGADO';
```

1.1.1.1.3 Delete()

```
UPDATE producto SET activo = false WHERE id = 'p0000000-0000-0000-0000-000000000004';
```

3.3.2.5. Consultas

1.1.1.1.4 Consultas simples

Consulta 1: Listar todas las prendas activas en el catálogo con su precio en USD.

```
SELECT id, nombre, precio FROM producto WHERE activo = true ORDER BY precio DESC;
```

id	nombre	precio (USD)
p0000000-0001	Vestido Seda Oliva Gala	180.00
p0000000-0002	Vestido Lino Blanco Verano	140.00
p0000000-0004	Falda Plisada Camel Lujo	110.00
p0000000-0003	Camisa Popelina Blanca Atelier	95.00

Consulta 2: Consultar las variantes con stock crítico inferior o igual al stock mínimo.

```
SELECT sku, stock, stock_minimo FROM producto_variante WHERE stock <= stock_minimo;
```

sku	stock	stock_minimo
FAL-CAM-S	2	2
VEST-OLI-M	3	3

1.1.1.1.5 Consultas múltiples

Consulta 16: Listar órdenes pagadas con datos del cliente, método de pago y monto liquidado.

```
SELECT nv.nro, u.nombre AS cliente, mp.nombre AS metodo, p.monto, p.transaccion_id
FROM nota_venta nv
JOIN usuario u ON nv.usuario_id = u.id
JOIN pago p ON p.notaventa_id = nv.id
JOIN metodo_pago mp ON p.metodopago_id = mp.id
```

```
WHERE nv.estado = 'PAGADO';
```

nro	cliente	metodo	monto	transaccion_id
NV-2026-0001	Camila Miranda	Stripe Tarjeta	180.00	pi_3Pabc1234567890
NV-2026-0002	Valeria Ortiz	Stripe Tarjeta	235.00	pi_3Pxyz0987654321

1.1.1.1.6 Subconsultas

Consulta 24: Identificar prendas cuyo precio de venta sea superior al promedio de todo el catálogo.

```
SELECT nombre, precio FROM producto WHERE precio > (SELECT AVG(precio) FROM producto) AND
activo = true;
```

nombre	precio (USD)	Promedio Catálogo
Vestido Seda Oliva Gala	180.00	131.25
Vestido Lino Blanco Verano	140.00	131.25

3.3.2.6. Procedimientos almacenados

```
CREATE OR REPLACE PROCEDURE sp_registrar_venta_atomica(
    p_nro VARCHAR, p_usuario_id UUID, p_direccion_id UUID, p_subtotal NUMERIC,
    p_flete NUMERIC, p_total NUMERIC, p_variante_id UUID, p_cantidad INT
)
LANGUAGE plpgsql AS $$
DECLARE v_stock_actual INT;
BEGIN
    SELECT stock INTO v_stock_actual FROM producto_variante WHERE id = p_variante_id FOR
UPDATE;
    IF v_stock_actual < p_cantidad THEN
        RAISE EXCEPTION 'Stock insuficiente para la variante %', p_variante_id;
    END IF;
    INSERT INTO nota_venta (nro, usuario_id, direccion_id, subtotal, costo_envio, total,
estado)
    VALUES (p_nro, p_usuario_id, p_direccion_id, p_subtotal, p_flete, p_total, 'PAGADO');
    UPDATE producto_variante SET stock = stock - p_cantidad WHERE id = p_variante_id;
    INSERT INTO bitacora (usuario_id, modulo, descripcion, ip)
    VALUES (p_usuario_id, 'VENTAS', 'Venta exitosa ' || p_nro, '127.0.0.1');
END; $$;
```

3.3.2.7. Triggers

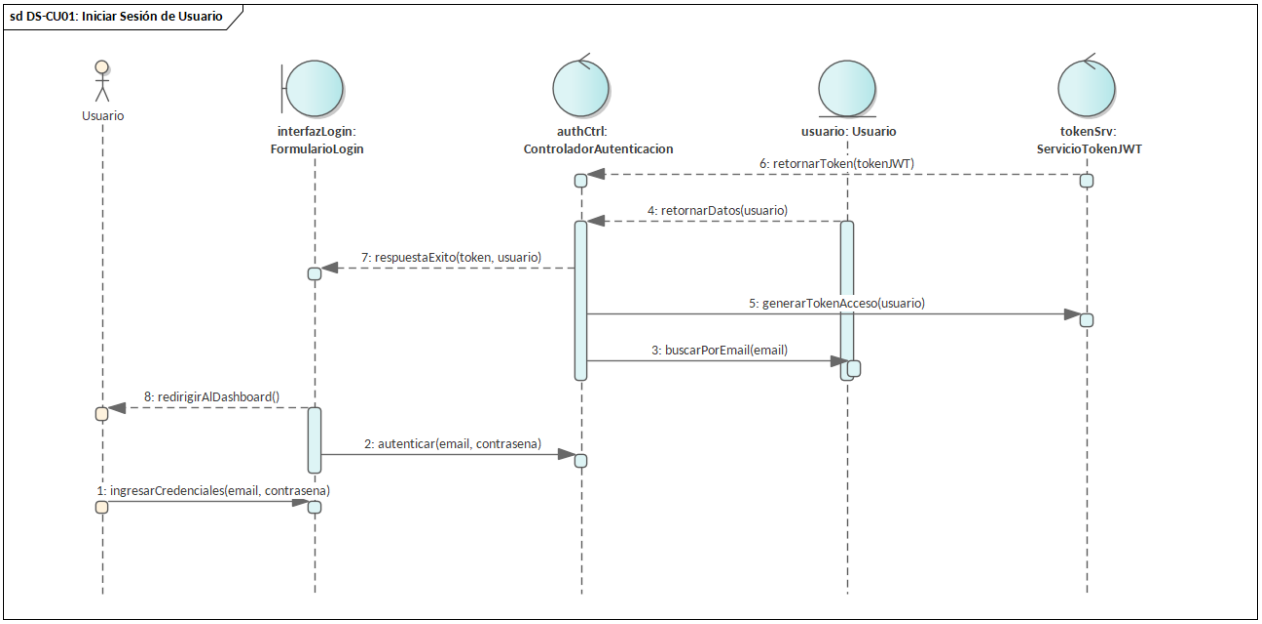
```
CREATE OR REPLACE FUNCTION fn_auditar_cambio_stock() RETURNS TRIGGER AS $$
BEGIN
    IF NEW.stock <> OLD.stock THEN
        INSERT INTO bitacora (modulo, descripcion, ip)
        VALUES ('INVENTARIO', 'Variante ' || NEW.sku || ' stock cambió de ' || OLD.stock
|| ' a ' || NEW.stock, '127.0.0.1');
```

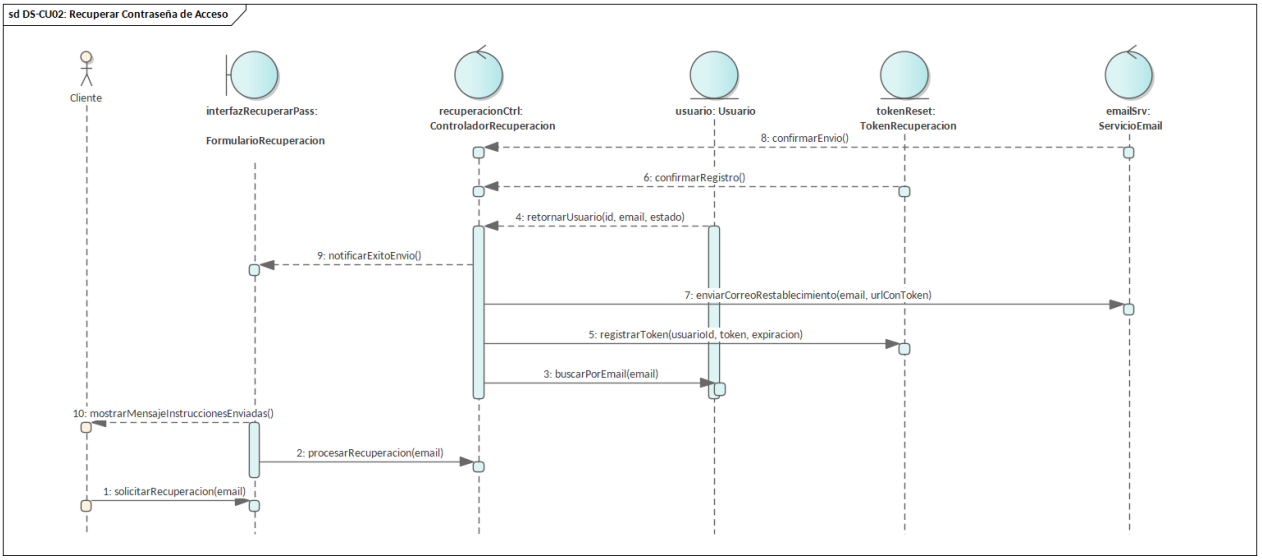
```
    END IF;
    RETURN NEW;
END; $$ LANGUAGE plpgsql;

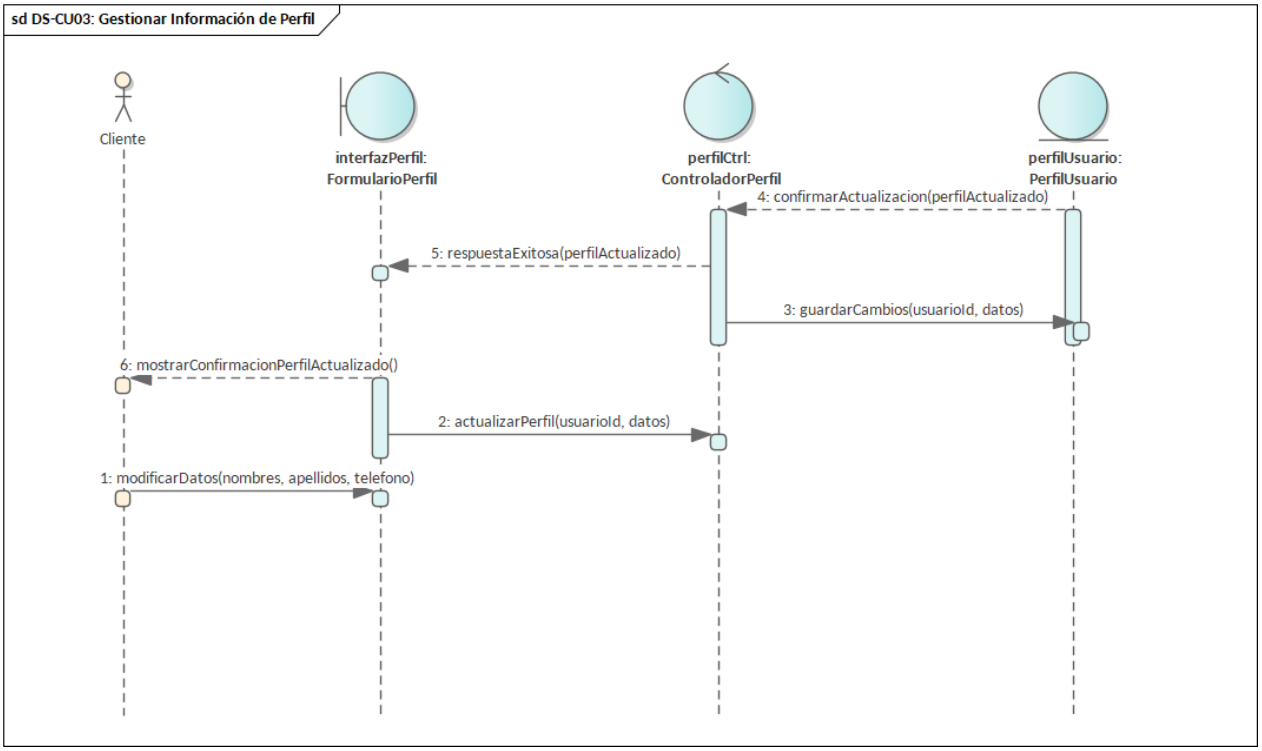
CREATE TRIGGER trg_auditar_stock AFTER UPDATE ON producto_variante
FOR EACH ROW EXECUTE FUNCTION fn_auditar_cambio_stock();
```

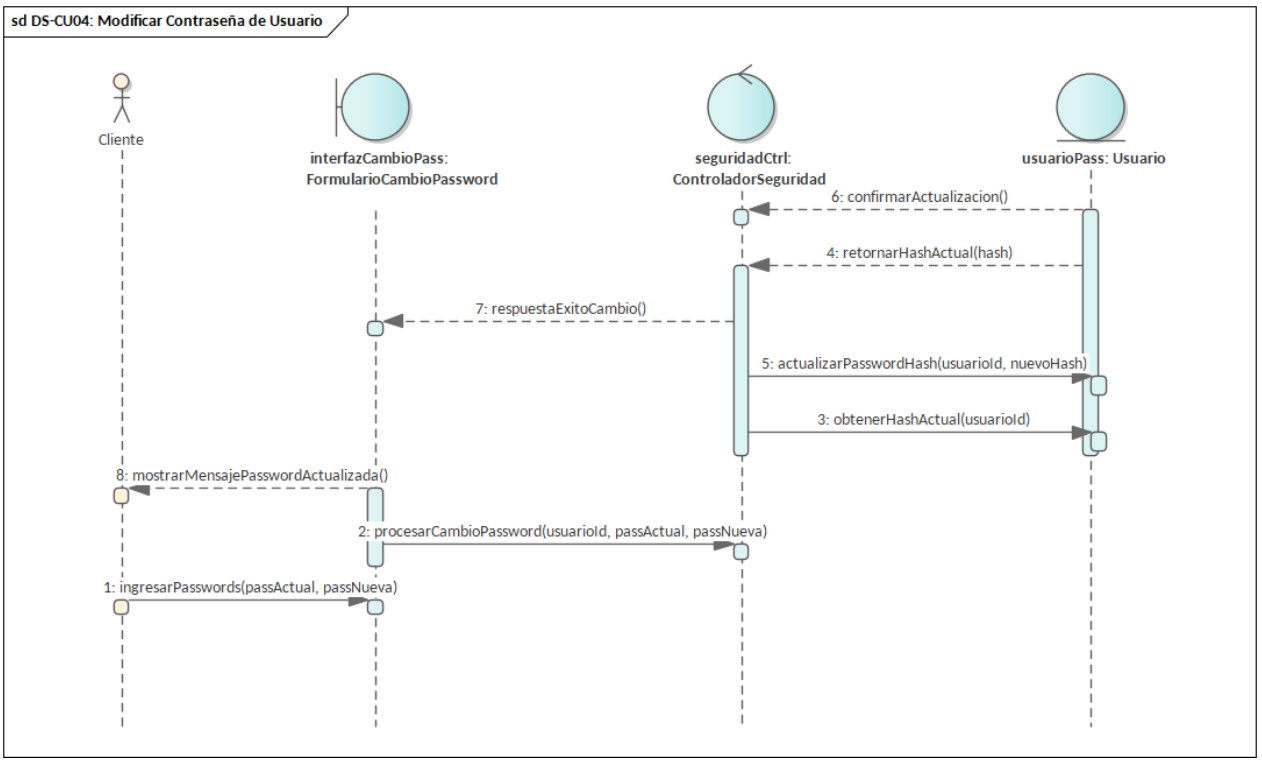
3.4. Diseñar caso de uso

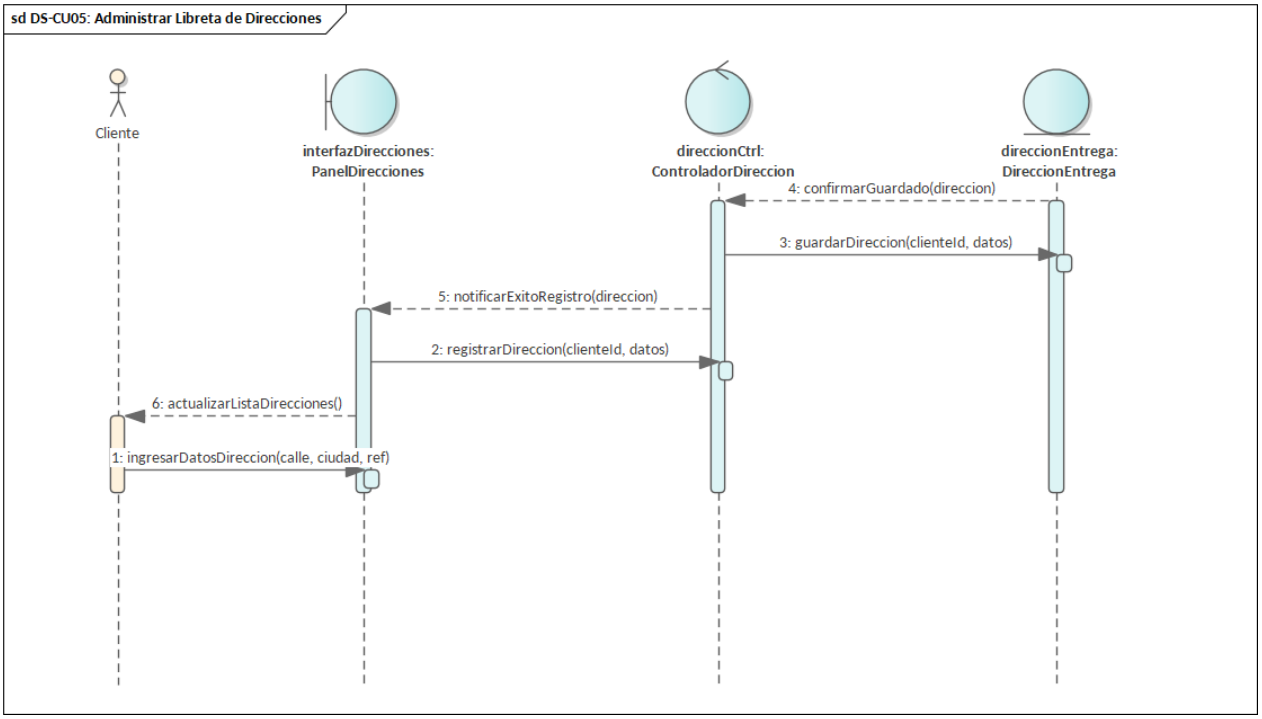
PARA EL CICLO #1: SEGURIDAD Y USUARIOS

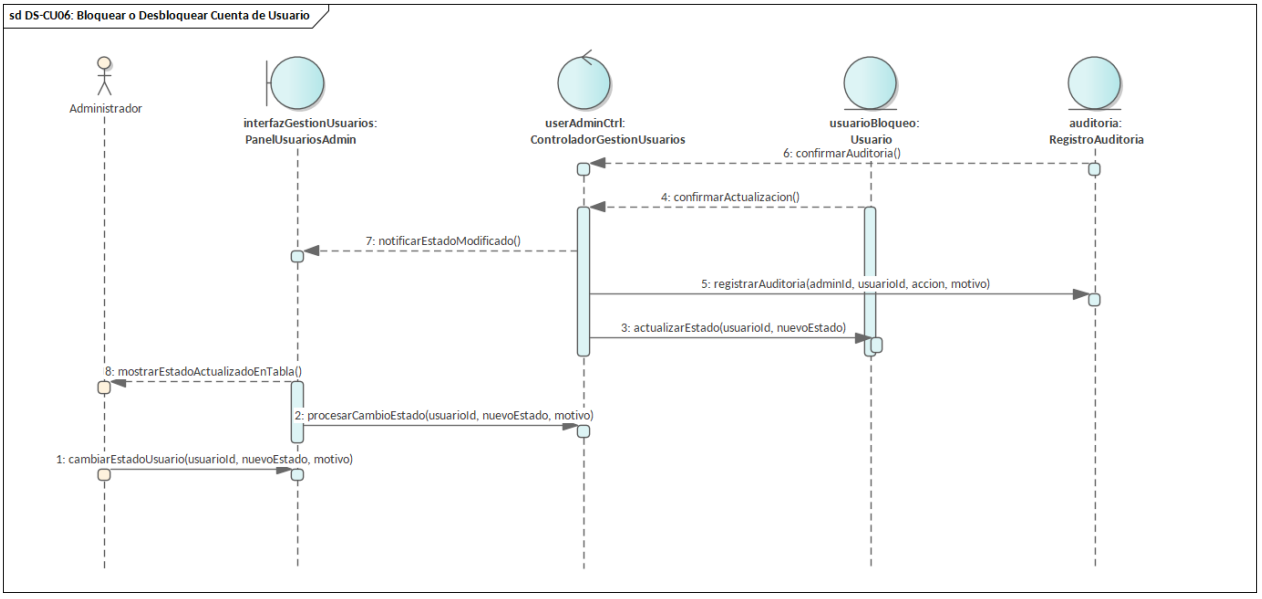


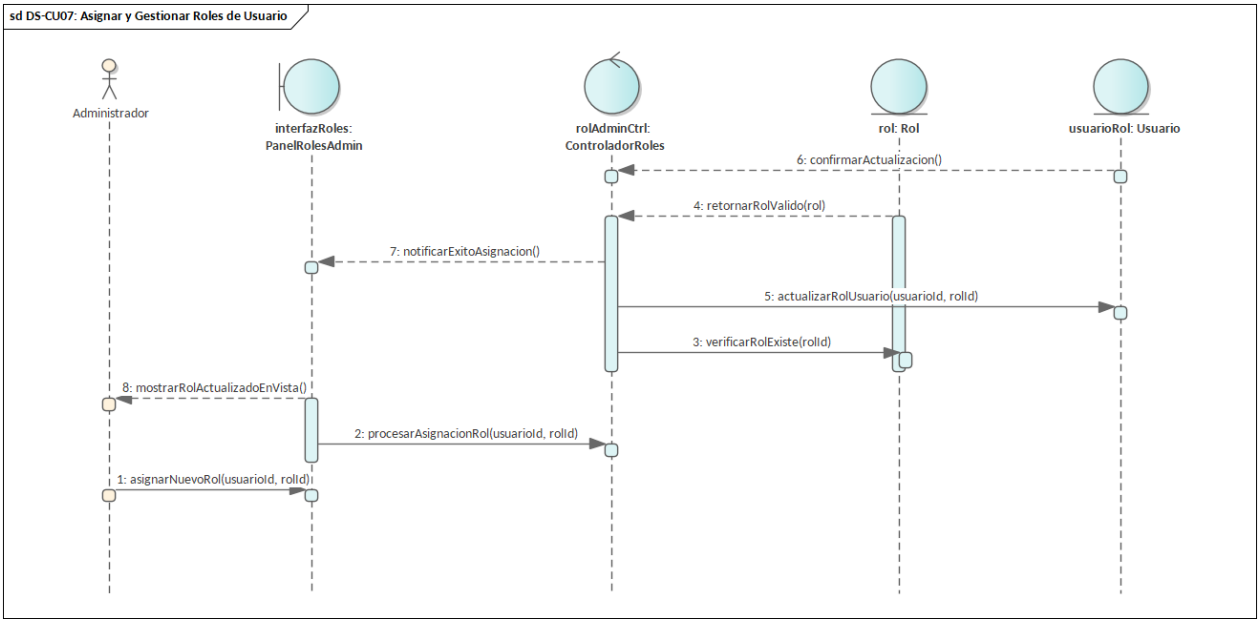


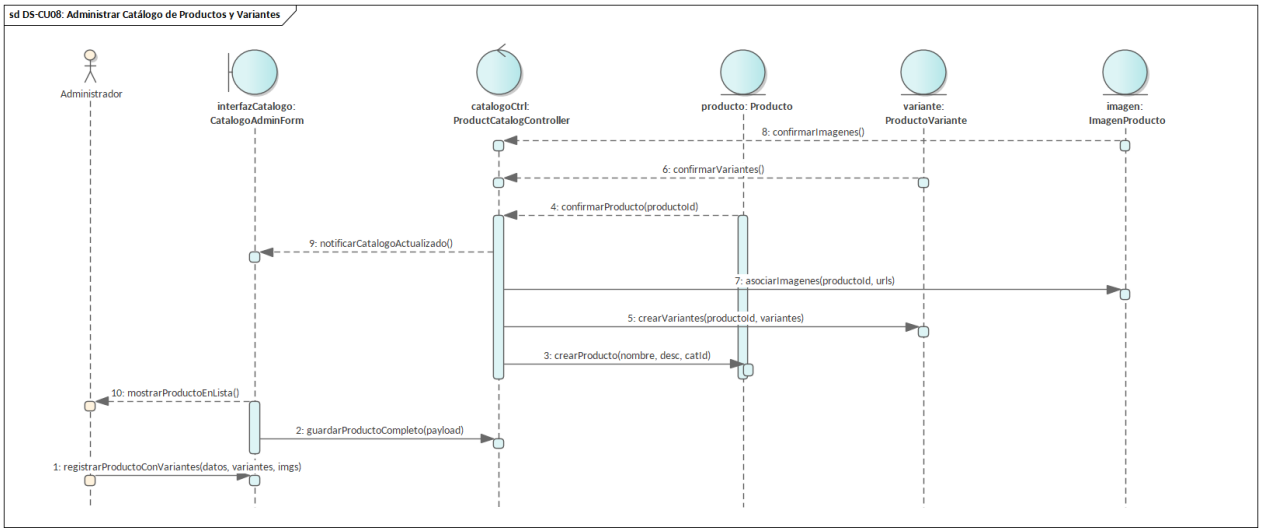

sd DS-CU03: Gestionar Información de Perfil
Cliente
interfazPerfil:
FormularioPerfil
perfilCtrl:
ControladorPerfil
perfilUsuario:
PerfilUsuario
4: confirmarActualizacion(perfilActualizado)
5: respuestaExitosa(perfilActualizado)
3: guardarCambios(usuarioId, datos)
6: mostrarConfirmacionPerfilActualizado()
2: actualizarPerfil(usuarioId, datos)
1: modificarDatos(nombres, apellidos, telefono)

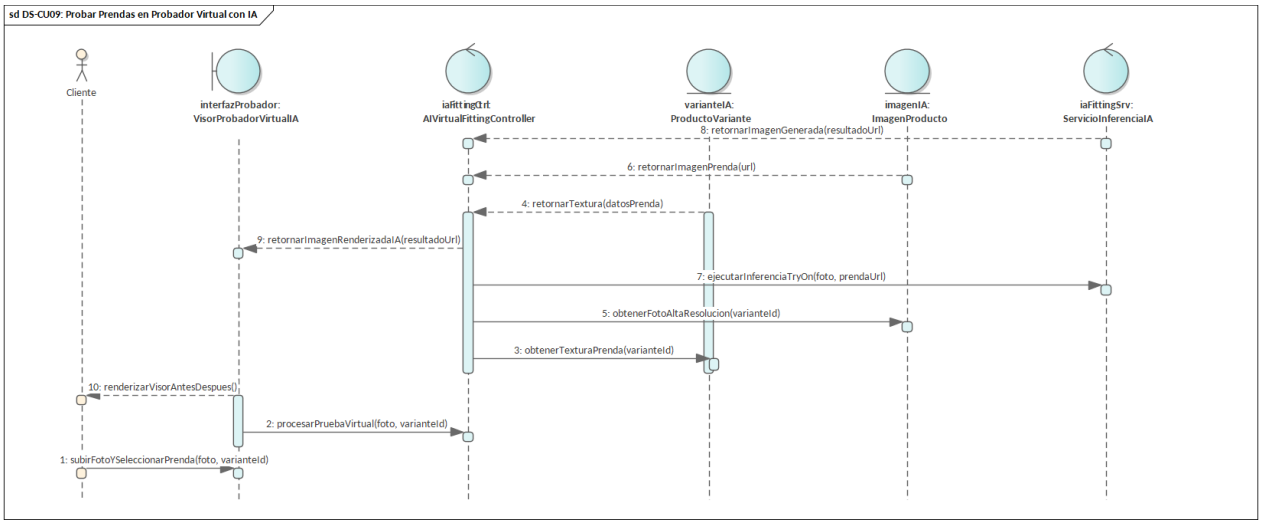

sd DS-CU04: Modificar Contraseña de Usuario
Cliente
interfazCambioPass:
FormularioCambioPassword
seguridadCtrl:
ControladorSeguridad
usuarioPass: Usuario
6: confirmarActualizacion()
4: retornarHashActual(hash)
7: respuestaExitoCambio()
5: actualizarPasswordHash(usuarioId, nuevoHash)
3: obtenerHashActual(usuarioId)
8: mostrarMensajePasswordActualizada()
2: procesarCambioPassword(usuarioId, passActual, passNueva)
1: ingresarPasswords(passActual, passNueva)

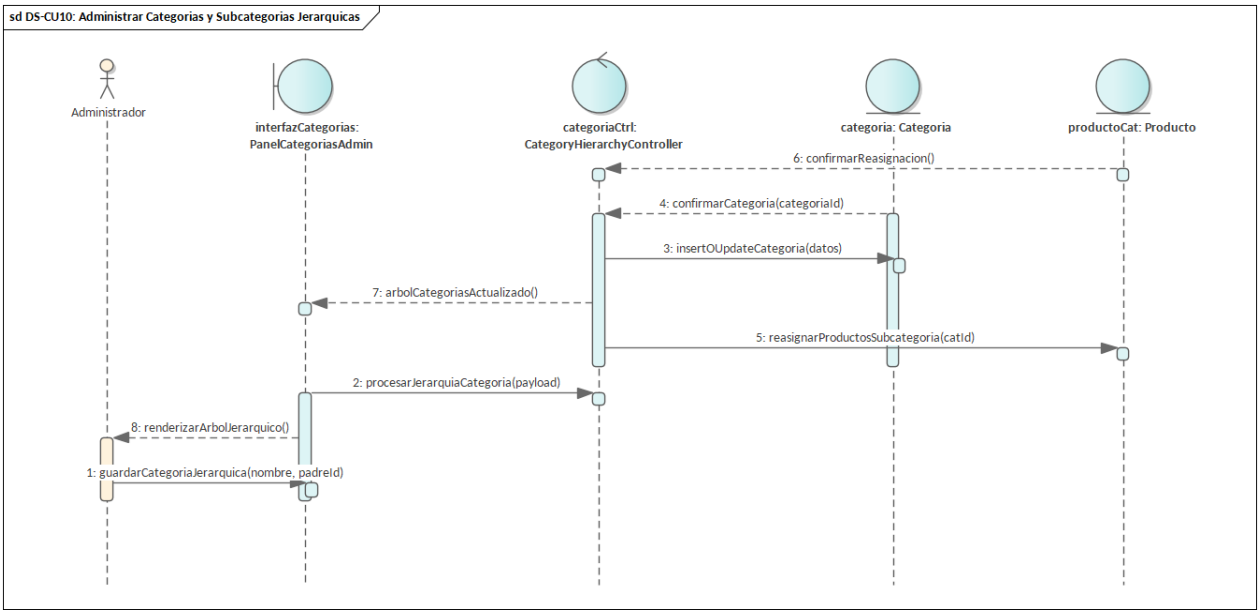

sd DS-CU05: Administrar Libreta de Direcciones
Cliente
interfazDirecciones:
PanelDirecciones
direccionCtrl:
ControladorDireccion
direccionEntrega:
DireccionEntrega
4: confirmarGuardado(direccion)
3: guardarDireccion(clienteId, datos)
5: notificarExitoRegistro(direccion)
2: registrarDireccion(clienteId, datos)
6: actualizarListaDirecciones()
1: ingresarDatosDireccion(calle, ciudad, ref)

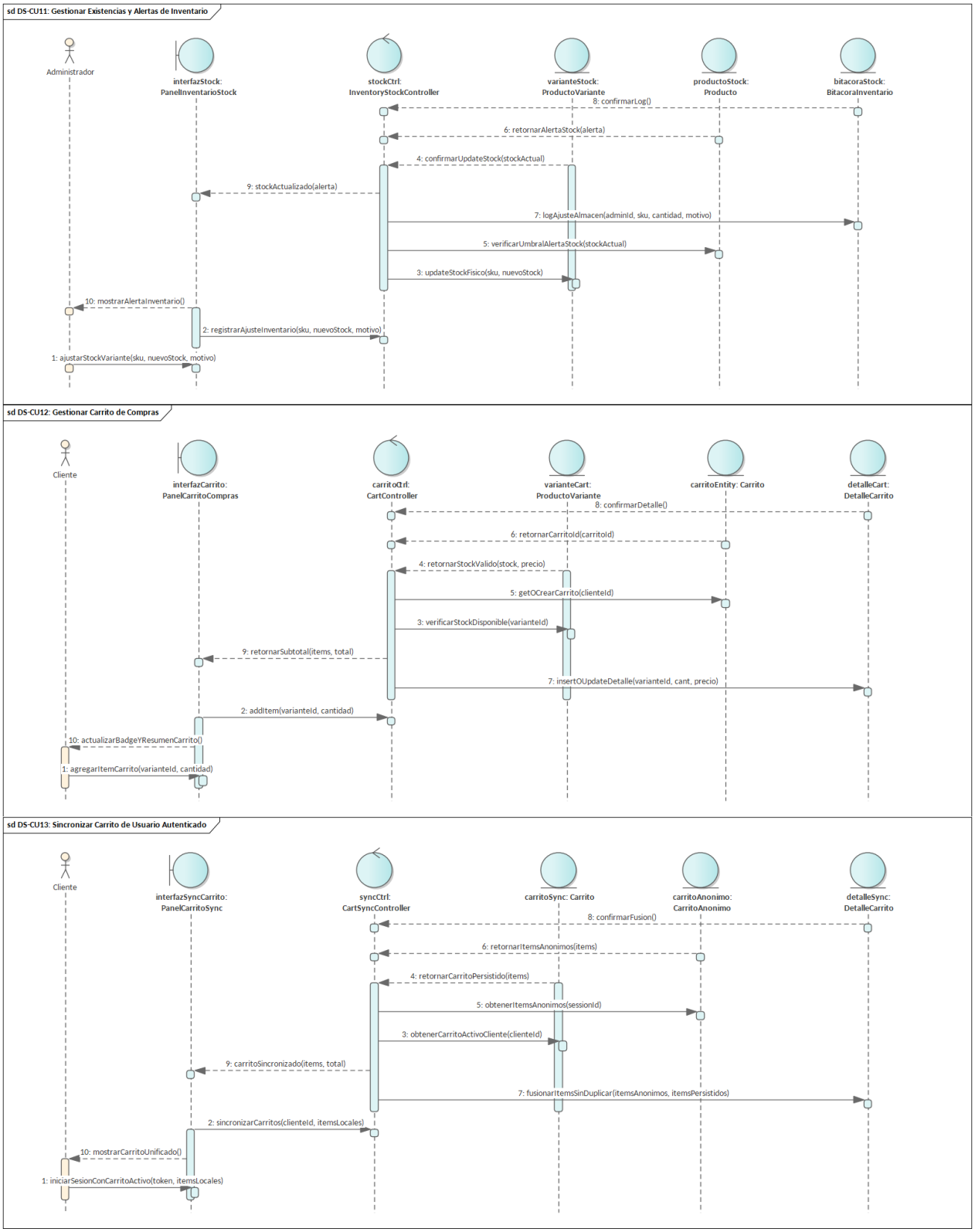

sd DS-CU06: Bloquear o Desbloquear Cuenta de Usuario
Administrador
interfazGestionUsuarios:
PanelUsuariosAdmin
userAdminCtrl:
ControladorGestionUsuarios
usuarioBloqueo:
Usuario
auditoria:
RegistroAuditoria
6: confirmarAuditoria()
4: confirmarActualizacion()
7: notificarEstadoModificado()
5: registrarAuditoria(adminId, usuarioId, accion, motivo)
3: actualizarEstado(usuarioId, nuevoEstado)
8: mostrarEstadoActualizadoEnTabla()
2: procesarCambioEstado(usuarioId, nuevoEstado, motivo)
1: cambiarEstadoUsuario(usuarioId, nuevoEstado, motivo)


sd DS-CU07: Asignar y Gestionar Roles de Usuario
Administrador
interfazRoles: PanelRolesAdmin
rolAdminCtrl: ControladorRoles
rol: Rol
usuarioRol: Usuario
6: confirmarActualizacion()
4: retornarRolValido(rol)
7: notificarExitoAsignacion()
5: actualizarRolUsuario(usuarioId, rolId)
3: verificarRolExiste(rolId)
8: mostrarRolActualizadoEnVista()
2: procesarAsignacionRol(usuarioId, rolId)
1: asignarNuevoRol(usuarioId, rolId)

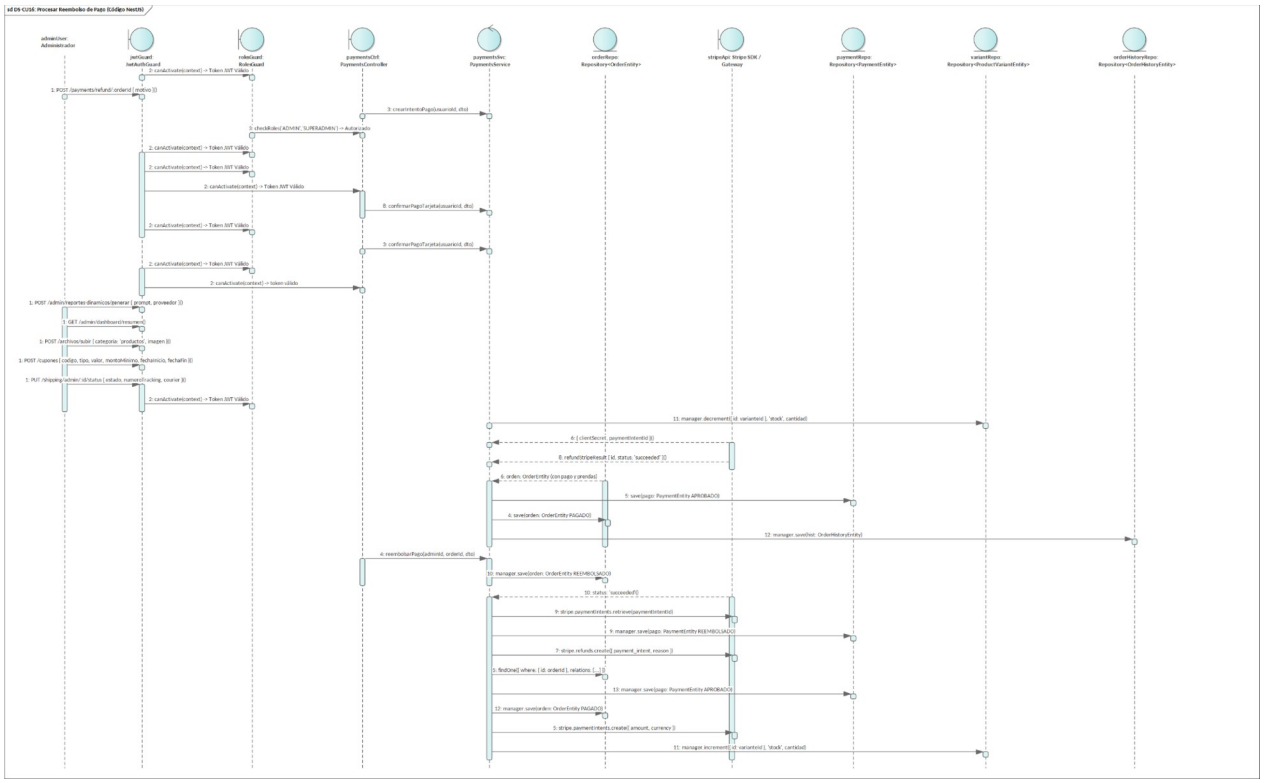

sd DS-CU08: Administrar Catálogo de Productos y Variantes
Administrador
interfazCatalogo: CatalogoAdminForm
catalogoCtrl: ProductCatalogController
producto: Producto
variante: ProductoVariante
imagen: ImagenProducto
8: confirmarImagenes()
6: confirmarVariantes()
4: confirmarProducto(productoId)
9: notificarCatalogoActualizado()
7: asociarImagenes(productoId, urls)
5: crearVariantes(productoId, variantes)
3: crearProducto(nombre, desc, catId)
10: mostrarProductoEnLista()
2: guardarProductoCompleto(payload)
1: registrarProductoConVariantes(datos, variantes, imgs)

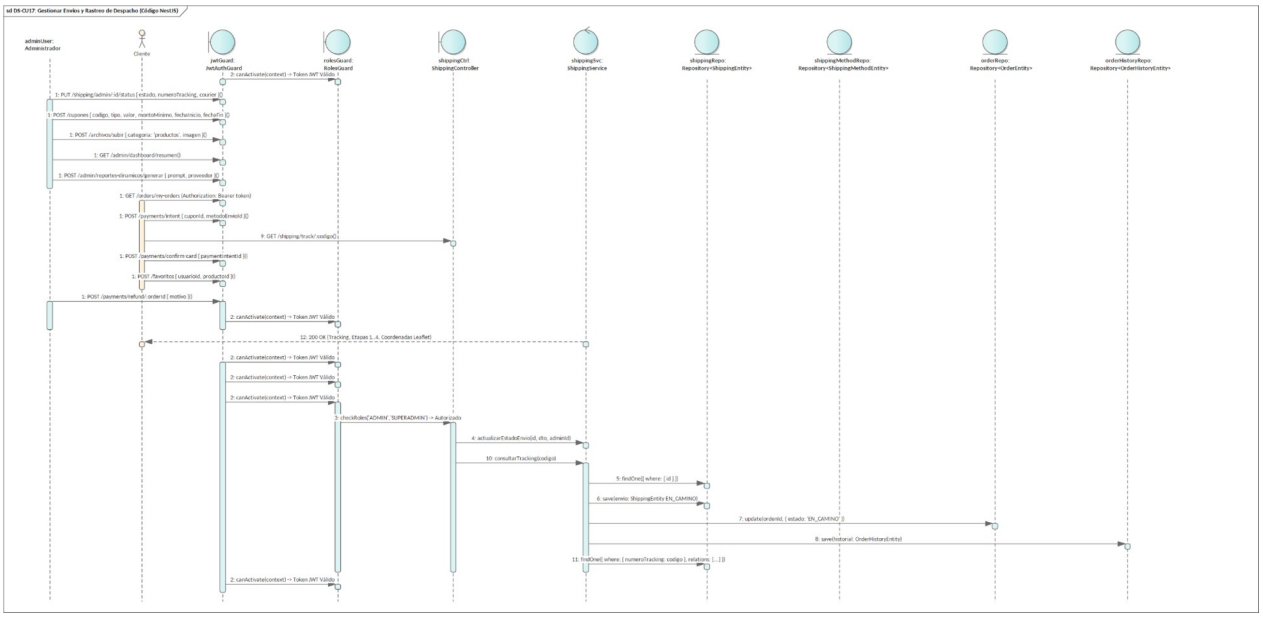

sd DS-CU09: Probar Prendas en Probador Virtual con IA
Cliente
interfazProbador: VisorProbadorVirtualIA
iaFittingCtrl: AIVirtualFittingController
varianteIA: ProductoVariante
imagenIA: ImagenProducto
iaFittingSrv: ServicioInferenciaIA
8: retornarImagenGenerada(resultadoUrl)
6: retornarImagenPrenda(url)
4: retornarTextura(datosPrenda)
9: retornarImagenRenderizadaIA(resultadoUrl)
7: ejecutarInferenciaTryOn(foto, prendaUrl)
5: obtenerFotoAltaResolucion(varianteId)
3: obtenerTexturaPrenda(varianteId)
10: renderizarVisorAntesDespues()
2: procesarPruebaVirtual(foto, varianteId)
1: subirFotoYSeleccionarPrenda(foto, varianteId)

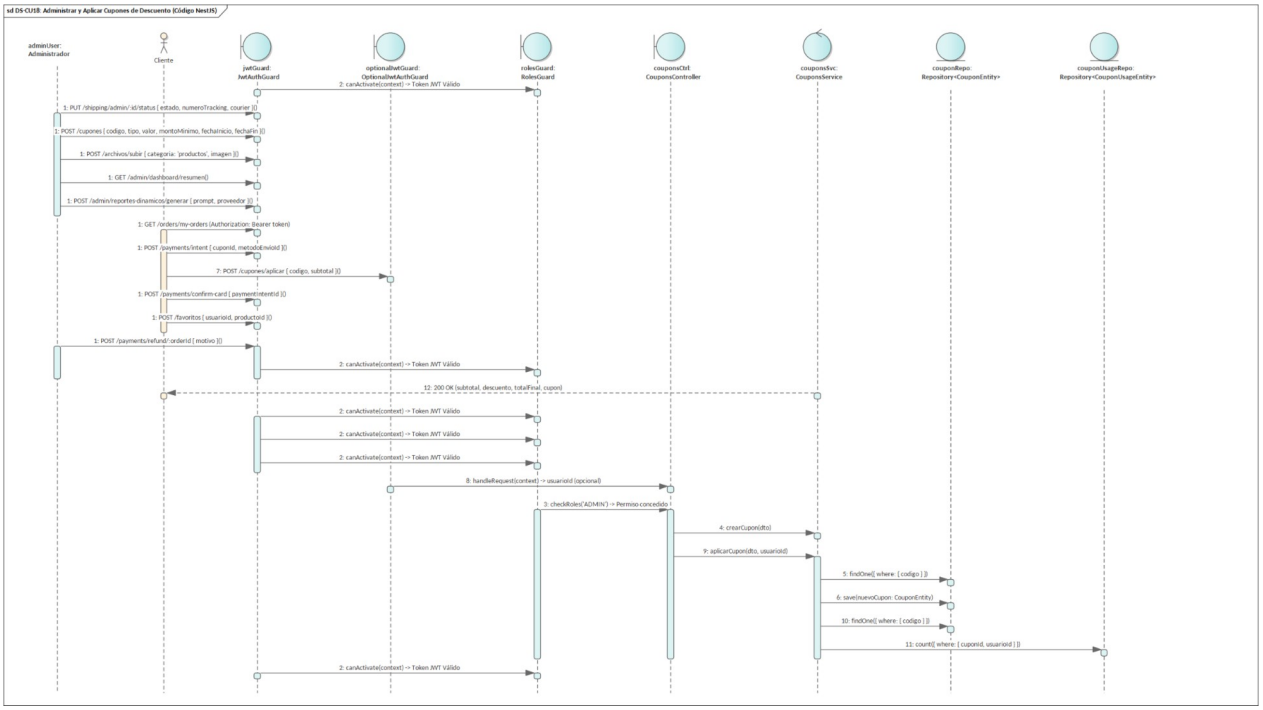

sd DS-CU10: Administrar Categorias y Subcategorias Jerarquicas
Administrador
interfazCategorias: PanelCategoriasAdmin
categoriaCtrl: CategoryHierarchyController
categoria: Categoria
productoCat: Producto
6: confirmarReasignacion()
4: confirmarCategoria(categoriaId)
3: insertOUpdateCategoria(datos)
7: arbolCategoriasActualizado()
5: reasignarProductosSubcategoria(catId)
2: procesarJerarquiaCategoria(payload)
8: renderizarArbolJerarquico()
1: guardarCategoriaJerarquica(nombre, padreId)

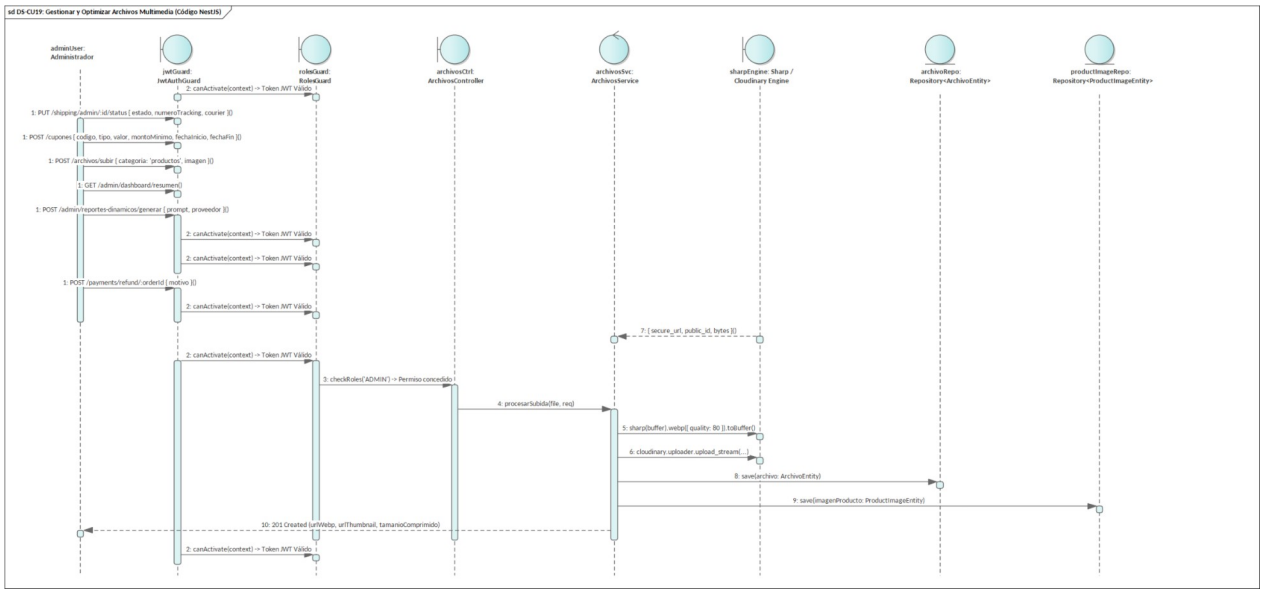

sd DS-CU11: Gestionar Existencias y Alertas de Inventario
Administrador
interfazStock: PanelInventarioStock
stockCtrl: InventoryStockController
varianteStock: ProductoVariante
productoStock: Producto
bitacoraStock: BitacoraInventario
8: confirmarLog()
6: retornarAlertaStock(alerta)
4: confirmarUpdateStock(stockActual)
9: stockActualizado(alerta)
7: logAjusteAlmacen(adminId, sku, cantidad, motivo)
5: verificarUmbralAlertaStock(stockActual)
3: updateStockFisico(sku, nuevoStock)
10: mostrarAlertaInventario()
2: registrarAjusteInventario(sku, nuevoStock, motivo)
1: ajustarStockVariante(sku, nuevoStock, motivo)
sd DS-CU12: Gestionar Carrito de Compras
Cliente
interfazCarrito: PanelCarritoCompras
carritoCtrl: CartController
varianteCart: ProductoVariante
carritoEntity: Carrito
detalleCart: DetalleCarrito
8: confirmarDetalle()
6: retornarCarritoId(carritoId)
4: retornarStockValido(stock, precio)
5: getOCrearCarrito(clienteId)
3: verificarStockDisponible(varianteId)
9: retornarSubtotal(items, total)
7: insertOUpdateDetalle(varianteId, cant, precio)
2: addItem(varianteId, cantidad)
10: actualizarBadgeYResumenCarrito()
1: agregarItemCarrito(varianteId, cantidad)
sd DS-CU13: Sincronizar Carrito de Usuario Autenticado
Cliente
interfazSyncCarrito: PanelCarritoSync
syncCtrl: CartSyncController
carritoSync: Carrito
carritoAnonimo: CarritoAnonimo
detalleSync: DetalleCarrito
8: confirmarFusion()
6: retornarItemsAnonimos(items)
4: retornarCarritoPersistido(items)
5: obtenerItemsAnonimos(sessionId)
3: obtenerCarritoActivoCliente(clienteId)
9: carritoSincronizado(items, total)
7: fusionarItemsSinDuplicar(itemsAnonimos, itemsPersistidos)
2: sincronizarCarritos(clienteId, itemsLocales)
10: mostrarCarritoUnificado()
1: iniciarSesionConCarritoActivo(token, itemsLocales)

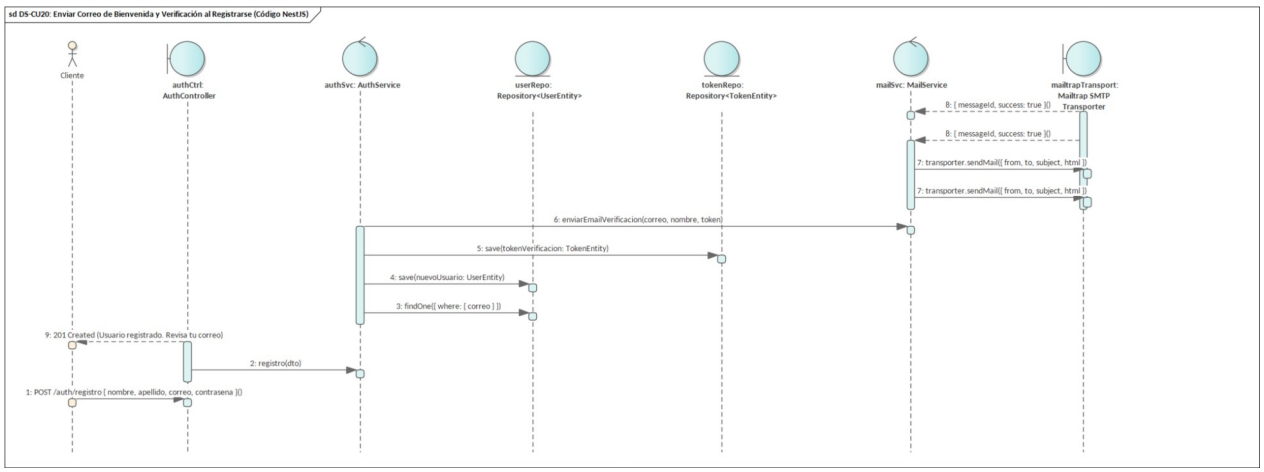

sd DS-CU15: Procesar Pago de Orden (Código Nest/S)
Cliente
jwtGuard: JwtAuthGuard
paymentsCtrl: PaymentsController
paymentsSvc: PaymentsService
stripeApi: Stripe SDK / Gateway
cartRepo: Repository<CartEntity>
orderRepo: Repository<OrderEntity>
paymentRepo: Repository<PaymentEntity>
variantRepo: Repository<ProductVariantEntity>
13: manager.save(pago: PaymentEntity APROBADO)
1: POST /payments/intent { cuponId, metodoEnvioId }()
7: POST /payments/confirm-card { paymentIntentId }()
1: POST /payments/confirm-card { paymentIntentId }()
1: POST /favoritos { usuarioId, productoId }()
2: canActivate(context) -> token válido
2: canActivate(context) -> Token JWT Válido
3: crearIntentoPago(usuarioId, dto)
4: reembolsarPago(adminId, orderId, dto)
8: confirmarPagoTarjeta(usuarioId, dto)
3: confirmarPagoTarjeta(usuarioId, dto)
9: 200 OK (Pago Aprobado & Orden Creada con Factura Digital)
4: findOne({ where: { usuarioId }, relations: ['items'] })
1: GET /orders/my-orders (Authorization: Bearer token)
12: manager.save(orden: OrderEntity PAGADO)
6: orden: OrderEntity (con pago y prendas)
14: vaciarCarrito(usuarioId)
5: findOne({ where: { id: orderId }, relations: [...] })
7: stripe.refunds.create({ payment_intent, reason })
9: manager.save(pago: PaymentEntity REEMBOLSADO)
9: stripe.paymentIntents.retrieve(paymentIntentId)
11: manager.decrement({ id: variantId }, 'stock', cantidad)
10: manager.save(orden: OrderEntity REEMBOLSADO)
11: manager.increment({ id: variantId }, 'stock', cantidad)
4: save(orden: OrderEntity PAGADO)
5: save(pago: PaymentEntity APROBADO)
8: refundStripeResult { id, status: 'succeeded' }()
6: { clientSecret, paymentIntentId }()
10: status: 'succeeded'()
5: stripe.paymentIntents.create({ amount, currency })

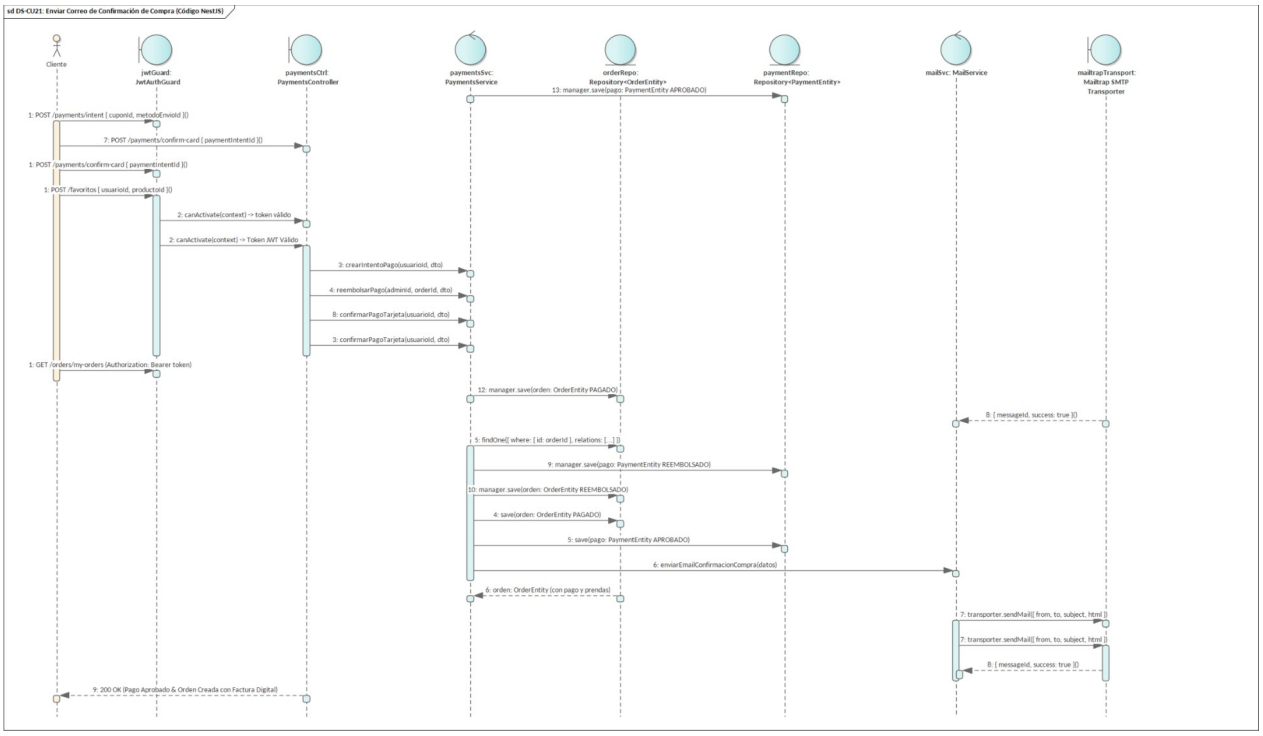

sd DS-CU16: Procesar Reembolso de Pago (Código Nest/N)
adminUser: Administrador
jwtGuard: JwtAuthGuard
roleGuard: RolesGuard
paymentsCtrl: PaymentsController
paymentsSvc: PaymentsService
orderRepo: Repository<OrderEntity>
stripeApi: Stripe SDK / Gateway
paymentRepo: Repository<PaymentEntity>
variantRepo: Repository<ProductVariantEntity>
orderHistoryRepo: Repository<OrderHistoryEntity>
2: canActivate(context) -> Token JWT Válido
1: POST /payments/refund/:orderId { motivo }()
3: crearIntentoPago(usuarioId, dto)
3: checkRoles('ADMIN', 'SUPERADMIN') -> Autorizado
2: canActivate(context) -> Token JWT Válido
2: canActivate(context) -> Token JWT Válido
2: canActivate(context) -> Token JWT Válido
8: confirmarPagoTarjeta(usuarioId, dto)
2: canActivate(context) -> Token JWT Válido
3: confirmarPagoTarjeta(usuarioId, dto)
2: canActivate(context) -> Token JWT Válido
2: canActivate(context) -> token válido
1: POST /admin/reportes-dinamicos/generar { prompt, proveedor }()
1: GET /admin/dashboard/resumen()
1: POST /archivos/subir { categoria: 'productos', imagen }()
1: POST /cupones { codigo, tipo, valor, montoMinimo, fechaInicio, fechaFin }()
1: PUT /shipping/admin/:id/status { estado, numeroTracking, courier }()
2: canActivate(context) -> Token JWT Válido
11: manager.decrement({ id: variantId }, 'stock', cantidad)
6: { clientSecret, paymentIntentId }()
8: refundStripeResult { id, status: 'succeeded' }()
6: orden: OrderEntity (con pago y prendas)
5: save(pago: PaymentEntity APROBADO)
4: save(orden: OrderEntity PAGADO)
12: manager.save(hist: OrderHistoryEntity)
4: reembolsarPago(adminId, orderId, dto)
10: manager.save(orden: OrderEntity REEMBOLSADO)
10: status: 'succeeded'()
9: stripe.paymentIntents.retrieve(paymentIntentId)
9: manager.save(pago: PaymentEntity REEMBOLSADO)
7: stripe.refunds.create({ payment_intent, reason })
5: findOne({ where: { id: orderId }, relations: [...] })
13: manager.save(pago: PaymentEntity APROBADO)
12: manager.save(orden: OrderEntity PAGADO)
5: stripe.paymentIntents.create({ amount, currency })
11: manager.increment({ id: variantId }, 'stock', cantidad)

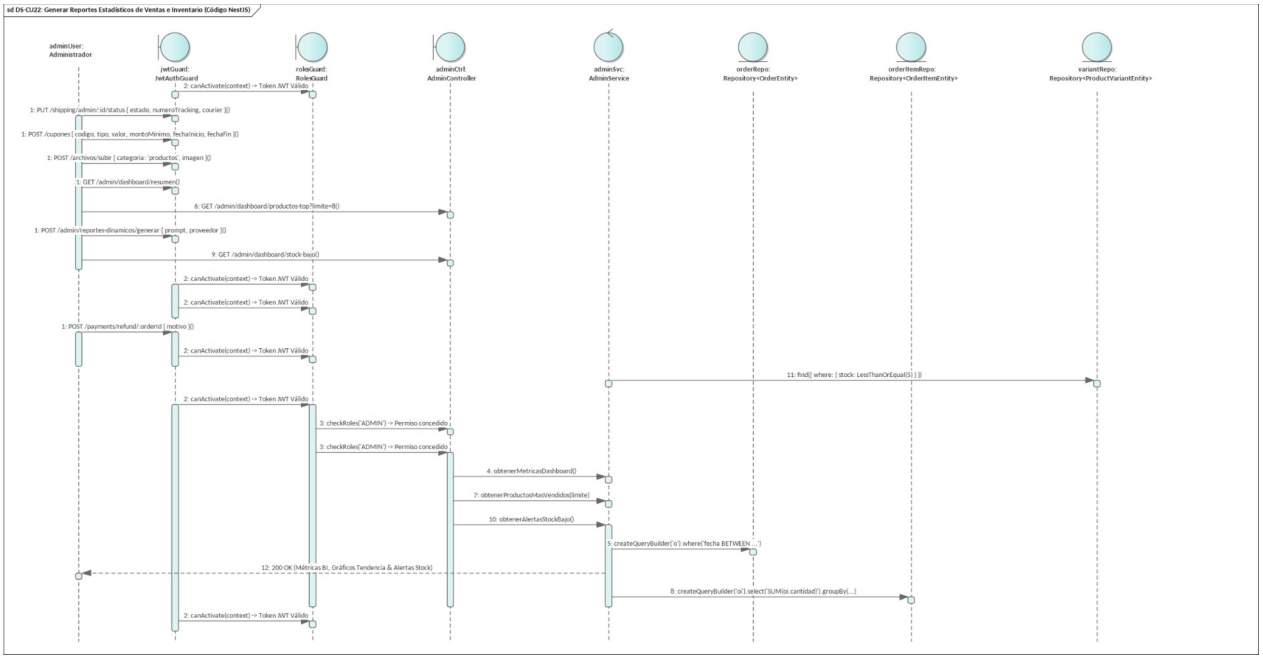

sd DS-CU17: Gestionar Envíos y Rastreo de Despacho (Código NestJS)
adminUser: Administrador
Cliente
jwtGuard: JwtAuthGuard
rolesGuard: RolesGuard
shippingCtrl: ShippingController
shippingSvc: ShippingService
shippingRepo: Repository<ShippingEntity>
shippingMethodRepo: Repository<ShippingMethodEntity>
orderRepo: Repository<OrderEntity>
orderHistoryRepo: Repository<OrderHistoryEntity>
1: PUT /shipping/admin/:id/status { estado, numeroTracking, courier }()
1: POST /cupones { codigo, tipo, valor, montoMinimo, fechaInicio, fechaFin }()
1: POST /archivos/subir { categoria: 'productos', imagen }()
1: GET /admin/dashboard/resumen()
1: POST /admin/reportes-dinamicos/generar { prompt, proveedor }()
1: GET /orders/my-orders (Authorization: Bearer token)
1: POST /payments/intent { cuponId, metodoEnvioId }()
9: GET /shipping/track/:codigo()
1: POST /payments/confirm-card { paymentIntentId }()
1: POST /favoritos { usuarioId, productoId }()
1: POST /payments/refund/:orderId { motivo }()
2: canActivate(context) -> Token JWT Válido
12: 200 OK (Tracking, Etapas 1..4, Coordenadas Leaflet)
3: checkRoles('ADMIN','SUPERADMIN') -> Autorizado
4: actualizarEstadoEnvio(id, dto, adminId)
10: consultarTracking(codigo)
5: findOne({ where: { id } })
6: save(envio: ShippingEntity EN_CAMINO)
7: update(orderId, { estado: 'EN_CAMINO' })
8: save(historial: OrderHistoryEntity)
11: findOne({ where: { numeroTracking: codigo }, relations: [...] })

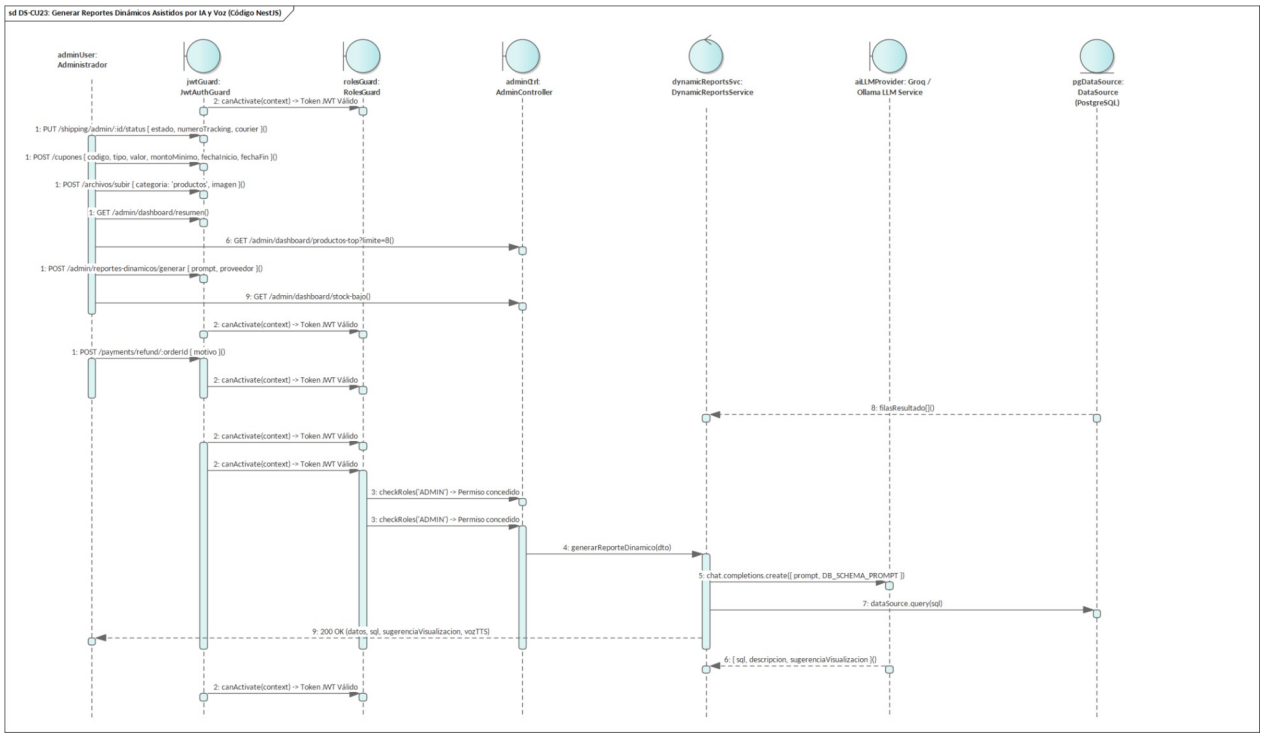

sd DS-CU18: Administrar y Aplicar Cupones de Descuento (Código NestJS)
adminUser: Administrador
Cliente
jwtGuard: JwtAuthGuard
optionalJwtGuard: OptionalJwtAuthGuard
rolesGuard: RolesGuard
couponsCtrl: CouponsController
couponsSvc: CouponsService
couponRepo: Repository<CouponEntity>
couponUsageRepo: Repository<CouponUsageEntity>
2: canActivate(context) -> Token JWT Válido
1: PUT /shipping/admin/:id/status { estado, numeroTracking, courier }()
1: POST /cupones { codigo, tipo, valor, montoMinimo, fechaInicio, fechaFin }()
1: POST /archivos/subir { categoria: 'productos', imagen }()
1: GET /admin/dashboard/resumen()
1: POST /admin/reportes-dinamicos/generar { prompt, proveedor }()
1: GET /orders/my-orders (Authorization: Bearer token)
1: POST /payments/intent { cuponId, metodoEnvioId }()
7: POST /cupones/aplicar { codigo, subtotal }()
1: POST /payments/confirm-card { paymentIntentId }()
1: POST /favoritos { usuarioId, productoId }()
1: POST /payments/refund/:orderId { motivo }()
12: 200 OK (subtotal, descuento, totalFinal, cupon)
8: handleRequest(context) -> usuarioId (opcional)
3: checkRoles('ADMIN') -> Permiso concedido
4: crearCupon(dto)
9: aplicarCupon(dto, usuarioId)
5: findOne({ where: { codigo } })
6: save(nuevoCupon: CouponEntity)
10: findOne({ where: { codigo } })
11: count({ where: { cuponId, usuarioId } })

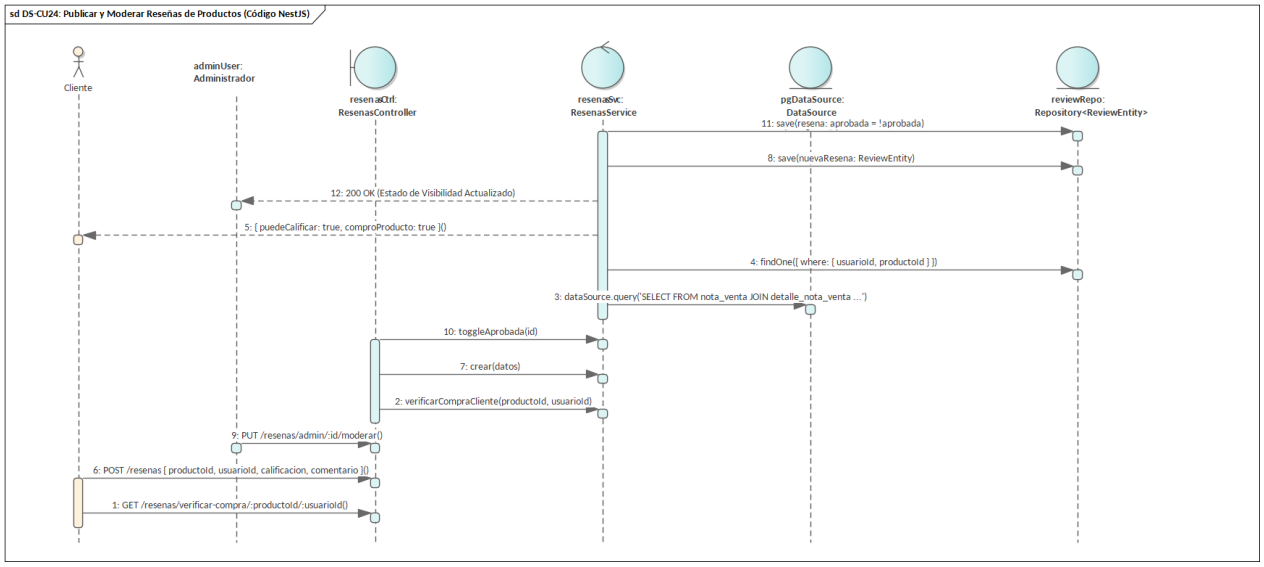

sd DS-CU19: Gestionar y Optimizar Archivos Multimedia (Código NestJS)
adminUser: Administrador
jwtGuard: JwtAuthGuard
rolesGuard: RolesGuard
archivosCtrl: ArchivosController
archivosSvc: ArchivosService
sharpEngine: Sharp / Cloudinary Engine
archivoRepo: Repository<ArchivoEntity>
productImageRepo: Repository<ProductImageEntity>
2: canActivate(context) -> Token JWT Válido
1: PUT /shipping/admin/:id/status [estado, numeroTracking, courier]()
1: POST /cupones [codigo, tipo, valor, montoMinimo, fechaInicio, fechaFin]()
1: POST /archivos/subir [categoria: 'productos', imagen]()
1: GET /admin/dashboard/resumen()
1: POST /admin/reportes-dinamicos/generar [prompt, proveedor]()
1: POST /payments/refund/:orderId [motivo]()
7: [secure_url, public_id, bytes]()
3: checkRoles('ADMIN') -> Permiso concedido
4: procesarSubida(file, req)
5: sharp(buffer).webp({ quality: 80 }).toBuffer()
6: cloudinary.uploader.upload_stream(...)
8: save(archivo: ArchivoEntity)
9: save(imagenProducto: ProductImageEntity)
10: 201 Created (urlWebp, urlThumbnail, tamanioComprimido)

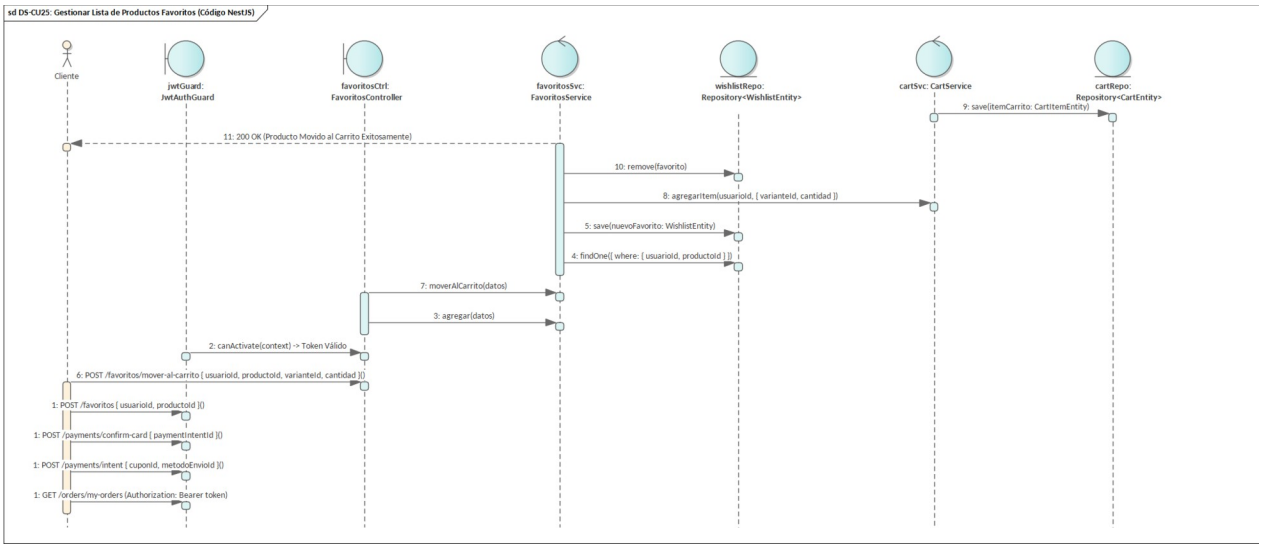

sd DS-CU20: Enviar Correo de Bienvenida y Verificación al Registrarse (Código NestJS)
Cliente
authCtrl: AuthController
authSvc: AuthService
userRepo: Repository<UserEntity>
tokenRepo: Repository<TokenEntity>
mailSvc: MailService
mailtrapTransport: Mailtrap SMTP Transporter
8: [messageId, success: true]()
8: [messageId, success: true]()
7: transporter.sendMail({ from, to, subject, html })
7: transporter.sendMail({ from, to, subject, html })
6: enviarEmailVerificacion(correo, nombre, token)
5: save(tokenVerificacion: TokenEntity)
4: save(nuevoUsuario: UserEntity)
3: findOne({ where: { correo } })
9: 201 Created (Usuario registrado. Revisa tu correo)
2: registro(dto)
1: POST /auth/registro [nombre, apellido, correo, contrasena]()


sd DS-CU21: Enviar Correo de Confirmación de Compra (Código NestJS)
Cliente
jwtGuard: JwtAuthGuard
paymentsCtrl: PaymentsController
paymentsSvc: PaymentsService
orderRepo: Repository<OrderEntity>
paymentRepo: Repository<PaymentEntity>
mailSvc: MailService
mailtrapTransport: Mailtrap SMTP Transporter
13: manager.save(pago: PaymentEntity APROBADO)
1: POST /payments/intent [cuponId, metodoEnvioId]()
7: POST /payments/confirm-card [paymentIntentId]()
1: POST /payments/confirm-card [paymentIntentId]()
1: POST /favoritos [usuarioId, productoId]()
2: canActivate(context) -> token válido
2: canActivate(context) -> Token JWT Válido
3: crearIntentoPago(usuarioId, dto)
4: reembolsarPago(adminId, orderId, dto)
8: confirmarPagoTarjeta(usuarioId, dto)
3: confirmarPagoTarjeta(usuarioId, dto)
1: GET /orders/my-orders (Authorization: Bearer token)
12: manager.save(orden: OrderEntity PAGADO)
8: [messageId, success: true]()
5: findOne({ where: { id: orderId }, relations: [...] })
9: manager.save(pago: PaymentEntity REEMBOLSADO)
10: manager.save(orden: OrderEntity REEMBOLSADO)
4: save(orden: OrderEntity PAGADO)
5: save(pago: PaymentEntity APROBADO)
6: enviarEmailConfirmacionCompra(datos)
6: orden: OrderEntity (con pago y prendas)
7: transporter.sendMail({ from, to, subject, html })
7: transporter.sendMail({ from, to, subject, html })
8: [messageId, success: true]()
9: 200 OK (Pago Aprobado & Orden Creada con Factura Digital)


sd DS-CU22: Generar Reportes Estadísticos de Ventas e Inventario (Código NestJS)
adminUser: Administrador
jwtGuard: JwtAuthGuard
rolesGuard: RolesGuard
adminCtrl: AdminController
adminSvc: AdminService
orderRepo: Repository<OrderEntity>
orderItemRepo: Repository<OrderItemEntity>
variantRepo: Repository<ProductVariantEntity>
2: canActivate(context) -> Token JWT Válido
1: PUT /shipping/admin/:id/status [estado, numeroTracking, courier]()
1: POST /cupones [codigo, tipo, valor, montoMinimo, fechaInicio, fechaFin]()
1: POST /archivos/subir [categoria: 'productos', imagen]()
1: GET /admin/dashboard/resumen()
6: GET /admin/dashboard/productos-top?limite=8()
1: POST /admin/reportes-dinamicos/generar [prompt, proveedor]()
9: GET /admin/dashboard/stock-bajo()
1: POST /payments/refund/:orderId [motivo]()
11: find({ where: { stock: LessThanOrEqual(5) } })
3: checkRoles('ADMIN') -> Permiso concedido
4: obtenerMetricasDashboard()
7: obtenerProductosMasVendidos(limite)
10: obtenerAlertasStockBajo()
5: createQueryBuilder('o').where('fecha BETWEEN ...')
12: 200 OK (Métricas BI, Gráficos Tendencia & Alertas Stock)
8: createQueryBuilder('oi').select('SUM(oi.cantidad)').groupBy(...)


sd DS-CU23: Generar Reportes Dinámicos Asistidos por IA y Voz (Código NestJS)
adminUser: Administrador
jwtGuard: JwtAuthGuard
rolesGuard: RolesGuard
adminCtrl: AdminController
dynamicReportsSvc: DynamicReportsService
aiLLMProvider: Groq / Ollama LLM Service
pgDataSource: DataSource (PostgreSQL)
2: canActivate(context) -> Token JWT Válido
1: PUT /shipping/admin/:id/status { estado, numeroTracking, courier }()
1: POST /cupones { codigo, tipo, valor, montoMinimo, fechaInicio, fechaFin }()
1: POST /archivos/subir { categoria: 'productos', imagen }()
1: GET /admin/dashboard/resumen()
6: GET /admin/dashboard/productos-top?limite=8()
1: POST /admin/reportes-dinamicos/generar { prompt, proveedor }()
9: GET /admin/dashboard/stock-bajo()
1: POST /payments/refund/:orderId { motivo }()
8: filasResultado[]()
3: checkRoles('ADMIN') -> Permiso concedido
4: generarReporteDinamico(dto)
5: chat.completions.create([prompt, DB_SCHEMA_PROMPT])
7: dataSource.query(sql)
9: 200 OK (datos, sql, sugerenciaVisualizacion, vozTTS)
6: { sql, descripcion, sugerenciaVisualizacion }()


sd DS-CU24: Publicar y Moderar Reseñas de Productos (Código NestJS)
Cliente
adminUser: Administrador
resenasCtrl: ResenasController
resenasSvc: ResenasService
pgDataSource: DataSource
reviewRepo: Repository<ReviewEntity>
11: save(resena: aprobada = !aprobada)
8: save(nuevaResena: ReviewEntity)
12: 200 OK (Estado de Visibilidad Actualizado)
5: { puedeCalificar: true, comproProducto: true }()
4: findOne({ where: { usuarioId, productoId } })
3: dataSource.query('SELECT FROM nota_venta JOIN detalle_nota_venta ...')
10: toggleAprobada(id)
7: crear(datos)
2: verificarCompraCliente(productoId, usuarioId)
9: PUT /resenas/admin/:id/moderar()
6: POST /resenas { productoId, usuarioId, calificacion, comentario }()
1: GET /resenas/verificar-compra/:productoId/:usuarioId()


sd DS-CU25: Gestionar Lista de Productos Favoritos (Código NestJS)
Cliente
jwtGuard: JwtAuthGuard
favoritosCtrl: FavoritosController
favoritosSvc: FavoritosService
wishlistRepo: Repository<WishlistEntity>
cartSvc: CartService
cartRepo: Repository<CartEntity>
9: save(itemCarrito: CartItemEntity)
11: 200 OK (Producto Movido al Carrito Exitosamente)
10: remove(favorito)
8: agregarItem(usuarioId, { varianteId, cantidad })
5: save(nuevoFavorito: WishlistEntity)
4: findOne({ where: { usuarioId, productoId } })
7: moverAlCarrito(datos)
3: agregar(datos)
2: canActivate(context) -> Token Válido
6: POST /favoritos/mover-al-carrito { usuarioId, productoId, varianteId, cantidad }()
1: POST /favoritos { usuarioId, productoId }()
1: POST /payments/confirm-card { paymentIntentId }()
1: POST /payments/intent { cuponId, metodoEnvioId }()
1: GET /orders/my-orders (Authorization: Bearer token)

CAPITULO 4: FLUJO DE TRABAJO: IMPLEMENTACION

4.1. Elección de plataforma de desarrollo del software

4.1.1. Framework (NestJS + Vite React)

NestJS v10 en backend (arquitectura modular basada en inyección de dependencias) y React 18 con Vite

en frontend.

4.1.2. Lenguaje de programación (TypeScript / Node.js)

TypeScript 5 estricto en monorepo, Node.js 20 LTS, HTML5 semántico, TailwindCSS v3, Three.js v0.186,

Google MediaPipe Tasks-Vision v1.0.1 y Zustand v4.

4.1.3. Base de datos (PostgreSQL 16)

PostgreSQL 16 ejecutado en contenedor Docker `ecommerce_postgres` bajo propiedades ACID y

extensiones `uuid-ossp`.

4.1.4. Sistema operativo (Linux / Docker)

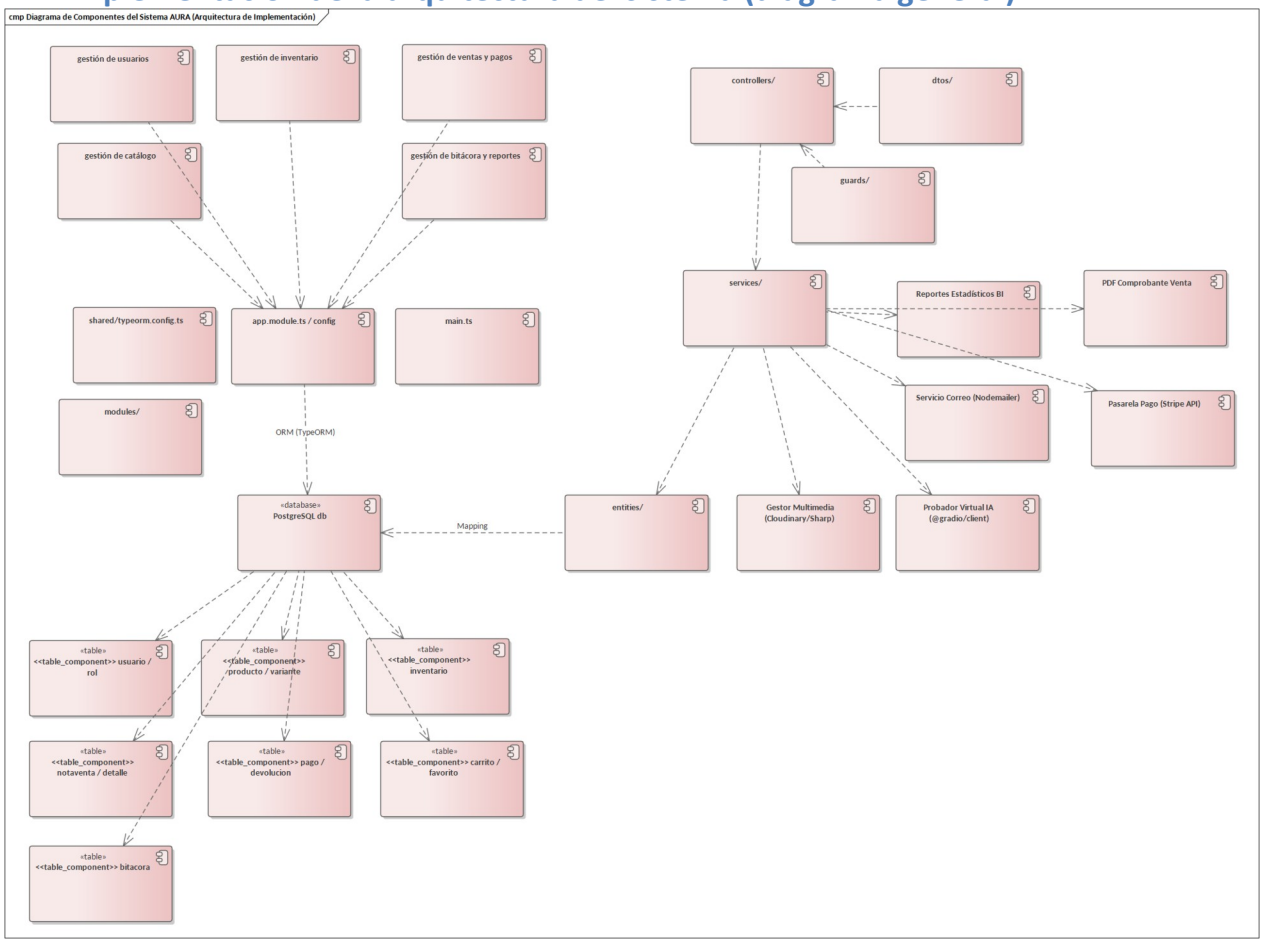
Contenedores Linux Debian/Alpine orquestados con Docker Compose para garantizar portabilidad.

4.1.5. Otros (VS Code, Git + GitHub)

Visual Studio Code como IDE, Git local y repositorio GitHub en `https://github.com/DenilCruz/e-

commerce-ropa.git`.

4.2. Implementación de la arquitectura del sistema (diagrama general)



4.3. Subsistema de Gestión de Usuarios y Autenticación

Codificado en `apps/api/src/modules/auth/` y `apps/api/src/modules/usuarios/` con JWT, Bcrypt y

modelo RBAC.

4.4. Subsistema de Catálogo y Variantes

Codificado en `apps/api/src/modules/productos/` y `apps/api/src/modules/categorias/` con matriz de

tallas, colores y SKUs.

4.5. Subsistema de Probador Virtual 3D y Realidad Aumentada

Codificado en `apps/api/src/modules/probador/` y `apps/web/src/features/carrito/components/` con

Three.js y MediaPipe Pose.

4.6. Subsistema de Ventas, Checkout y Facturación Stripe

Codificado en `apps/api/src/modules/pedidos/` y `apps/api/src/modules/pagos/` con Stripe

PaymentIntents y transacciones ACID.

CAPITULO 5: FLUJO DE TRABAJO: PRUEBAS

5.1. Planificar Pruebas

5.1.1. Objetivo General

Validar el flujo transaccional y la experiencia interactiva completa de AURA, desde la prueba virtual 3D/AR hasta el checkout Stripe.

5.2. Alcance de las Pruebas

- Probador virtual 3D y AR con MediaPipe Pose.
- Persistencia híbrida de la bolsa de compras (carrito local y remoto).
- Procesamiento de pagos seguros con tarjeta bancaria en Stripe y rebaja atómica de existencias.
- Trazabilidad y auditoría de eventos en bitácora.

5.3. Tipos de Pruebas

Tipo de Prueba	Descripción del Alcance	Responsable
Funcionales	Verificar que el catálogo, el probador virtual y el carrito operen conforme a los 25 CUs.	QA / Desarrollador
Integración	Validar la comunicación entre Frontend React, Backend NestJS, Stripe API y PostgreSQL.	QA
Seguridad	Comprobar tokens JWT, cifrado de contraseñas Bcrypt y transacciones seguras PCI-DSS.	Security QA
Usabilidad	Evaluar la fluidez de los controles 3D y la tasa de detección corporal de MediaPipe a 60 fps.	UX / QA
Regresión	Asegurar que la incorporación del probador 3D no afecte el flujo de checkout.	QA

5.4. Caso de Pruebas (Implementar pruebas)

5.4.1. CASO DE PRUEBA 01: Compra y Pago Electrónico con Tarjeta Stripe en AURA

• Prueba de Funcionalidad del Caso de Uso CU15 (Procesar Pago con Stripe):

Descripción: El cliente selecciona un vestido de seda en talla M, lo añade a la bolsa, ingresa su dirección de envío en Santa Cruz, introduce una tarjeta de prueba en Stripe Elements y confirma el pago. El

sistema procesa la transacción, crea la nota de venta, descuenta el stock de la variante y emite el comprobante digital.

ID Caso	CP-001
Prioridad	Crítica
Módulo	Ventas / Pasarela de Pagos Stripe
Caso de Uso Asociado	CU15: Procesar Pago con Stripe (con impacto en CU11 y CU14)

• Datos de Entrada:

- **Usuario:** Camila Miranda (cliente autenticada).
- **Prenda:** Vestido Seda Oliva Gala (Variante `VEST-OLI-M`, ID `v0000000-0002`).
- **Total Liquidado:** 1 unidad ($180.00 USD) + Envío Estándar ($5.00 USD) = Total: $185.00 USD.
- **Método de Pago:** Tarjeta Visa Stripe de prueba `4242 •••• •••• 4242`, Exp: 12/28, CVC: 123.

• Resultado y Verificación:

- Se genera la nota de venta correlativa oficial `NV-2026-0001`.
- El stock de la variante `VEST-OLI-M` se reduce de 8 a 7 unidades en `producto_variante`.
- Stripe retorna `pi_3Pabc1234567890` registrado en tabla `pago` con estado `COMPLETADO`.
- Se asienta el evento de auditoría en la tabla `bitacora`.

5.4.2. CASO DE PRUEBA 02: Simulación y Superposición en Probador Virtual

• Prueba de Funcionalidad del Caso de Uso CU09 (Probar Prendas en Probador Virtual):

Descripción: El usuario abre el modal del probador virtual, inspecciona el modelo tridimensional 360° en WebGL con Three.js y luego activa el modo AR por cámara web para verificar que MediaPipe Pose trace en vivo los 33 landmarks corporales y ajuste la prenda a sus hombros a 60 fps.

ID Caso	CP-002

Prioridad	Alta
Módulo	Probador Virtual Inteligente (3D / Realidad Aumentada)
Caso de Uso Asociado	CU09: Probar Prendas en Probador Virtual 3D y AR

• Resultado y Verificación:

- Three.js inicializa el canvas WebGL renderizando la geometría 3D sin errores de shaders.

- Controles de órbita fluidos para rotar 360°, zoom y paneo.

- MediaPipe Pose detecta hombros y caderas a 60 fps deformando la prenda sobre el cuerpo en vivo.

6. CONCLUSION

La ejecución del proyecto «AURA» ha permitido consolidar de forma práctica e integral los principios de la ingeniería de software moderna bajo la metodología PUDS (Proceso Unificado de Desarrollo de Software), satisfaciendo los 25 Casos de Uso formales planteados para la materia de Sistemas de Información 2.

Se demostró que la combinación de una arquitectura modular desacoplada (NestJS y React bajo Monorepo) junto con un motor de base de datos relacional robusto (PostgreSQL 16) proporciona la solidez requerida para administrar inventarios complejos multivariante (talla/color/SKU) y flujos de pago internacionales seguros con Stripe. Asimismo, la incorporación pionera de herramientas de Inteligencia Artificial (probador virtual 3D con Three.js, tracking corporal con MediaPipe Pose y reportes dinámicos de voz con NLP) eleva la propuesta de valor del software a estándares de clase mundial, resolviendo la incertidumbre del comprador y optimizando la toma de decisiones directivas.

7. RECOMENDACIÓN

- **1. Expansión a Realidad Mixta (WebXR):** Extender el probador virtual hacia tecnologías WebXR que permitan colocar prendas en avatares hiperrealistas mediante cascos de realidad mixta.
- **2. Alta Disponibilidad y Clústeres:** Configurar índices de replicación y balanceadores de carga Nginx para distribuir peticiones entre múltiples réplicas de contenedores NestJS ante eventos de alta demanda como Black Friday.
- **3. Recomendaciones Personalizadas:** Implementar un subsistema de recomendación colaborativa (Machine Learning) que sugiera accesorios y combinaciones de ropa según el historial de compras del cliente.

- **4. Transcripción Local de Voz:** Incorporar modelos locales de transcripción como Whisper (OpenAI) en contenedores dedicados con GPU para reducir la latencia en el procesamiento de voz.

8. BIBLIOGRAFIA

- • Repositorio Oficial en GitHub: https://github.com/DenilCruz/e-commerce-ropa.git

- • URL de la Aplicación en Producción: https://aura-atelier.vercel.app (o servidor local)

Metodología e Ingeniería de Software:

Bibliografía Académica y Referencias Técnicas:

- Jacobson, I., Booch, G., & Rumbaugh, J. (2000). El Proceso Unificado de Desarrollo de Software. Addison-Wesley / Pearson Educación.
- Booch, G., Rumbaugh, J., & Jacobson, I. (2006). El Lenguaje Unificado de Modelado (2da Edición). Pearson Educación.
- Pressman, R. S. (2010). Ingeniería del Software: Un enfoque práctico (7ma Edición). McGraw-Hill Interamericana.
- Sommerville, I. (2011). Ingeniería de Software (9na Edición). Pearson Addison-Wesley.

Frameworks, Motores y Seguridad:

Tecnologías y Estándares Web:

- NestJS Framework Documentation. (2025). Enterprise TypeScript Architecture. https://docs.nestjs.com/

- React Documentation. (2025). The Library for Web and Native User Interfaces. https://react.dev/
- Three.js Manual. (2025). JavaScript 3D Library for WebGL. https://threejs.org/docs/
- Google MediaPipe Tasks-Vision. (2025). Pose Landmark Detection on the Web. https://developers.google.com/mediapipe
- Stripe API Reference. (2025). PaymentIntents & PCI Compliance. https://stripe.com/docs/api
- PostgreSQL Global Development Group. (2025). PostgreSQL 16 Documentation. https://www.postgresql.org/docs/16/
- OWASP Foundation. (2024). OWASP Top 10 Web Application Security Risks. https://owasp.org/Top10/

9. Anexos y QR

• Repositorio de Código Fuente de AURA:

https://github.com/DenilCruz/e-commerce-ropa.git

link de tienda web